

System & Data Design Masterclass

The Complete Guide to Building Enterprise Data Systems

From foundations to production — learn how Netflix, Uber,
Spotify, Airbnb, and Twitter design systems that serve
hundreds of millions of users.

Table of Contents

00	— Introduction: Welcome to System & Data Design
01	— Module 1: Foundations of System Design
02	— Module 2: Data Modeling & Schema Design
03	— Module 3: Storage Systems Deep Dive
04	— Module 4: Batch Processing & ETL
05	— Module 5: Stream Processing & Real-Time Systems
06	— Module 6: Data Warehouse Architecture
07	— Module 7: Data Lakes & Lakehouse Architecture
08	— Module 8: ML Platforms & Feature Stores
09	— Module 9: Event-Driven & Distributed Systems
10	— Module 10: Monitoring, Observability & Data Quality
11	— Module 11: Cost Optimization & Capacity Planning
12	— Module 12: Case Studies — Real-World Systems
13	— Module 13: Interview Preparation & Portfolio Projects

Introduction: Welcome to System & Data Design

What This Course Is — and Who It's For

If you've ever wondered how Netflix serves personalized recommendations to 200 million people in under 100 milliseconds, or how Uber recalculates surge pricing across 10,000 cities every 30 seconds, you're in the right place.

This course is a complete guide to designing data-intensive systems at scale. It's written for software engineers, data engineers, and anyone who wants to understand how modern enterprise systems are built — from the ground up. You don't need prior system design experience. You just need curiosity and a willingness to think about *why* systems are built the way they are, not just *how*.

By the end of this course, you'll be able to:

- Design end-to-end data systems that handle millions of users and petabytes of data
- Choose the right storage, processing, and serving technologies for any use case
- Build reliable, scalable data pipelines that don't break at 3 AM
- Evaluate trade-offs like a senior engineer — because there are always trade-offs
- Ace system design interviews at top technology companies
- Speak confidently about architectures used by Netflix, Uber, Spotify, Airbnb, and Twitter

How to Use This Course

This course is structured as a journey. Each module builds on the ones before it, like floors of a building. Here's the path:

Modules 1-3 lay the foundation: how systems grow, how data is modeled, and how it's stored. These are the concepts everything else rests on.

Modules 4-5 introduce the two fundamental processing paradigms: batch and stream. You'll understand when data can wait to be processed and when it can't.

Modules 6-7 cover where processed data lives for analysis: data warehouses and data lakes, including the modern "lakehouse" that combines the best of both.

Modules 8-9 tackle advanced patterns: ML platforms, feature stores, event-driven architectures, and distributed system design patterns like CQRS and Data Mesh.

Modules 10-11 address the operational reality: monitoring, data quality, cost optimization, and capacity planning — because building a system is only half the job.

Module 12 brings everything together with deep-dive case studies of five real-world systems at massive scale.

Module 13 prepares you for interviews and helps you build a portfolio that demonstrates your skills.

The best way to learn: Read each module, study the code examples, then try the exercises. The code isn't meant to be copied into production — it's meant to teach concepts. When you see a Python snippet, focus on the *pattern* it demonstrates, not the specific syntax.

What is System Design?

Writing code and designing systems are fundamentally different skills. Writing code is like being a bricklayer — you need to know how to lay each brick precisely. Designing systems is like being an architect — you need to decide *what* to build, *where* to put it, and *how* the pieces connect.

System design is the process of defining the architecture, components, and data flow of a system to satisfy a set of requirements.

Here's what makes it interesting: there's almost never one "right" answer. Every design decision involves trade-offs. Choosing a relational database over a document store isn't right or wrong — it depends on your data access patterns, consistency requirements, and team expertise. The mark of a great system designer isn't knowing every technology; it's knowing how to *choose* between them.

Consider a simple example. You're building a social media feed. Seems straightforward — users post content, followers see it. But immediately, questions cascade:

- When a user with 10 million followers posts, do you write that post to 10 million feeds immediately (fan-out on write), or do you assemble each user's feed when they open the app (fan-out on read)?
- How fresh does the feed need to be? Real-time? Within a minute? Within an hour?
- What happens if a server goes down mid-write? Do some followers see the post and others don't?
- How do you rank posts? Chronologically? By relevance? By a mix?

Every one of these questions changes the architecture. That's system design.

Key Takeaway: System design is about making informed decisions under constraints. The goal isn't perfection — it's building something that works reliably for your specific requirements and can evolve as those requirements change.

The System Design Thinking Framework

When faced with any system design challenge — whether in an interview or at work — follow this structured approach. It prevents you from jumping to solutions before understanding the problem.

Step 1: Gather Requirements

Every system exists to solve a business problem. Before drawing a single box on a whiteboard, understand *what* the system needs to do.

Functional requirements are the features: “Users can upload photos,” “The system sends email notifications,” “Analysts can query sales data by region and date.”

Non-functional requirements are the quality attributes: How fast? How reliable? How many users? These often matter more than features. A system that handles 100 users per second needs a very different architecture than one handling 100,000.

Step 2: Estimate Scale with the 4 Vs

The **4 Vs of data** help you quantify the challenge:

- **Volume:** How much data? Gigabytes? Terabytes? Petabytes? This determines your storage strategy.
- **Velocity:** How fast does data arrive? 10 events per second? 10 million? This determines your processing approach.
- **Variety:** What forms does data take? Structured tables? JSON documents? Images? Videos? Log files? This influences your storage and processing choices.
- **Veracity:** How reliable is the data? Is it clean and validated, or messy and inconsistent? This drives your data quality strategy.

A system processing 1 GB of clean, structured data per day is a fundamentally different challenge than one processing 1 TB of messy, semi-structured data per second.

Step 3: Design Top-Down

Start with the big picture, then zoom in:

1. **High-level design:** What are the major components? How does data flow between them?
2. **Component design:** How does each component work internally? What technology does it use?
3. **Detailed design:** What are the data models, APIs, algorithms, and configurations?

This top-down approach prevents you from getting lost in details before the overall picture is clear.

Step 4: Analyze Trade-offs

Every design decision involves giving something up to get something else. The key trade-offs you'll encounter throughout this course:

TRADE-OFF	YOU GET	YOU GIVE UP
Consistency vs. Availability	All users see the same data	System may be unavailable during failures
Latency vs. Throughput	Fast individual responses	Lower total processing capacity
Simplicity vs. Flexibility	Easy to understand and maintain	Harder to adapt to new requirements
Cost vs. Performance	Lower cloud bills	Slower queries or higher latency

There is no universal “right” side. The answer depends on your specific requirements.

Key Takeaway: Follow the framework — Requirements → Scale Estimation → High-Level Design → Component Design → Trade-offs. This structured approach works for interview questions, architecture reviews, and real production systems.

The Building Blocks

Every complex system is built from a small set of fundamental components. Think of them as LEGO bricks — individually simple, but capable of creating anything when combined thoughtfully. Here's your vocabulary for the rest of the course:

Databases store your data persistently. Relational databases (PostgreSQL, MySQL) organize data in tables with strict schemas. NoSQL databases (MongoDB, Cassandra, Redis) trade some of that structure for flexibility or performance. We'll spend entire modules on these.

Caches store frequently accessed data in memory for fast retrieval. Instead of querying your database for the same popular product page 10,000 times per second, you query the cache — which responds in microseconds instead of milliseconds. Redis and Memcached are the most common.

Message Queues and Event Streams decouple producers from consumers. When a user places an order, instead of your order service directly calling the inventory service, the email service, and the analytics service, it publishes an “order placed” event to a queue. Each downstream service picks it up independently. Apache Kafka is the dominant technology here.

Load Balancers distribute incoming traffic across multiple servers. When one server isn't enough, you add more and put a load balancer in front. Requests are distributed across servers so no single one is overwhelmed.

CDNs (Content Delivery Networks) cache static content (images, CSS, JavaScript) at servers geographically close to users. A user in Tokyo shouldn't wait for data to travel from a server in Virginia.

APIs (Application Programming Interfaces) are the contracts between services. They define how one component communicates with another — what requests it accepts and what responses it returns. REST, GraphQL, and gRPC are the three main styles.

Blob/Object Storage (S3, GCS) stores large, unstructured data — files, images, videos, backups, data lake contents. It's extremely cheap and virtually unlimited in capacity.

A Simple End-to-End Example: The URL Shortener

Let's tie these building blocks together by designing something concrete: a URL shortener like bit.ly. This is deliberately simple — the goal is to see how the pieces connect.

Requirements

Functional: Users submit a long URL and get back a short one. When someone visits the short URL, they're redirected to the original. Users can optionally see how many times their link was clicked.

Non-functional: The system handles 1,000 new URLs per day and 100,000 redirects per day. Redirects must be fast (under 50ms). Short URLs should work forever (high durability). The system should be available 99.9% of the time.

Scale Estimation

- **Write volume:** 1,000 URLs/day $\approx$ 0.01 writes/second. Tiny.
- **Read volume:** 100,000 redirects/day $\approx$ 1.2 reads/second. Still small, but 100x the writes. This is a read-heavy system.
- **Storage:** Each URL record is ~500 bytes. Over 10 years: $500 \text{ bytes} \times 1,000/\text{day} \times 365 \times 10 \approx 1.8 \text{ GB}$. A single database can handle this easily.

High-Level Architecture

User → Load Balancer → Web Server → Cache (Redis) → Database (PostgreSQL)
↓
Analytics Store

1. User submits a long URL to the **Web Server** via the API
2. Server generates a short code, stores the mapping in **PostgreSQL**, and returns the short URL
3. When someone visits the short URL, the server checks **Redis** first (cache hit = fast redirect)
4. On cache miss, it queries PostgreSQL, caches the result, and redirects
5. Every redirect increments a counter in the **Analytics Store**

Implementation

The following Python code implements the core of this system using Flask. It demonstrates how a web server, database, and cache work together. Notice how each component has a clear responsibility:

```

import hashlib
import time
from flask import Flask, redirect, request, jsonify
import redis
import psycopg2

app = Flask(__name__)

# Redis cache for fast URL lookups (the "fast path")
cache = redis.Redis(host='localhost', port=6379, db=0, decode_responses=True)

# PostgreSQL for durable storage (the "source of truth")
db = psycopg2.connect(
    host='localhost', dbname='urlshortener', user='app', password='secret'
)

def generate_short_code(url: str) -> str:
    """Generate a 7-character short code from the URL.

    We hash the URL + timestamp to avoid collisions, then take the
    first 7 characters of the base62 encoding. This gives us  $62^7 \approx$ 
    3.5 trillion possible codes – plenty for our scale.
    """
    hash_input = f"{url}{time.time()}".encode()
    hash_hex = hashlib.sha256(hash_input).hexdigest()
    # Convert first 10 hex chars to base62 (simplified)
    chars = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"
    num = int(hash_hex[:10], 16)
    code = ""
    while len(code) < 7:
        code += chars[num % 62]
        num //= 62
    return code

@app.route('/shorten', methods=['POST'])
def shorten_url():
    """Create a new short URL.

    Writes to both the database (durability) and cache (performance).
    The database is our source of truth; the cache is an optimization.
    """
    long_url = request.json['url']
    short_code = generate_short_code(long_url)

    # Store in PostgreSQL (durable)
    cursor = db.cursor()
    cursor.execute(
        "INSERT INTO urls (short_code, long_url, created_at, click_count) "
        "VALUES (%s, %s, NOW(), 0)",
        (short_code, long_url)
    )
    db.commit()

    # Cache in Redis (fast reads) with 24-hour expiry

```

```

cache.setex(f"url:{short_code}", 86400, long_url)

return jsonify({
    'short_url': f'https://short.ly/{short_code}',
    'long_url': long_url
})

@app.route('/<short_code>')
def redirect_url(short_code: str):
    """Redirect a short URL to its original destination.

    This is the hot path – it handles 100x more traffic than creation.
    We check the cache first (microseconds), falling back to the
    database (milliseconds) only on cache miss.
    """
    # Try cache first (fast path)
    long_url = cache.get(f"url:{short_code}")

    if not long_url:
        # Cache miss – query database (slow path)
        cursor = db.cursor()
        cursor.execute(
            "SELECT long_url FROM urls WHERE short_code = %s",
            (short_code,)
        )
        result = cursor.fetchone()
        if not result:
            return jsonify({'error': 'URL not found'}), 404

        long_url = result[0]
        # Populate cache for next time
        cache.setex(f"url:{short_code}", 86400, long_url)

    # Increment click counter asynchronously
    # (In production, you'd publish to a message queue instead)
    cache.incr(f"clicks:{short_code}")

    return redirect(long_url, code=302)

```

Notice a few things about this code:

- **The cache is an optimization, not a requirement.** If Redis goes down, the system still works — it's just slower because every request hits PostgreSQL.
- **We write to both cache and database on creation**, so the first read is always a cache hit. This is called “write-through caching.”
- **Click counting uses Redis `INCR`** for speed, but in a production system you'd publish click events to a message queue (like Kafka) for reliable analytics processing.
- **The 302 redirect** (temporary) is intentional — it ensures browsers always come back to our server, so we can track clicks. A 301 (permanent) would let browsers cache the redirect and skip us.

Key Takeaway: Even a simple system like a URL shortener involves multiple components (web server, database, cache), clear data flow, and deliberate trade-off decisions. Every concept in this example — caching, read-heavy optimization, write-through patterns, async analytics — scales up to the massive systems we'll study in later modules.

What Comes Next

In Module 1, we'll zoom in on the foundations: how systems evolve from a single server to a distributed architecture, the fundamentals of networking and APIs, and the critical skill of thinking in trade-offs. From there, we'll progressively build your understanding — data modeling, storage engines, batch and stream processing, warehouses and lakehouses, ML platforms, and distributed systems — until you can design systems like the ones powering the world's most demanding applications.

Let's get started.

Module 1: Foundations of System Design

How Systems Grow: From Monolith to Distributed

Every large-scale system started as a small one. Understanding how systems evolve helps you recognize *why* certain architectural patterns exist — they're not arbitrary; each one solves a specific problem that appears at a specific stage of growth.

Stage 1: The Single Server

Every application starts here. Your web server, application code, and database all live on one machine. It's simple, easy to deploy, and easy to debug. For a side project or an app with a few hundred users, this is perfectly fine.

```
[Users] → [Single Server: Web + App + Database]
```

Stage 2: Separate the Database

Your first scaling bottleneck is almost always the database. Your application and database compete for the same CPU, memory, and disk I/O. Separating them onto different machines lets each be optimized independently — your app server can have lots of CPU and RAM, while your database server gets fast SSDs and extra memory for caching.

```
[Users] → [App Server] → [Database Server]
```

This is also where **backups** become critical. A single server failure shouldn't lose your data. You add database replication — a primary that handles writes and one or more replicas that handle reads.

Stage 3: Add a Cache

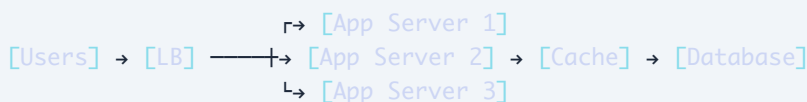
As traffic grows, you notice your database is doing the same work over and over. The homepage shows the same trending products to every user. The user profile page loads the same data on every visit. A cache (Redis or Memcached) stores these frequently accessed results in memory, where lookups take microseconds instead of the milliseconds a database query requires.

```
[Users] → [App Server] → [Cache (Redis)] → [Database]
```

Key Takeaway: Caching is the single most effective performance optimization in most systems. A well-designed cache can reduce database load by 80-90%. But caching introduces a new problem: cache invalidation — making sure the cache doesn't serve stale data. As Phil Karlton famously said, "There are only two hard things in Computer Science: cache invalidation and naming things."

Stage 4: Horizontal Scaling and Load Balancing

One app server can handle perhaps 1,000-5,000 concurrent connections. When you need more, you add more servers and put a **load balancer** in front of them. The load balancer distributes incoming requests across your fleet of servers.



This is **horizontal scaling** (adding more machines), as opposed to **vertical scaling** (getting a bigger machine). Horizontal scaling has a key advantage: it has no theoretical limit. You can always add another server. Vertical scaling hits a ceiling — there's only so much RAM and CPU you can put in one box.

But horizontal scaling introduces a challenge: your app servers must be **stateless**. If user session data lives on Server 1, and the load balancer sends the next request to Server 2, the session is lost. Solution: store session state in the cache or database, not on the app server.

Stage 5: CDN and Global Distribution

For users far from your servers, network latency dominates response time. A request from Tokyo to a server in Virginia takes ~150ms just for the round trip — before your server even starts processing. A **CDN** (Content Delivery Network) caches static assets (images, CSS, JavaScript) at edge servers worldwide, so users load them from a nearby location.

Stage 6: Message Queues and Async Processing

Not everything needs an immediate response. When a user places an order, they need instant confirmation — but sending the receipt email, updating inventory, and notifying the warehouse can happen asynchronously. A **message queue** (Kafka, RabbitMQ, SQS) decouples these operations. The order service publishes an "order placed" event, and downstream services process it at their own pace.

This is where the transition from monolith to **distributed system** becomes real. You now have multiple independent services communicating through events rather than direct function calls.

Reliability, Scalability, and Maintainability

These three properties define a well-designed system. They're the lens through which every architecture decision should be evaluated.

Reliability: It Keeps Working When Things Go Wrong

Things *will* go wrong. Hard drives fail. Networks partition. Developers deploy bugs. A reliable system continues to function correctly even when components fail.

Hardware faults are solved with redundancy: RAID arrays for disks, dual power supplies, database replicas, multi-region deployment. The key insight is that hardware faults are *random and independent* — two hard drives rarely fail at the same moment, so having two copies of your data makes loss extremely unlikely.

Software faults are harder because they're *correlated* — a bug in your code affects every server running that code simultaneously. These are mitigated through testing, gradual rollouts (deploy to 1% of servers first), monitoring, and the ability to quickly roll back.

Human errors are the most common cause of outages. Mitigate them with good abstractions that make it easy to “do the right thing,” sandbox environments for experimentation, thorough testing at all levels, easy rollback mechanisms, and clear monitoring with alerting.

Scalability: It Handles Growth

Scalability isn't a binary yes/no — it's about answering specific questions: “If traffic doubles, what happens to response time?” or “If data volume grows 10x, what needs to change?”

Describing load requires picking the right metrics for your system. For a web app, it might be requests per second. For a database, it might be the read-to-write ratio. For a data pipeline, it might be events per second or data volume per day.

Describing performance means understanding percentiles. The *average* response time is almost useless — it hides the experience of your slowest users, who are often your most important (they may have the most data or the most complex queries).

The following Python code shows how to track and report p50, p95, and p99 latency in an application. These percentiles tell you: “50% of requests complete in X ms, 95% in Y ms, and 99% in Z ms.” The p99 is especially important because it represents the experience of your most affected users:

```

import time
import statistics
from collections import deque
from functools import wraps

class LatencyTracker:
    """Track request latencies and report percentiles.

    Uses a sliding window of the most recent 10,000 measurements
    to keep memory bounded while still capturing current behavior.
    """

    def __init__(self, window_size: int = 10_000):
        self.latencies = deque(maxlen=window_size)

    def record(self, latency_ms: float):
        self.latencies.append(latency_ms)

    def get_percentiles(self) -> dict:
        if not self.latencies:
            return {}
        sorted_latencies = sorted(self.latencies)
        n = len(sorted_latencies)
        return {
            'p50': sorted_latencies[int(n * 0.50)],
            'p95': sorted_latencies[int(n * 0.95)],
            'p99': sorted_latencies[int(n * 0.99)],
            'max': sorted_latencies[-1],
            'count': n,
        }

    def report(self):
        p = self.get_percentiles()
        if p:
            print(
                f"Latency: p50={p['p50']:.1f}ms, "
                f"p95={p['p95']:.1f}ms, "
                f"p99={p['p99']:.1f}ms, "
                f"max={p['max']:.1f}ms "
                f"(n={p['count']})"
            )

# Use as a decorator to automatically track any function's latency
tracker = LatencyTracker()

def track_latency(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.monotonic()
        result = func(*args, **kwargs)
        elapsed_ms = (time.monotonic() - start) * 1000
        tracker.record(elapsed_ms)
        return result
    return wrapper

```

In production, you'd integrate this with Prometheus, Datadog, or a similar monitoring system. The key insight is that **you should always monitor percentiles, not averages**.

Maintainability: Future Engineers Can Work With It

The majority of software cost is not in initial development — it's in ongoing maintenance: fixing bugs, adapting to new requirements, adding features, and keeping it running. Maintainability has three facets:

- **Operability:** Making life easy for the operations team. Good monitoring, clear runbooks, predictable behavior.
- **Simplicity:** Removing accidental complexity. The system should be as simple as possible for what it does — but no simpler.
- **Evolvability:** Making it easy to change. Requirements *will* change. The system should accommodate new features without a rewrite.

Key Takeaway: When evaluating any design, ask: “Is it reliable? Does it scale? Can my team maintain it?” If any answer is no, reconsider.

Networking Fundamentals for System Designers

You don't need to be a networking expert, but you need to understand the basics — because network behavior drives many system design decisions.

What Happens When You Type a URL

1. **DNS Resolution:** Your browser asks a DNS server to translate “example.com” into an IP address like 93.184.216.34. This might involve multiple DNS servers (recursive resolver → root → TLD → authoritative).
2. **TCP Connection:** Your browser opens a TCP connection to that IP address. This involves a three-way handshake (SYN → SYN-ACK → ACK) — that's 1.5 round trips before any data flows.
3. **TLS Handshake** (for HTTPS): Another 1-2 round trips to establish encryption.
4. **HTTP Request:** Your browser sends the actual request (GET /page HTTP/1.1).
5. **Server Processing:** The server runs your application logic, queries databases, etc.
6. **HTTP Response:** The server sends back HTML, which triggers further requests for CSS, JavaScript, and images.

Latency Numbers Every Engineer Should Know

These approximate numbers shape system design decisions. Memorize the order of magnitude:

OPERATION	TIME
L1 cache reference	1 ns
L2 cache reference	4 ns
RAM reference	100 ns
SSD random read	16 μ s
HDD random read	2 ms
Network round trip (same datacenter)	0.5 ms
Network round trip (cross-continent)	150 ms

The key insight: **memory is 100,000x faster than disk, and disk within the same datacenter is 300x faster than a cross-continent network call.** This is why caches (in-memory) are so effective, and why globally distributed systems are so challenging.

TCP vs UDP

TCP guarantees delivery and ordering. Every packet is acknowledged; lost packets are retransmitted. Use it when correctness matters: web requests, database queries, file transfers.

UDP is fire-and-forget. Faster and lighter, but packets can be lost or arrive out of order. Use it when speed matters more than completeness: video streaming, real-time gaming, DNS lookups.

APIs: The Contracts Between Systems

APIs define how services communicate. Choosing the right API style affects performance, developer experience, and system evolution.

REST

The most common style. Uses HTTP methods (GET, POST, PUT, DELETE) on resource URLs. Stateless, cacheable, well-understood. Great for CRUD operations and public-facing APIs.

GraphQL

Lets clients request exactly the data they need — no more, no less. Solves the over-fetching problem (REST endpoint returns 50 fields when you need 3) and under-fetching problem (you need data from 3 REST endpoints). Best for complex UIs with varying data needs.

gRPC

Uses Protocol Buffers for binary serialization and HTTP/2 for transport. Extremely fast, supports streaming. Best for internal service-to-service communication where performance matters and you control both endpoints.

Here's a practical REST API in FastAPI. Notice how the framework handles validation, serialization, and documentation automatically — this is what good abstractions look like:

```

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from typing import Optional
from datetime import datetime

app = FastAPI(title="Product API")

class Product(BaseModel):
    id: Optional[int] = None
    name: str
    price: float
    category: str
    in_stock: bool = True
    created_at: Optional[datetime] = None

# In-memory store (a real app would use a database)
products = {}
next_id = 1

@app.post("/products", response_model=Product, status_code=201)
def create_product(product: Product):
    """Create a new product.

    FastAPI automatically validates the request body against the
    Product model. If 'name' is missing or 'price' isn't a number,
    the client gets a clear 422 error – no manual validation needed.
    """
    global next_id
    product.id = next_id
    product.created_at = datetime.utcnow()
    products[next_id] = product
    next_id += 1
    return product

@app.get("/products/{product_id}", response_model=Product)
def get_product(product_id: int):
    """Retrieve a product by ID.

    Returns 404 if the product doesn't exist. The response_model
    parameter ensures the response matches the Product schema,
    even if the internal representation has extra fields.
    """
    if product_id not in products:
        raise HTTPException(status_code=404, detail="Product not found")
    return products[product_id]

```

Key Takeaway: Choose REST for simplicity and broad compatibility, GraphQL when clients need flexibility over what data they fetch, and gRPC for high-performance internal service communication.

Databases: A First Look

We'll devote entire modules to storage systems, but here's the vocabulary you need going forward.

SQL vs NoSQL — Different Tools, Not a Religious War

Relational databases (PostgreSQL, MySQL) store data in tables with predefined schemas. They enforce relationships between tables through foreign keys and guarantee ACID properties: **A**tomicity (transactions are all-or-nothing), **C**onsistency (data always satisfies constraints), **I**solation (concurrent transactions don't interfere), **D**urability (committed data survives crashes).

NoSQL databases is an umbrella term for everything else: - **Key-value stores** (Redis, DynamoDB): Simple lookup by key. Blazingly fast. Good for caches, sessions, leaderboards. - **Document stores** (MongoDB, Couchbase): Store JSON-like documents. Flexible schemas. Good when data is naturally hierarchical. - **Wide-column stores** (Cassandra, HBase): Store data in column families. Excellent write throughput and horizontal scaling. Good for time-series and IoT data. - **Graph databases** (Neo4j, Neptune): Store nodes and edges. Optimized for traversing relationships. Good for social networks, recommendation engines, fraud detection.

The CAP Theorem

The CAP theorem states that a distributed system can provide at most two of three guarantees simultaneously:

- **Consistency**: Every read returns the most recent write
- **Availability**: Every request receives a response
- **Partition tolerance**: The system continues to operate despite network failures

Since network partitions *will* happen in any distributed system, the real choice is between **CP** (consistent but sometimes unavailable — e.g., most relational databases) and **AP** (available but sometimes inconsistent — e.g., Cassandra, DynamoDB).

In practice, this means: if you need strong consistency (financial transactions, inventory counts), lean toward CP systems. If you need high availability and can tolerate brief inconsistency (social media feeds, analytics counters), lean toward AP systems.

Key Takeaway: Don't choose a database because it's trendy. Choose it based on your access patterns (reads vs writes, point lookups vs range scans, simple queries vs complex joins), consistency requirements, and scale needs. When in doubt, start with PostgreSQL — it's remarkably capable and well-understood.

Thinking in Trade-offs

The single most important skill in system design is recognizing and evaluating trade-offs. There is no free lunch. Every optimization creates a cost somewhere else.

Consistency vs. Availability: Strongly consistent systems must coordinate between nodes, which adds latency and creates failure modes. Eventually consistent systems respond faster but may return stale data. The right choice depends on what your users can tolerate — a bank balance should always be consistent; a social media “like” count can lag by a few seconds.

Latency vs. Throughput: You can process requests one at a time for minimum latency, or batch them together for maximum throughput. A real-time fraud detection system optimizes for latency (each transaction needs a fast answer). A data warehouse query optimizes for throughput (scan a billion rows efficiently).

Cost vs. Performance: You can always make a system faster by throwing more hardware at it. The question is whether the improvement justifies the cost. A 10x faster query that costs 100x more per month isn’t a good trade-off unless that query is business-critical.

Simplicity vs. Flexibility: A simple system is easier to understand, debug, and maintain — but may not handle future requirements. A flexible system can adapt to anything — but may be over-engineered for what you actually need today. The sweet spot: build for what you know you need, with clean abstractions that allow extension.

Build vs. Buy: Building custom infrastructure gives you control and avoids vendor lock-in. Buying (or using managed services) saves engineering time and operational burden. Most teams should buy unless they have a genuinely unique requirement.

When evaluating trade-offs, ask yourself: 1. What are we optimizing for? (Speed? Cost? Reliability? Developer velocity?) 2. What’s the cost of getting this wrong? 3. Is this decision reversible? (If yes, move fast. If no, deliberate carefully.)

Key Takeaway: Great system designers don’t have perfect knowledge of every technology. They have great judgment about trade-offs. Practice articulating “I chose X over Y because Z” — this is the skill that separates senior engineers from everyone else.

What’s Next

With these foundations in place, you’re ready to dive into the first deep technical topic: how to model data effectively. In Module 2, we’ll explore relational modeling, dimensional modeling,

document and graph models, and event-driven schemas — the building blocks for every storage and processing decision that follows.

Module 2: Data Modeling & Schema Design

Why Data Modeling Matters

Bad data models are the number one cause of system redesigns. Not server outages, not scaling problems, not technology choices — *how you structure your data* determines whether a system is a joy to work with or a nightmare that gets slower and more confusing with every new feature.

Think of data modeling like designing the floor plan of a building. Get it right, and every room connects logically — people flow naturally from the lobby to the elevator to their office. Get it wrong, and you end up with hallways that dead-end, bathrooms accessible only through the kitchen, and a conference room that requires walking through someone's office. You *can* work in that building, but everything takes longer and feels wrong.

The good news: there isn't one "right" data model for all situations. Different parts of the same system might use different modeling approaches. The key is matching the model to the access patterns — how will the data actually be read and written?

Key Takeaway: Before choosing any technology or writing any SQL, understand your access patterns. Are reads or writes more frequent? Do you need complex joins or simple lookups? Is the data naturally tabular, hierarchical, or graph-structured? The answers to these questions determine your model.

Relational Modeling Fundamentals

Relational databases have dominated data storage for 40+ years because the relational model is remarkably versatile. Let's build up the core concepts using a concrete example: an e-commerce order system.

Tables, Rows, and Relationships

A relational database organizes data into **tables** (relations). Each table has a fixed set of **columns** (attributes) and contains **rows** (tuples). A **primary key** uniquely identifies each row. A **foreign key** references a row in another table, creating a relationship.

Normalization: Eliminating Redundancy

Normalization is the process of organizing data to reduce redundancy and prevent update anomalies. There are several “normal forms,” but in practice, you mostly care about the first three.

First Normal Form (1NF): Every column holds a single, atomic value. No arrays, no comma-separated lists, no nested structures.

Here’s a table that violates 1NF — the `phone_numbers` column holds multiple values:

```
-- BAD: Violates 1NF - multi-valued column
CREATE TABLE customers_bad (
  customer_id SERIAL PRIMARY KEY,
  name        TEXT NOT NULL,
  phone_numbers TEXT -- Stores "555-0100, 555-0101"
);
```

The fix is to move phone numbers to their own table, with one row per phone number:

```
-- GOOD: 1NF - each value gets its own row
CREATE TABLE customers (
  customer_id SERIAL PRIMARY KEY,
  name        TEXT NOT NULL
);

CREATE TABLE customer_phones (
  phone_id     SERIAL PRIMARY KEY,
  customer_id  INTEGER REFERENCES customers(customer_id),
  phone_number TEXT NOT NULL,
  phone_type   TEXT DEFAULT 'mobile' -- mobile, home, work
);
```

Second Normal Form (2NF): Every non-key column depends on the *entire* primary key, not just part of it. This only matters for tables with composite primary keys.

Third Normal Form (3NF): Every non-key column depends *directly* on the primary key, not on another non-key column. For example, if your `orders` table has both `customer_id` and `customer_name`, the name depends on the customer, not the order. It should live only in the `customers` table.

Here’s a properly normalized e-commerce schema. Notice how each piece of information lives in exactly one place:

```

-- Customers: one row per customer
CREATE TABLE customers (
  customer_id SERIAL PRIMARY KEY,
  email       TEXT UNIQUE NOT NULL,
  name        TEXT NOT NULL,
  created_at  TIMESTAMP DEFAULT NOW()
);

-- Products: one row per product
CREATE TABLE products (
  product_id SERIAL PRIMARY KEY,
  name        TEXT NOT NULL,
  category    TEXT NOT NULL,
  price       DECIMAL(10,2) NOT NULL,
  description  TEXT
);

-- Orders: one row per order (header information)
CREATE TABLE orders (
  order_id    SERIAL PRIMARY KEY,
  customer_id INTEGER REFERENCES customers(customer_id),
  order_date  TIMESTAMP DEFAULT NOW(),
  status      TEXT DEFAULT 'pending',
  shipping_addr TEXT NOT NULL
);

-- Order items: one row per product per order (the line items)
CREATE TABLE order_items (
  item_id     SERIAL PRIMARY KEY,
  order_id    INTEGER REFERENCES orders(order_id),
  product_id  INTEGER REFERENCES products(product_id),
  quantity    INTEGER NOT NULL CHECK (quantity > 0),
  unit_price  DECIMAL(10,2) NOT NULL -- Price at time of purchase
);

```

Notice that `unit_price` is stored in `order_items` even though `products` has a `price` column. This is intentional — the product price might change, but the price a customer paid should never change. This is called “snapshotting” and it’s a common pattern.

When to Denormalize

Normalization optimizes for *write correctness* — no data is duplicated, so updates happen in one place. But it can hurt *read performance* because answering questions requires joining multiple tables.

Consider this query: “Show me all orders with customer names and product names.” In our normalized schema, that’s a four-table join:

```

SELECT c.name AS customer_name,
       o.order_id,
       o.order_date,
       p.name AS product_name,
       oi.quantity,
       oi.unit_price
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
WHERE o.order_date > CURRENT_DATE - INTERVAL '30 days';

```

At small scale, this is fine. At millions of orders, these joins become expensive. **Denormalization** trades storage space and write complexity for read performance by duplicating data:

```

-- Denormalized order view - all info in one table
CREATE TABLE order_details_flat (
  order_id      INTEGER,
  order_date    TIMESTAMP,
  customer_id   INTEGER,
  customer_name TEXT,      -- Duplicated from customers
  customer_email TEXT,     -- Duplicated from customers
  product_id    INTEGER,
  product_name  TEXT,      -- Duplicated from products
  category      TEXT,      -- Duplicated from products
  quantity      INTEGER,
  unit_price    DECIMAL(10,2),
  line_total    DECIMAL(10,2) -- Pre-computed
);

```

Now the same query is a simple scan of one table — no joins needed. The trade-off: if a customer changes their name, you need to update every row where they appear. In an analytics context where you're reading historical data, this is usually acceptable.

Key Takeaway: Normalize your operational/transactional data (OLTP) for correctness. Denormalize your analytical data (OLAP) for read performance. Most systems need both — a normalized source of truth and denormalized views for queries.

Dimensional Modeling: Star and Snowflake Schemas

When your data is primarily used for analytics — dashboards, reports, ad-hoc queries — **dimensional modeling** provides a structure optimized for those patterns.

Fact Tables and Dimension Tables

A **fact table** records business events (measurements). Each row is something that happened: an order was placed, a page was viewed, a payment was processed. Fact tables are tall (many rows) and narrow-ish (mostly foreign keys and numeric measures).

A **dimension table** provides context for facts. Each row describes a business entity: a customer, a product, a date, a location. Dimension tables are short (fewer rows) and wide (many descriptive columns).

Star Schema

In a star schema, the fact table sits in the center, connected to dimension tables radiating outward like points of a star. There's exactly one join between the fact table and any dimension.

Here's a star schema for retail analytics:

```

-- Fact table: one row per order line item
CREATE TABLE fact_sales (
    sale_id          SERIAL PRIMARY KEY,
    date_key          INTEGER REFERENCES dim_date(date_key),
    customer_key      INTEGER REFERENCES dim_customer(customer_key),
    product_key       INTEGER REFERENCES dim_product(product_key),
    store_key         INTEGER REFERENCES dim_store(store_key),
    -- Measures (the numbers you aggregate)
    quantity          INTEGER,
    unit_price        DECIMAL(10,2),
    total_amount      DECIMAL(10,2),
    discount          DECIMAL(10,2),
    profit            DECIMAL(10,2)
);

-- Date dimension: one row per day, pre-populated
CREATE TABLE dim_date (
    date_key          INTEGER PRIMARY KEY,    -- e.g., 20260401
    full_date         DATE,
    day_of_week       TEXT,                  -- Monday, Tuesday, ...
    month_name        TEXT,
    quarter           INTEGER,
    year              INTEGER,
    is_weekend        BOOLEAN,
    is_holiday        BOOLEAN,
    fiscal_quarter    INTEGER,
    fiscal_year       INTEGER
);

-- Customer dimension
CREATE TABLE dim_customer (
    customer_key      INTEGER PRIMARY KEY,
    customer_id       INTEGER,              -- Business key (from source system)
    name              TEXT,
    email             TEXT,
    segment           TEXT,                  -- VIP, Regular, New
    city              TEXT,
    state             TEXT,
    country           TEXT,
    -- SCD Type 2 fields (explained below)
    effective_date    DATE,
    expiry_date       DATE,
    is_current        BOOLEAN
);

-- Product dimension
CREATE TABLE dim_product (
    product_key       INTEGER PRIMARY KEY,
    product_id        INTEGER,
    name              TEXT,
    category          TEXT,
    subcategory       TEXT,
    brand             TEXT,

```

```
supplier      TEXT
);
```

Now analytical queries become simple and fast:

```
-- Monthly revenue by product category – no complex joins needed
SELECT d.year,
       d.month_name,
       p.category,
       SUM(f.total_amount) AS revenue,
       SUM(f.profit) AS profit,
       COUNT(*) AS transactions
FROM fact_sales f
JOIN dim_date d ON f.date_key = d.date_key
JOIN dim_product p ON f.product_key = p.product_key
GROUP BY d.year, d.month_name, p.category
ORDER BY d.year, d.month_name;
```

Slowly Changing Dimensions (SCD)

What happens when dimension data changes? A customer moves to a new city. A product gets recategorized. There are three standard approaches:

Type 1 — Overwrite: Just update the row. Simple but you lose history. “What city was this customer in when they made that purchase?” becomes unanswerable.

Type 2 — Add a New Row: Create a new row with the updated values and an `effective_date`. Mark the old row as expired. The `is_current` flag tells you which row is active. You keep full history, but the dimension table grows and queries need to filter on `is_current` or date ranges.

Type 3 — Add a Column: Keep both old and new values in the same row (`previous_city`, `current_city`). Simple to query but only tracks one change.

Most data warehouses use Type 2 for important dimensions (customers, products) and Type 1 for less important ones (where history doesn’t matter).

Key Takeaway: Dimensional modeling (star schemas) is the standard for analytical data. Fact tables record “what happened,” dimension tables provide “context.” This separation makes analytical queries simple, fast, and intuitive.

Document and Graph Models

Relational databases work beautifully for structured, tabular data with well-defined relationships. But not all data fits neatly into tables.

Document Databases

When your data is naturally hierarchical — a product listing with nested specifications, a user profile with varying fields, a blog post with embedded comments — a **document database** stores it as a single JSON-like document:

```
{
  "_id": "prod_12345",
  "name": "Wireless Noise-Canceling Headphones",
  "brand": "AudioTech",
  "price": 299.99,
  "category": ["Electronics", "Audio", "Headphones"],
  "specifications": {
    "driver_size": "40mm",
    "frequency_response": "4Hz-40kHz",
    "battery_life_hours": 30,
    "weight_grams": 250,
    "connectivity": ["Bluetooth 5.2", "USB-C", "3.5mm"]
  },
  "reviews": [
    {
      "user_id": "user_789",
      "rating": 5,
      "text": "Best headphones I've ever owned",
      "date": "2026-03-15"
    },
    {
      "user_id": "user_456",
      "rating": 4,
      "text": "Great sound, slightly tight fit",
      "date": "2026-03-20"
    }
  ],
  "inventory": {
    "warehouse_a": 150,
    "warehouse_b": 80,
    "warehouse_c": 200
  }
}
```

The advantages: no joins needed to load a complete product (it's all in one document), flexible schema (different products can have different specifications), and natural mapping to how applications use data.

The trade-off: if you need to find “all reviews by user_789 across all products,” you’re scanning every document. Relationships *between* documents are awkward. Document databases shine for *self-contained entities* read as a whole; they struggle with *cross-entity queries*.

Embed vs Reference: Put data inside the document (embed) when it's always accessed together and owned by the parent. Use references (foreign keys) when data is shared across documents or updated independently.

Graph Databases

When *relationships* are the most important part of your data, graph databases are the natural fit. Social networks, recommendation engines, fraud detection, knowledge graphs — all are fundamentally about connections.

A graph database stores **nodes** (entities) and **edges** (relationships), both of which can have **properties**. Here's how you'd model a social network in Neo4j's Cypher query language:

```
// Create users (nodes)
CREATE (alice:User {name: 'Alice', joined: date('2025-01-15')})
CREATE (bob:User {name: 'Bob', joined: date('2025-02-20')})
CREATE (charlie:User {name: 'Charlie', joined: date('2025-03-10')})

// Create relationships (edges)
CREATE (alice)-[:FOLLOWS {since: date('2025-03-01')}]>(bob)
CREATE (bob)-[:FOLLOWS {since: date('2025-03-15')}]>(charlie)
CREATE (alice)-[:FOLLOWS {since: date('2025-04-01')}]>(charlie)

// Find friends-of-friends (2 hops) - trivial in a graph, painful in SQL
MATCH (alice:User {name: 'Alice'})-[:FOLLOWS]->()-[:FOLLOWS]->(recommended)
WHERE NOT (alice)-[:FOLLOWS]->(recommended) AND recommended <> alice
RETURN DISTINCT recommended.name AS suggestion;
```

That “friends-of-friends” query would require self-joins and subqueries in SQL and would get slower as the network grows. In a graph database, it's a simple traversal that stays fast regardless of total data size — it only touches the nodes it visits.

Key Takeaway: Use relational databases for structured data with complex queries and transactions. Use document databases for hierarchical, self-contained entities with flexible schemas. Use graph databases when relationships are the primary query pattern.

Event-Driven Schemas

In modern distributed systems, events are often the primary data format. An **event** records something that happened: “user clicked button,” “order was placed,” “temperature exceeded threshold.” Designing schemas for events requires thinking differently from traditional database design.

The Event Envelope Pattern

Every event should include metadata (the “envelope”) alongside its payload:

```
{
  "event_id": "evt_a1b2c3d4",
  "event_type": "order.placed",
  "event_version": "2.1",
  "timestamp": "2026-04-01T14:30:00Z",
  "source": "order-service",
  "correlation_id": "req_x7y8z9",
  "payload": {
    "order_id": "ord_12345",
    "customer_id": "cust_678",
    "items": [
      {"product_id": "prod_111", "quantity": 2, "price": 29.99}
    ],
    "total": 59.98
  }
}
```

The envelope (`event_id` , `event_type` , `timestamp` , `source` , `correlation_id`) is standardized across all events. The `payload` varies by event type. The `correlation_id` lets you trace a request across multiple services.

Schema Evolution with Avro

In streaming systems, producers and consumers evolve independently. You can't take down every consumer to change the event format. **Apache Avro** handles this with explicit schema definitions and compatibility rules.

Here's an Avro schema for an order event. The schema is registered in a Schema Registry, and each message includes the schema ID so consumers know how to deserialize it:

```
{
  "type": "record",
  "name": "OrderPlaced",
  "namespace": "com.example.events",
  "fields": [
    {"name": "order_id", "type": "string"},
    {"name": "customer_id", "type": "string"},
    {"name": "total_amount", "type": "double"},
    {"name": "currency", "type": "string", "default": "USD"},
    {"name": "item_count", "type": "int"},
    {"name": "timestamp", "type": "long"}
  ]
}
```

Schema evolution rules:

- **Backward compatible:** New schema can read old data. Adding a field *with a default value* is backward compatible — old messages just get the default.
- **Forward compatible:** Old schema can read new data. Removing a field *that had a default* is forward compatible — new messages just ignore the removed field.
- **Full compatible:** Both directions work. This is the safest option for production systems.

Key Takeaway: In event-driven systems, schema evolution is not optional — it's a fundamental design concern. Use a schema registry and enforce compatibility rules. Adding optional fields is safe; removing required fields or changing types is not.

Case Study: Designing the Data Model for an E-Commerce Platform

Let's tie everything together by designing the data model for a mid-sized e-commerce platform. This isn't just one model — different parts of the system need different approaches.

The Operational Database (Relational, Normalized)

The transactional core — handling orders, payments, inventory — uses a normalized relational model. Correctness is paramount here: you can't oversell inventory or charge the wrong amount.

```
-- Core operational tables (PostgreSQL)
CREATE TABLE users (
  user_id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email        TEXT UNIQUE NOT NULL,
  name         TEXT NOT NULL,
  password_hash TEXT NOT NULL,
  created_at   TIMESTAMP DEFAULT NOW()
);

CREATE TABLE inventory (
  product_id   INTEGER REFERENCES products(product_id),
  warehouse_id INTEGER REFERENCES warehouses(warehouse_id),
  quantity     INTEGER NOT NULL CHECK (quantity >= 0),
  reserved     INTEGER NOT NULL DEFAULT 0,
  PRIMARY KEY (product_id, warehouse_id)
);
```

The Product Catalog (Document Store)

Product data varies wildly — a laptop has specifications (RAM, CPU, screen size) that a t-shirt doesn't (size, color, material). A document store handles this naturally with flexible schemas per product type.

The Analytics Warehouse (Dimensional Model)

Business intelligence queries use a star schema built by nightly ETL from the operational database. Analysts query fact_orders joined to dimension tables, never touching the production database.

The Recommendation Engine (Graph)

User-product interactions (viewed, purchased, wishlisted) form a graph. Finding “users who bought X also bought Y” is a graph traversal.

The Event Stream (Avro on Kafka)

Every user action — page view, add to cart, purchase, review — is published as an Avro event to Kafka. This feeds the analytics warehouse (batch consumption), the recommendation engine (stream processing), and the real-time dashboards.

The key insight: a single e-commerce platform uses at least four different data modeling approaches, each matched to its specific access patterns and requirements. The operational database is normalized for correctness. The product catalog is denormalized for flexibility. The analytics warehouse is dimensionally modeled for query performance. The event stream is schema-evolved for decoupling.

Key Takeaway: Real systems use multiple data models. Don't force everything into one paradigm. Match the model to the workload: relational for transactions, documents for flexible entities, dimensional for analytics, graphs for relationships, events for decoupled communication.

What's Next

Now that you understand how to model data, Module 3 dives into *where* to store it. We'll explore the internals of storage engines — how B-trees and LSM-trees work, why columnar storage revolutionized analytics, and how modern table formats like Delta Lake and Apache Iceberg brought database-like reliability to data lakes.

Module 3: Storage Systems Deep Dive

The Storage Landscape

Choosing the right storage engine is one of the highest-impact decisions in system design. It's also one of the most misunderstood. Teams often pick a database because they've used it before or because it's popular, without considering whether it actually fits their workload.

Here's the fundamental distinction: **OLTP** (Online Transaction Processing) systems handle lots of small, fast reads and writes — a user adding an item to their cart, a payment being processed, a profile being updated. **OLAP** (Online Analytical Processing) systems handle few, large, complex reads — “What was our revenue by region last quarter?” or “Which products have declining sales trends?”

Think of it this way: an online store has two very different data needs. The checkout flow is OLTP — it processes one order at a time, needs to be fast, and must be correct (you can't charge the wrong amount). The monthly sales report is OLAP — it scans millions of rows, aggregates numbers, and generates summaries. Trying to do both with the same storage engine leads to one workload starving the other.

Key Takeaway: There is no universal “best” database. The right choice depends on whether your primary workload is transactional (OLTP), analytical (OLAP), or a mix. Most real systems need both, often using different storage engines connected by data pipelines.

OLTP Storage Engines

OLTP databases are optimized for point lookups and small writes. The two dominant approaches differ in how they organize data on disk.

B-Tree Engines (PostgreSQL, MySQL InnoDB)

B-trees are the classic approach, used by most relational databases. They organize data in a balanced tree structure where each “page” (typically 4-16 KB) contains sorted keys and pointers to child pages. Finding any record requires traversing from the root to a leaf — typically 3-4 page reads, regardless of how much data you have.

Strengths: Fast reads, efficient range queries, mature and well-understood, good for read-heavy workloads.

How writes work: To update a record, the database finds the page containing that record and overwrites it in place. To ensure durability, every change is first written to a **Write-Ahead Log (WAL)** — a sequential append-only file. If the database crashes mid-write, the WAL can replay uncommitted changes on restart.

LSM-Tree Engines (RocksDB, Cassandra, LevelDB)

LSM-trees (Log-Structured Merge Trees) take a different approach: all writes go to an in-memory buffer (memtable). When the buffer fills up, it's flushed to disk as a sorted file called an **SSTable** (Sorted String Table). Over time, background processes merge SSTables together (**compaction**) to keep read performance manageable.

Strengths: Extremely fast writes (sequential I/O only), good compression, excellent for write-heavy workloads.

Trade-off: Reads may need to check multiple SSTables before finding a value. Bloom filters help skip SSTables that definitely don't contain a key, but reads are still generally slower than B-tree point lookups.

When to Choose Which

FACTOR	B-TREE (POSTGRESQL)	LSM-TREE (CASSANDRA)
Read speed	Faster (single lookup)	Slower (check multiple SSTables)
Write speed	Slower (random I/O, in-place updates)	Faster (sequential I/O, append-only)
Space efficiency	Moderate (fragmentation)	Better (compaction reduces waste)
Read-heavy workloads	Excellent	Good
Write-heavy workloads	Good	Excellent
Transactions	Full ACID support	Limited or none
Scaling	Primarily vertical	Horizontally scalable
Use cases	Web apps, e-commerce, CMS	IoT, time-series, messaging, logging

Here's a PostgreSQL example showing typical OLTP patterns — notice how each query touches a small number of rows:

```

-- OLTP pattern: point lookup by primary key
SELECT name, email, subscription_tier
FROM users
WHERE user_id = 'usr_12345';

-- OLTP pattern: insert a single row with constraints
INSERT INTO orders (customer_id, total_amount, status)
VALUES ('usr_12345', 89.99, 'pending')
RETURNING order_id;

-- OLTP pattern: update one row in a transaction
BEGIN;
UPDATE inventory
SET quantity = quantity - 1, reserved = reserved + 1
WHERE product_id = 'prod_67890' AND quantity > 0;
COMMIT;

```

These queries are fast because they touch few rows and use indexed lookups. The same database would struggle with “scan all 500 million orders and compute monthly revenue by region” — that’s an OLAP query.

OLAP and Columnar Storage

Analytical queries have a fundamentally different access pattern than transactional ones. Instead of reading all columns for one row, they read a few columns across millions of rows.

Why Row-Based Storage Fails for Analytics

In a traditional row-based database, data is stored row by row on disk:

```

Row 1: [user_id=1, name="Alice", email="alice@ex.com", city="NYC",
signup_date="2025-01-15"]
Row 2: [user_id=2, name="Bob", email="bob@ex.com", city="LA", signup_date="2025-02-20"]
Row 3: [user_id=3, name="Charlie", email="charlie@ex.com", city="NYC",
signup_date="2025-03-10"]

```

To answer “How many users are in NYC?”, the database reads *every column of every row* from disk, even though it only needs `city`. For a table with 50 columns and a billion rows, that’s 49 columns of wasted I/O.

Column-Oriented Storage

Columnar databases store data column by column:

```
user_id column:    [1, 2, 3, ...]
name column:       ["Alice", "Bob", "Charlie", ...]
city column:       ["NYC", "LA", "NYC", ...]
signup_date column: ["2025-01-15", "2025-02-20", "2025-03-10", ...]
```

Now “How many users are in NYC?” only reads the `city` column — a fraction of the data. Better still, columns compress extremely well because adjacent values are often similar:

- **Run-Length Encoding:** `["NYC", "NYC", "NYC", "LA", "LA"]` becomes `[("NYC", 3), ("LA", 2)]`
- **Dictionary Encoding:** Replace repeated strings with small integers. If “NYC” appears a million times, store the integer `0` a million times and keep one mapping: `{0: "NYC", 1: "LA", ...}`
- **Bit-Packing:** A column with 4 unique values needs only 2 bits per value instead of a full string

These techniques can reduce storage by 10x or more, which also means 10x less I/O for queries.

Modern Columnar Engines

ClickHouse: Open-source, extremely fast for aggregation queries. Uses its own MergeTree engine. Popular for real-time analytics dashboards and log analysis.

Apache Druid: Optimized for sub-second OLAP queries on event data. Pre-aggregates data on ingestion. Popular for user-facing analytics.

Google BigQuery: Serverless columnar warehouse based on Google’s Dremel technology. You pay per query (data scanned) with no infrastructure to manage. Excellent for variable workloads.

Key Takeaway: Columnar storage is dramatically faster for analytics because it reads only the columns a query needs and compresses those columns efficiently. If your queries scan millions of rows but only use a few columns — and most analytical queries do — columnar storage is the clear choice.

Modern Table Formats: Parquet, Delta Lake, Iceberg, Hudi

The traditional approach — store raw files on a data lake, query them with Spark or Presto — worked but had serious problems: no transactions, no schema enforcement, terrible performance with many small files, and no way to update or delete individual records.

Modern table formats solve these problems by adding database-like features on top of object storage.

Apache Parquet

Parquet is a **columnar file format**, not a table format. It's the building block that the others are built on. A Parquet file organizes data into row groups (typically 128 MB each), where each column within a row group is stored contiguously and compressed independently.

Every Parquet file contains metadata: min/max values per column per row group, null counts, and encoding information. Query engines use this metadata to skip entire row groups that can't contain relevant data — a technique called **predicate pushdown**.

Delta Lake

Delta Lake, created by Databricks, adds **ACID transactions** to data lakes. It works by maintaining a transaction log (`_delta_log/`) alongside your Parquet data files. Every operation (write, update, delete) creates a new log entry describing what changed.

Key capabilities:

- **ACID transactions**: Multiple writers can safely update the same table concurrently
- **Time travel**: Query data as it existed at any previous point in time
- **Schema enforcement**: Reject writes that don't match the expected schema
- **Schema evolution**: Safely add new columns without rewriting existing data

Here's how Delta Lake operations look in PySpark. Notice how they feel like database operations, but they're running on files in S3 or HDFS:

```

from delta.tables import DeltaTable
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("DeltaLakeDemo") \
    .config("spark.jars.packages", "io.delta:delta-core_2.12:2.4.0") \
    .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension") \
    .getOrCreate()

# Write a DataFrame as a Delta table – creates Parquet files + transaction log
df = spark.read.json("s3://data-lake/raw/events/2026-04-01/")
df.write.format("delta").mode("overwrite").save("s3://data-lake/silver/events")

# Read the Delta table – exactly like reading a regular table
events = spark.read.format("delta").load("s3://data-lake/silver/events")

# MERGE operation – upsert (update existing rows, insert new ones)
# This is impossible with plain Parquet files
delta_table = DeltaTable.forPath(spark, "s3://data-lake/silver/events")
delta_table.alias("target").merge(
    new_data.alias("source"),
    "target.event_id = source.event_id"
).whenMatchedUpdateAll() \
.whenNotMatchedInsertAll() \
.execute()

# Time travel – query the table as it was 24 hours ago
yesterday = spark.read.format("delta") \
    .option("timestampAsOf", "2026-04-01") \
    .load("s3://data-lake/silver/events")

```

Apache Iceberg

Created at Netflix, Iceberg focuses on **correctness and performance at scale**. Its killer features:

- **Hidden partitioning:** You define partition transforms (e.g., “partition by month of event_date”) and Iceberg handles the details. Users don’t need to know the partitioning scheme to write efficient queries.
- **Partition evolution:** Change your partitioning strategy without rewriting data. Start partitioned by day; switch to hour when volume increases.
- **Snapshot isolation:** Readers and writers never conflict. Readers see a consistent snapshot while writes proceed concurrently.

Apache Hudi

Created at Uber for their incremental processing needs, Hudi (Hadoop Upserts Deletes and Incrementals) provides two storage types:

- **Copy-on-Write (CoW):** Rewrites entire files on every update. Optimal for read-heavy workloads.

- **Merge-on-Read (MoR):** Writes updates to a delta log, merging on read. Optimal for write-heavy workloads with less frequent reads.

Comparison

FEATURE	DELTA LAKE	ICEBERG	HUDI
ACID transactions	Yes	Yes	Yes
Time travel	Yes	Yes	Yes
Schema evolution	Yes	Yes	Yes
Hidden partitioning	No	Yes	No
Partition evolution	Limited	Yes	Limited
Upsert support	Yes	Yes	Yes (core feature)
Incremental queries	Yes	Yes	Yes (core feature)
Best for	Databricks ecosystem	Multi-engine, correctness	Incremental processing
Origin	Databricks	Netflix	Uber

Key Takeaway: If you're building a data lake today, use a table format. Delta Lake is the default if you're in the Databricks ecosystem. Iceberg is the best choice for multi-engine environments and complex partitioning needs. Hudi excels at incremental processing and near-real-time ingestion.

Object Storage Patterns

Object storage (S3, GCS, Azure Blob Storage) is the foundation of modern data systems. It's virtually unlimited in capacity, extremely durable (11 nines), and cheap — roughly \$0.023/GB/month for standard storage. Nearly every data lake, lakehouse, and modern data platform stores its data here.

Partitioning Strategies

How you organize files in object storage dramatically affects query performance. The standard approach is **Hive-style partitioning**, where the directory path encodes partition values:

```
s3://data-lake/events/  
year=2026/month=04/day=01/part-00000.parquet  
year=2026/month=04/day=01/part-00001.parquet  
year=2026/month=04/day=02/part-00000.parquet  
year=2026/month=03/day=31/part-00000.parquet
```

A query filtering on `WHERE year=2026 AND month=04 AND day=01` only lists and reads files in that specific directory, skipping everything else. Without partitioning, the query engine would need to scan every file in the entire dataset.

Here's a Python example showing how to write partitioned data to S3 and set up lifecycle policies for cost optimization. The lifecycle policy automatically moves old data to cheaper storage tiers:

```

import boto3
from datetime import datetime

s3 = boto3.client('s3')

def upload_partitioned_data(bucket: str, data: bytes, event_date: datetime):
    """Upload data to S3 with Hive-style partitioning.

    The key (path) encodes the date, so query engines can use
    partition pruning to skip irrelevant files.
    """
    key = (
        f"events/"
        f"year={event_date.year}/"
        f"month={event_date.month:02d}/"
        f"day={event_date.day:02d}/"
        f"data-{datetime.utcnow().strftime('%H%M%S')}.parquet"
    )
    s3.put_object(Bucket=bucket, Key=key, Body=data)
    return key

def setup_lifecycle_policy(bucket: str):
    """Configure storage tiering to reduce costs automatically.

    Hot data (recent) stays in Standard storage for fast access.
    After 90 days, it moves to Infrequent Access (50% cheaper).
    After 365 days, it moves to Glacier (90% cheaper).
    After 7 years, it's deleted (compliance retention met).
    """
    s3.put_bucket_lifecycle_configuration(
        Bucket=bucket,
        LifecycleConfiguration={
            'Rules': [
                {
                    'ID': 'tiered-storage',
                    'Status': 'Enabled',
                    'Filter': {'Prefix': 'events/'},
                    'Transitions': [
                        {'Days': 90, 'StorageClass': 'STANDARD_IA'},
                        {'Days': 365, 'StorageClass': 'GLACIER'},
                    ],
                    'Expiration': {'Days': 2555}, # ~7 years
                }
            ]
        }
    )

```

Storage Tiers

TIER	COST (PER GB/MONTH)	ACCESS TIME	USE CASE
Standard	\$0.023	Milliseconds	Active data, frequent access
Infrequent Access	\$0.0125	Milliseconds	Older data, occasional access
Glacier	\$0.004	Minutes to hours	Archives, compliance
Glacier Deep Archive	\$0.00099	12+ hours	Regulatory retention

A well-designed lifecycle policy can cut storage costs by 60-80% for data that ages out of active use.

HTAP: Bridging OLTP and OLAP

The traditional pattern — separate OLTP and OLAP systems connected by ETL — works but introduces latency. Your analytics are always hours behind reality. **HTAP** (Hybrid Transactional/Analytical Processing) databases aim to handle both workloads in a single system.

TiDB: A MySQL-compatible distributed database that separates transaction processing (TiKV, row-based) from analytical processing (TiFlash, columnar). Writes go to TiKV; TiFlash maintains a columnar replica for analytics. Queries are automatically routed to the appropriate engine.

CockroachDB: A distributed SQL database with strong consistency. Handles OLTP well and has improving analytical capabilities, though it's not a replacement for a dedicated warehouse at large scale.

SingleStore (formerly MemSQL): An in-memory database that supports both row and column storage. Tables can be configured as row-store (OLTP) or column-store (OLAP).

When HTAP makes sense: Small to medium scale where operational and analytical queries run on the same data and you want to avoid the complexity of a separate data warehouse and ETL pipeline. When your analytics need to reflect the latest transactions immediately.

When to keep systems separate: Large scale where OLTP and OLAP workloads would interfere with each other, when you need specialized analytical features (star schemas, materialized views, window functions), or when your analytical data comes from multiple sources.

Choosing the Right Storage: A Decision Framework

When facing a storage decision, work through these questions in order:

1. What's the primary access pattern?

- 2. Point lookups and small writes → OLTP (PostgreSQL, MySQL, DynamoDB)
- 3. Full table scans and aggregations → OLAP (BigQuery, Snowflake, ClickHouse)
- 4. Key-value lookups at extreme speed → Cache (Redis) or Key-Value store (DynamoDB)

5. What's the data structure?

- 6. Structured with relationships → Relational (PostgreSQL)
- 7. Hierarchical/flexible → Document (MongoDB)
- 8. Relationship-heavy → Graph (Neo4j)
- 9. Time-series → Specialized (InfluxDB, TimescaleDB)

10. What are the consistency requirements?

- 11. Strong consistency (financial, inventory) → ACID database
- 12. Eventual consistency acceptable (social feeds, analytics) → AP systems (Cassandra, DynamoDB)

13. What's the scale?

- 14. Moderate (< 1TB, < 10K QPS) → Almost anything works; pick what your team knows
- 15. Large (1-100 TB, 10K-100K QPS) → Requires careful selection and potentially sharding
- 16. Massive (100+ TB, 100K+ QPS) → Specialized distributed systems, often multiple engines

17. What's the team's expertise?

- 18. This matters more than people admit. A team that knows PostgreSQL deeply will outperform one using Cassandra for the first time, even if Cassandra is the “better” theoretical fit.

Key Takeaway: Start with the simplest storage that meets your requirements. PostgreSQL is remarkably capable — it handles JSON documents, full-text search, geospatial queries, and moderate analytics in addition to traditional relational workloads. Only introduce specialized systems when you have a clear need that PostgreSQL can't meet.

What's Next

Now that you understand how data is stored, Module 4 covers how it's *processed*. We'll explore batch processing — the workhorse of data engineering — including ETL pipelines, workflow orches-

tration with Airflow, and the architectural patterns (Lambda and Kappa) that determine how batch and stream processing coexist.

Module 4: Batch Processing & ETL

What is Batch Processing?

Batch processing is the workhorse of data engineering. While real-time systems get the headlines, the vast majority of data work — loading data warehouses, training ML models, generating reports, computing aggregations — happens in batches.

Batch processing means collecting data over a period of time, then processing it all at once. Your bank doesn't update your monthly statement in real-time — it waits until the month ends, processes all transactions, and generates the statement. That's batch processing.

The advantages are significant: batch jobs can be optimized for throughput (process a billion rows efficiently), they're easier to debug (you can re-run the same input and get the same output), and they're more cost-effective (you spin up compute resources, do the work, then shut them down).

Key Takeaway: Not everything needs to be real-time. If the business can tolerate data that's hours old, batch processing is simpler, cheaper, and more reliable. Save real-time processing for use cases that genuinely require it (fraud detection, live dashboards, real-time recommendations).

Lambda vs. Kappa Architecture

How should batch and stream processing coexist in your system? Two dominant architectures address this question.

Lambda Architecture

The Lambda architecture runs two parallel processing paths:

```
[Data Source] → [Queue] → {
    r→ [Batch Layer: Spark/Hadoop] → [Serving Layer]
    L→ [Speed Layer: Flink/Storm] → [Serving Layer]
} → [Query]
```

The **batch layer** processes all historical data periodically (e.g., every hour) to produce accurate, complete results. The **speed layer** processes data in real-time to produce approximate, up-to-date results. The query layer merges both views — recent data from the speed layer and historical data from the batch layer.

Pros: The batch layer is a simple, reliable source of truth. If the speed layer has a bug, the batch layer corrects it on the next run. You get both completeness and freshness.

Cons: You maintain two codebases doing essentially the same computation in different frameworks. This is the Lambda architecture's fatal flaw — the dual maintenance burden is brutal in practice.

Kappa Architecture

The Kappa architecture simplifies by eliminating the batch layer entirely:

```
[Data Source] → [Event Log (Kafka)] → [Stream Processor] → [Serving Layer] → [Query]
```

All data flows through a single stream processing pipeline. If you need to reprocess historical data (because of a bug or a new computation), you replay events from the log.

Pros: One codebase, one processing framework, simpler operations.

Cons: Stream processing is inherently more complex than batch. Replaying a year of events through a stream processor is slow. Some computations (complex ML training, graph algorithms) don't fit naturally into streaming.

Which to Choose

For most teams starting out: **start with batch, add streaming when you have a genuine real-time requirement.** Lambda is appropriate when you need guaranteed correctness *and* real-time speed (e.g., financial reporting). Kappa works well when your primary processing is event-driven and reprocessing is manageable.

ETL vs. ELT

ETL (Extract, Transform, Load): Data is extracted from sources, transformed in a separate processing engine (Spark, Python scripts), then loaded into the destination. This was the standard when data warehouses had limited compute and expensive storage.

ELT (Extract, Load, Transform): Data is extracted and loaded into the destination *first* (raw), then transformed using the destination's own compute engine. This is the modern approach, enabled by cloud warehouses with massive, elastic compute.

```
ETL: Source → [Transform in Spark] → Warehouse (clean data)
ELT: Source → Warehouse (raw data) → [Transform in SQL/dbt] → Warehouse (clean data)
```

Why ELT won for cloud warehouses: Cloud warehouses like Snowflake and BigQuery have virtually unlimited compute. It's cheaper and simpler to load raw data and transform it using SQL than to

maintain a separate Spark cluster. Plus, keeping the raw data means you can always re-transform it if your logic changes.

dbt (data build tool) is the poster child of ELT. It lets you define transformations as SQL SELECT statements, manages dependencies between transformations, runs tests, and generates documentation:

```
-- models/staging/stg_orders.sql
-- dbt model: clean and standardize raw order data

SELECT
    id AS order_id,
    user_id AS customer_id,
    CAST(created_at AS TIMESTAMP) AS order_date,
    CAST(total AS DECIMAL(10,2)) AS order_total,
    LOWER(TRIM(status)) AS order_status,
    CASE
        WHEN LOWER(TRIM(status)) IN ('completed', 'delivered') THEN TRUE
        ELSE FALSE
    END AS is_completed
FROM {{ source('raw', 'orders') }}
WHERE id IS NOT NULL
    AND created_at IS NOT NULL
```

Workflow Orchestration

A real data pipeline isn't a single script — it's a sequence of interdependent tasks: extract from 5 sources, clean each one, join them together, load into the warehouse, run quality checks, refresh dashboards. If the extraction fails, everything downstream should wait. If the quality check fails, the dashboard shouldn't update.

Workflow orchestrators manage these dependencies, handle retries, send alerts on failure, and provide visibility into what's running.

Apache Airflow

Airflow is the dominant orchestrator. You define workflows as **DAGs** (Directed Acyclic Graphs) in Python. Each node in the DAG is a **task** — a unit of work like “run a SQL query” or “execute a Python function.” Edges define dependencies.

Here's a complete Airflow DAG for a daily analytics pipeline. This is a realistic example — it extracts data from multiple sources, transforms it, runs quality checks, and loads the results:

```

from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.providers.postgres.operators.postgres import PostgresOperator
from airflow.operators.email import EmailOperator
from airflow.utils.task_group import TaskGroup

default_args = {
    'owner': 'data-engineering',
    'depends_on_past': False,
    'email_on_failure': True,
    'email': ['data-alerts@company.com'],
    'retries': 2,
    'retry_delay': timedelta(minutes=5),
}

# The DAG runs daily at 6 AM UTC, processing the previous day's data.
# catchup=False means it won't backfill missed runs on first deploy.
with DAG(
    dag_id='daily_analytics_pipeline',
    default_args=default_args,
    description='Daily ETL: extract, transform, quality check, load',
    schedule_interval='0 6 * * *',
    start_date=datetime(2026, 1, 1),
    catchup=False,
    tags=['analytics', 'daily'],
) as dag:

    def extract_orders(**context):
        """Extract orders from the transactional database.

        Uses the execution_date to only pull yesterday's orders,
        making this pipeline idempotent – running it twice for
        the same date produces the same result.
        """
        execution_date = context['ds'] # YYYY-MM-DD string
        # In production: query source DB, write to staging area
        print(f"Extracting orders for {execution_date}")

    def extract_payments(**context):
        """Extract payment data from the payment provider API."""
        execution_date = context['ds']
        print(f"Extracting payments for {execution_date}")

    def transform_and_join(**context):
        """Join orders with payments, compute derived metrics.

        This task depends on both extractions completing successfully.
        If either fails, this task won't run.
        """
        execution_date = context['ds']
        print(f"Transforming data for {execution_date}")

    def run_quality_checks(**context):

```

```

    """Validate transformed data before loading to production.

    Checks: row counts within expected range, no null primary keys,
    amounts are positive, dates are valid. Raises an exception
    (failing the task) if any check fails.
    """

    execution_date = context['ds']
    print(f"Running quality checks for {execution_date}")

# Task Group: Extraction (these run in parallel)
with TaskGroup('extraction') as extract_group:
    extract_orders_task = PythonOperator(
        task_id='extract_orders',
        python_callable=extract_orders,
    )
    extract_payments_task = PythonOperator(
        task_id='extract_payments',
        python_callable=extract_payments,
    )

# Transformation (depends on all extractions)
transform_task = PythonOperator(
    task_id='transform_and_join',
    python_callable=transform_and_join,
)

# Quality checks (depends on transformation)
quality_task = PythonOperator(
    task_id='quality_checks',
    python_callable=run_quality_checks,
)

# Load to production warehouse
load_task = PostgresOperator(
    task_id='load_to_warehouse',
    postgres_conn_id='warehouse',
    sql='sql/load_daily_analytics.sql',
)

# Send completion notification
notify_task = EmailOperator(
    task_id='send_notification',
    to='analytics-team@company.com',
    subject='Daily Analytics Pipeline Complete - {{ ds }}',
    html_content='Pipeline completed successfully for {{ ds }}.',
)

# Define the dependency chain
extract_group >> transform_task >> quality_task >> load_task >> notify_task

```

The key things to notice: - **Parallel extraction:** Orders and payments are extracted simultaneously (both are in the same TaskGroup with no dependency between them) - **Idempotency:** Each task uses `execution_date` to process exactly one day's data, so re-running is safe - **Failure handling:**

2 retries with 5-minute delays, email alerts on failure - **Clear dependencies:** The `>>` operator makes the flow explicit and readable

Dagster: A Modern Alternative

Dagster takes an **asset-based** approach instead of Airflow's **task-based** approach. Instead of defining "what to do" (tasks), you define "what to produce" (assets). Dagster figures out the execution order from asset dependencies. This is arguably a more natural way to think about data pipelines — you care about the outputs, not the steps.

Key Takeaway: Use an orchestrator for any pipeline with more than one step. Airflow is the industry standard with the largest community and ecosystem. Dagster is a strong alternative with a more modern developer experience. Avoid running important pipelines as cron jobs — you'll miss the dependency management, retry logic, and visibility that orchestrators provide.

Data Quality in Batch Pipelines

The most insidious production issue isn't a pipeline that fails — it's a pipeline that *succeeds with wrong data*. A silent data quality bug can propagate through your warehouse for weeks before anyone notices that revenue numbers are wrong.

Data quality checks should be a first-class part of every pipeline — not an afterthought.

Practical Quality Checks

Here's a data quality validation framework you can use in any Python pipeline. Each check is simple, but together they catch the vast majority of data quality issues:

```

from dataclasses import dataclass
from typing import List, Optional
import pandas as pd

@dataclass
class QualityCheckResult:
    check_name: str
    passed: bool
    details: str

def run_quality_checks(df: pd.DataFrame, table_name: str) -> List[QualityCheckResult]:
    """Run a suite of data quality checks on a DataFrame.

    Returns a list of results. If any check fails, the pipeline
    should stop and alert the team – loading bad data is worse
    than loading no data.
    """
    results = []

    # Check 1: Row count within expected range
    row_count = len(df)
    expected_min, expected_max = 1000, 1_000_000
    results.append(QualityCheckResult(
        check_name='row_count',
        passed=expected_min <= row_count <= expected_max,
        details=f"Got {row_count} rows (expected {expected_min}-{expected_max})"
    ))

    # Check 2: No null primary keys
    null_pks = df['id'].isnull().sum()
    results.append(QualityCheckResult(
        check_name='no_null_primary_keys',
        passed=null_pks == 0,
        details=f"Found {null_pks} null primary keys"
    ))

    # Check 3: No duplicate primary keys
    duplicate_pks = df['id'].duplicated().sum()
    results.append(QualityCheckResult(
        check_name='no_duplicate_primary_keys',
        passed=duplicate_pks == 0,
        details=f"Found {duplicate_pks} duplicate primary keys"
    ))

    # Check 4: Freshness – most recent record is within expected window
    if 'created_at' in df.columns:
        max_date = pd.to_datetime(df['created_at']).max()
        hours_old = (pd.Timestamp.now() - max_date).total_seconds() / 3600
        results.append(QualityCheckResult(
            check_name='data_freshness',
            passed=hours_old < 48,
            details=f"Most recent record is {hours_old:.1f} hours old"
        ))

```

```
# Check 5: Numeric values in valid range
if 'amount' in df.columns:
    negative_amounts = (df['amount'] < 0).sum()
    results.append(QualityCheckResult(
        check_name='no_negative_amounts',
        passed=negative_amounts == 0,
        details=f"Found {negative_amounts} negative amounts"
    ))

return results
```

Key Takeaway: Build quality checks into your pipeline, not after it. The pattern is: extract → transform → **validate** → load. If validation fails, the load doesn't happen. It's always better to have no data than wrong data.

Cost Optimization for Batch

Batch processing has a natural cost advantage: you can schedule jobs for off-peak hours, use spot instances, and shut down resources when they're idle.

Spot/Preemptible Instances: Cloud providers sell excess compute capacity at 60-90% discounts. The catch: they can reclaim the instance with 2 minutes notice. Batch jobs are perfect for spot instances because they can be checkpointed and restarted. Configure your Spark or Airflow workers to use spot instances with on-demand instances as fallback.

Right-Size Your Clusters: Don't run a 20-node Spark cluster when 5 nodes would finish the job in the same time. Profile your jobs to find the sweet spot between parallelism and overhead.

Incremental Processing: Instead of reprocessing all historical data daily, only process what's new. If your pipeline processes the last 24 hours of data instead of the full history, it runs faster and costs less. Use `WHERE created_at > last_processed_timestamp` patterns, or leverage Delta Lake's Change Data Feed.

Schedule Off-Peak: Cloud compute often costs less at night and on weekends (lower demand). Schedule non-urgent batch jobs for off-peak hours.

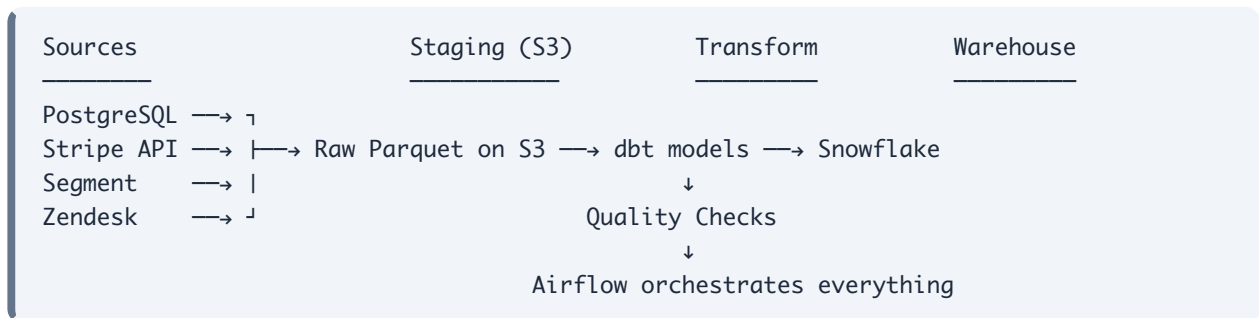
Case Study: Building a Daily Analytics Pipeline

Let's walk through designing a complete daily analytics pipeline for a SaaS company with 50,000 users generating events across a web app, mobile app, and API.

Requirements

- **Sources:** PostgreSQL (transactional data), Stripe API (payments), Segment (web/mobile events), Zendesk (support tickets)
- **Output:** A Snowflake data warehouse feeding Tableau dashboards
- **Freshness:** Business stakeholders need yesterday's data by 9 AM
- **Scale:** ~2 GB of new data per day across all sources

Architecture



Key Design Decisions

1. **ELT over ETL:** Load raw data to S3 first, then use dbt + Snowflake for transformation. This keeps raw data available for re-transformation and leverages Snowflake's compute.
2. **Idempotent extractions:** Each extractor writes to a date-partitioned path (`s3://staging/stripe/dt=2026-04-01/`). Re-running overwrites the same partition — safe and predictable.
3. **Quality gates:** After transformation, quality checks validate row counts, null rates, and referential integrity. If checks fail, the old data stays in place and the team is alerted.
4. **Incremental models:** dbt models are configured as incremental where possible — only processing rows with `updated_at` after the last run. This keeps dbt execution under 15 minutes even as data grows.
5. **Monitoring:** Pipeline duration, row counts per source, and Snowflake credit usage are tracked in Datadog. Alerts fire if the pipeline hasn't completed by 8 AM or if row counts deviate by more than 20% from the previous day.

Key Takeaway: A good batch pipeline is idempotent (safe to re-run), observable (you know when it fails and why), incremental (processes only new data), and quality-gated (never loads bad data). These properties matter more than the specific technologies you choose.

What's Next

Batch processing handles the bulk of data work, but some use cases can't wait for the next scheduled run. Module 5 introduces stream processing — how to handle data that arrives continuously and needs to be acted on immediately.

Module 5: Stream Processing & Real-Time Systems

Why Real-Time?

Not everything needs to be real-time — but some things absolutely do. When a credit card transaction looks fraudulent, you can't wait for tomorrow's batch job to flag it. When a user is browsing your website, you want recommendations based on what they're doing *right now*, not what they did yesterday. When a server is about to run out of disk space, you need an alert within seconds, not hours.

Real-time processing isn't better than batch processing — it's more complex and more expensive. The question isn't "should we do real-time?" but "which specific use cases justify the added complexity?"

Common real-time use cases: - **Fraud detection**: Score every transaction in milliseconds - **Live dashboards**: Show business metrics that update every few seconds - **Recommendations**: Adapt to user behavior in the current session - **Alerting**: Detect anomalies and notify humans immediately - **IoT**: Process sensor data as it arrives from thousands of devices

Key Takeaway: Batch and real-time aren't competing approaches — they're complementary. Most systems use both. Use batch for the 90% of work where latency doesn't matter. Use streaming for the 10% where freshness is critical.

Message Brokers and Kafka Fundamentals

At the heart of every streaming system is a **message broker** — a system that receives messages from producers and delivers them to consumers. Apache Kafka has become the de facto standard.

Why Kafka Won

Before Kafka, message brokers (RabbitMQ, ActiveMQ) followed the traditional queue model: a message is delivered to one consumer, then deleted. Kafka introduced a fundamentally different model: messages are *appended to a log* and *retained for a configurable period* (days, weeks, or indefin-

itely). Consumers read from the log at their own pace, and multiple consumers can independently read the same messages.

This design gives you: - **Durability**: Messages survive broker restarts (they're on disk) - **Replayability**: New consumers can start from the beginning and process historical data - **Decoupling**: Producers and consumers don't need to know about each other - **Scalability**: Partitioning distributes load across multiple brokers

Core Concepts

Topics are named feeds of messages. "user-events" might contain all user interactions. "orders" might contain all order events. Choose topic granularity based on how consumers need the data.

Partitions divide a topic across multiple brokers for parallelism. Messages within a partition are strictly ordered. A topic with 12 partitions can be consumed by up to 12 parallel consumer instances.

Consumer Groups allow multiple instances of a service to divide the work. Each partition is assigned to exactly one consumer in the group — so with 12 partitions and 4 consumers, each consumer handles 3 partitions.

Offsets track each consumer's position in a partition. If a consumer crashes and restarts, it resumes from its last committed offset — no data is lost.

Here's a practical example showing a Kafka producer and consumer in Python. The producer publishes user events; the consumer processes them. Notice how the consumer commits offsets after successful processing to ensure at-least-once delivery:

```

from confluent_kafka import Producer, Consumer
import json
from datetime import datetime

# --- Producer: Publish user events to Kafka ---

def create_producer():
    """Create a Kafka producer with reliability settings.

    'acks=all' means the producer waits for all replicas to confirm
    receipt before considering a write successful. This is slower
    but guarantees no data loss.
    """
    return Producer({
        'bootstrap.servers': 'kafka-broker-1:9092,kafka-broker-2:9092',
        'acks': 'all',
        'retries': 3,
        'retry.backoff.ms': 1000,
    })

def publish_user_event(producer, user_id: str, event_type: str, data: dict):
    """Publish a user event to the 'user-events' topic.

    The key (user_id) determines which partition receives this message.
    All events for the same user go to the same partition, preserving
    ordering per user – critical for session analysis.
    """
    event = {
        'user_id': user_id,
        'event_type': event_type,
        'timestamp': datetime.utcnow().isoformat(),
        'data': data,
    }
    producer.produce(
        topic='user-events',
        key=user_id.encode('utf-8'),
        value=json.dumps(event).encode('utf-8'),
    )
    producer.flush() # Ensure delivery (in production, batch flushes)

# --- Consumer: Process user events ---

def create_consumer(group_id: str):
    """Create a Kafka consumer.

    'auto.offset.reset=earliest' means a new consumer group starts
    from the beginning of the topic. 'enable.auto.commit=False'
    gives us manual control over when offsets are committed –
    we only commit after successful processing.
    """
    return Consumer({
        'bootstrap.servers': 'kafka-broker-1:9092,kafka-broker-2:9092',
        'group.id': group_id,
        'auto.offset.reset': 'earliest',
    })

```

```

        'enable.auto.commit': False,
    })

def process_events():
    consumer = create_consumer('analytics-pipeline')
    consumer.subscribe(['user-events'])

    while True:
        msg = consumer.poll(timeout=1.0)
        if msg is None:
            continue
        if msg.error():
            print(f"Consumer error: {msg.error()}")
            continue

        event = json.loads(msg.value().decode('utf-8'))
        try:
            handle_event(event)
            # Only commit offset after successful processing
            consumer.commit(asynchronous=False)
        except Exception as e:
            print(f"Processing failed: {e}")
            # Don't commit - message will be reprocessed on next poll

def handle_event(event: dict):
    """Route events to appropriate handlers."""
    handlers = {
        'page_view': handle_page_view,
        'purchase': handle_purchase,
        'search': handle_search,
    }
    handler = handlers.get(event['event_type'])
    if handler:
        handler(event)

```

Schema Evolution with Avro

In a streaming system, producers and consumers are different services deployed independently. You can't update them simultaneously. When the producer starts sending events with a new field, old consumers need to handle it gracefully. When a consumer expects a field that old producers don't send, it needs a sensible default.

Apache Avro solves this with explicit schemas and a **Schema Registry** that enforces compatibility rules.

Here's an example showing schema evolution. Version 1 of the event has basic fields. Version 2 adds an optional `device_type` field with a default value — this is backward compatible because

old consumers can still read v2 messages (they just ignore the new field), and new consumers can read v1 messages (they get the default value):

```
// Version 1: Original event schema
{
  "type": "record",
  "name": "UserEvent",
  "namespace": "com.example.events",
  "fields": [
    {"name": "user_id", "type": "string"},
    {"name": "event_type", "type": "string"},
    {"name": "timestamp", "type": "long"},
    {"name": "page_url", "type": ["null", "string"], "default": null}
  ]
}

// Version 2: Added device_type with a default – backward compatible
{
  "type": "record",
  "name": "UserEvent",
  "namespace": "com.example.events",
  "fields": [
    {"name": "user_id", "type": "string"},
    {"name": "event_type", "type": "string"},
    {"name": "timestamp", "type": "long"},
    {"name": "page_url", "type": ["null", "string"], "default": null},
    {"name": "device_type", "type": "string", "default": "unknown"}
  ]
}
```

Safe changes (backward compatible): Adding a field with a default value. Making a required field optional.

Unsafe changes (breaking): Removing a field without a default. Changing a field's type. Renaming a field.

Key Takeaway: Always use a schema registry in production streaming systems. It prevents breaking changes from reaching production and documents the contract between producers and consumers. Enforce at least backward compatibility; full compatibility is even safer.

Stream Processing Frameworks

Raw Kafka consumers work for simple event handling, but complex logic — windowed aggregations, stateful processing, joining multiple streams — requires a stream processing framework.

Apache Flink

Flink is the most powerful stream processing framework. It provides **exactly-once processing** guarantees, **event-time processing** (handling late and out-of-order events), and **stateful computation** (maintaining state across events).

Flink's key concepts: - **Event time vs. processing time**: Event time is when the event actually happened; processing time is when the system processes it. Using event time produces correct results even when events arrive late. - **Watermarks**: A mechanism for tracking progress in event time. A watermark of T means "all events with timestamp $\leq T$ have arrived." - **Windows**: Group events for aggregation. Tumbling windows (fixed, non-overlapping), sliding windows (overlapping), session windows (gap-based).

Kafka Streams

Kafka Streams is a *library*, not a framework. You embed it in a regular Java/Kotlin application — no separate cluster to manage. It's perfect for moderate-scale stream processing where you want simplicity over raw power.

Spark Structured Streaming

Spark's streaming model is **micro-batch**: it processes data in tiny batches (as small as 100ms). This is technically not true streaming, but for many use cases the distinction doesn't matter. The big advantage: you use the same Spark API for both batch and streaming.

Here's a practical Spark Structured Streaming example that reads from Kafka, computes real-time aggregations, and writes results to a sink:

```

from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    from_json, col, window, count, avg, max as spark_max
)
from pyspark.sql.types import (
    StructType, StructField, StringType, DoubleType, TimestampType
)

spark = SparkSession.builder \
    .appName("RealTimeAnalytics") \
    .getOrCreate()

# Define the schema for incoming events
event_schema = StructType([
    StructField("user_id", StringType()),
    StructField("event_type", StringType()),
    StructField("page_url", StringType()),
    StructField("amount", DoubleType()),
    StructField("timestamp", TimestampType()),
])

# Read from Kafka – this creates an unbounded streaming DataFrame
raw_stream = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "kafka:9092") \
    .option("subscribe", "user-events") \
    .option("startingOffsets", "latest") \
    .load()

# Parse JSON messages into structured columns
events = raw_stream \
    .selectExpr("CAST(value AS STRING) as json_str") \
    .select(from_json(col("json_str"), event_schema).alias("event")) \
    .select("event.*")

# Compute 5-minute windowed aggregations:
# For each 5-minute window, count events, unique users, and total revenue.
# The watermark ("10 minutes") tells Spark to wait up to 10 minutes
# for late-arriving events before finalizing a window.
windowed_stats = events \
    .withWatermark("timestamp", "10 minutes") \
    .groupBy(
        window("timestamp", "5 minutes"),
        "event_type"
    ) \
    .agg(
        count("*").alias("event_count"),
        avg("amount").alias("avg_amount"),
    )

# Write results to console (in production: write to a database or dashboard)
query = windowed_stats.writeStream \
    .outputMode("update") \
    .format("console") \

```

```

.option("truncate", False) \
.trigger(processingTime="30 seconds") \
.start()

query.awaitTermination()

```

This code computes sliding-window analytics on a continuous stream. The watermark handles late events — if an event arrives 8 minutes late, it's still counted in the correct window. If it arrives 12 minutes late, it's dropped.

When to Use Each

FRAMEWORK	BEST FOR	TRADE-OFF
Apache Flink	Complex stateful processing, exactly-once semantics, event-time processing	Steeper learning curve, separate cluster
Kafka Streams	Moderate complexity, Java/Kotlin teams, no separate cluster	Limited to Kafka ecosystem, less powerful windowing
Spark Structured Streaming	Teams already using Spark, batch+stream unified API	Micro-batch (not true streaming), higher latency floor

Real-Time Analytics Engines

Sometimes you don't need to *process* a stream — you need to *query* it. Real-time analytics engines ingest streaming data and make it queryable with sub-second latency.

ClickHouse: Open-source columnar database designed for real-time analytical queries. Ingests millions of rows per second and queries billions of rows in milliseconds. Popular for log analysis, user analytics, and real-time dashboards.

Apache Druid: Pre-aggregates data on ingestion, enabling sub-second OLAP queries. Designed for time-series event data. Used by companies like Airbnb, Netflix, and Walmart for user-facing analytics.

Apache Pinot: Similar to Druid but designed at LinkedIn for user-facing analytics at massive scale. Low-latency queries on freshly ingested data. Powers LinkedIn's "Who Viewed Your Profile" and similar features.

When to use these vs. a warehouse: Use a real-time analytics engine when you need sub-second query latency on data that's seconds or minutes old. Use a warehouse (Snowflake, BigQuery) when queries can tolerate minutes to hours of latency and you need more complex SQL support, joins, and ad-hoc exploration.

Event Sourcing and CQRS

Event Sourcing

Traditional systems store current state: "Account balance is \$500." Event sourcing stores every state change: "Deposited \$200, withdrew \$50, deposited \$350." The current state is derived by replaying all events.

Why this matters: - **Complete audit trail**: You know exactly how the system got to its current state - **Time travel**: Reconstruct the state at any point in time - **Debugging**: Replay events to reproduce bugs - **Flexibility**: Build new views of the same data by replaying events through new logic

```

from dataclasses import dataclass, field
from datetime import datetime
from typing import List

@dataclass
class Event:
    event_type: str
    data: dict
    timestamp: datetime = field(default_factory=datetime.utcnow)

class BankAccount:
    """An event-sourced bank account.

    Instead of storing the balance directly, we store every transaction
    as an event. The balance is computed by replaying all events.
    """

    def __init__(self, account_id: str):
        self.account_id = account_id
        self.events: List[Event] = []

    def deposit(self, amount: float, description: str = ""):
        event = Event(
            event_type="deposited",
            data={"amount": amount, "description": description}
        )
        self.events.append(event)

    def withdraw(self, amount: float, description: str = ""):
        if amount > self.balance:
            raise ValueError("Insufficient funds")
        event = Event(
            event_type="withdrawn",
            data={"amount": amount, "description": description}
        )
        self.events.append(event)

    @property
    def balance(self) -> float:
        """Derive current balance from event history."""
        balance = 0.0
        for event in self.events:
            if event.event_type == "deposited":
                balance += event.data["amount"]
            elif event.event_type == "withdrawn":
                balance -= event.data["amount"]
        return balance

    @property
    def statement(self) -> List[dict]:
        """Generate a complete statement from events."""
        running_balance = 0.0
        entries = []
        for event in self.events:

```

```

if event.event_type == "deposited":
    running_balance += event.data["amount"]
elif event.event_type == "withdrawn":
    running_balance -= event.data["amount"]
entries.append({
    "date": event.timestamp.isoformat(),
    "type": event.event_type,
    "amount": event.data["amount"],
    "balance": running_balance,
})
return entries

```

CQRS (Command Query Responsibility Segregation)

CQRS separates the write model (commands) from the read model (queries). Instead of one database serving both reads and writes, you have optimized models for each:

- **Write side:** Optimized for accepting commands and storing events. Might use an event store or append-only log.
- **Read side:** Optimized for queries. Might be a denormalized table, a search index, or a materialized view. Updated asynchronously from the write side's events.

This is powerful when reads and writes have very different requirements — for example, writes need ACID guarantees while reads need sub-millisecond lookups across many dimensions.

Key Takeaway: Event sourcing and CQRS add complexity. Use them when you genuinely need an audit trail, time travel, or very different read/write optimization. For most CRUD applications, a regular database is simpler and sufficient.

Case Study: Real-Time Fraud Detection Pipeline

Let's design a fraud detection system that scores every credit card transaction in under 100 milliseconds.

Architecture

```

[Transaction] → [Kafka] → [Feature Enrichment (Flink)] → [ML Scoring] → [Decision]
                    ↑                               ↓
                [Feature Store (Redis)]       [Model Registry]

```

1. **Ingestion:** Every transaction arrives on a Kafka topic, partitioned by `user_id` for ordering guarantees.

2. **Feature Enrichment:** A Flink job enriches each transaction with features from Redis: user's average transaction amount (30-day), number of transactions in the last hour, distance from the user's typical location, device fingerprint reputation score.
3. **ML Scoring:** The enriched transaction is scored by an ML model (served via a low-latency gRPC endpoint). The model outputs a fraud probability between 0 and 1.
4. **Decision Logic:** Based on the score and business rules:
 5. Score < 0.3: Approve immediately
 6. Score 0.3-0.7: Approve but flag for manual review
 7. Score > 0.7: Decline and alert the fraud team
8. **Feedback Loop:** Confirmed fraud cases and false positives are fed back into model training. The feature store is updated in real-time with the latest transaction data.

The 100ms budget: Feature lookup from Redis (~5ms) + ML inference (~30ms) + Kafka overhead (~15ms) + Flink processing (~20ms) + decision logic (~5ms) = ~75ms, leaving headroom for network variability.

Key Takeaway: Real-time systems require strict latency budgets. Break the total budget into components, and ensure each component has headroom. The latency budget shapes every technology choice — you can't use a technology that takes 200ms in a 100ms pipeline, no matter how good its other features are.

What's Next

Now that you understand both batch and stream processing, Module 6 covers where all this processed data goes for analysis: the data warehouse. We'll dive deep into Snowflake, BigQuery, and Redshift — how they work internally, how to optimize queries, and how to design a warehouse that serves your entire organization.

Module 6: Data Warehouse Architecture

What is a Data Warehouse?

A data warehouse is a central repository where structured data from across your organization is collected, integrated, and stored for analytical querying. It's the single source of truth for business intelligence — the place where analysts, executives, and data scientists go to answer questions about the business.

How it differs from a regular database: your transactional database (PostgreSQL, MySQL) is optimized for running your application — fast point lookups, concurrent writes, ACID transactions. A data warehouse is optimized for *analytical queries* — scanning millions of rows, computing aggregations, joining large tables. Trying to run heavy analytics on your production database is like using a sports car to haul freight: technically possible, but a terrible idea that will slow down both workloads.

The data warehouse landscape has shifted dramatically over the past decade. On-premises warehouses (Teradata, Oracle, Netezza) required massive upfront investment and a dedicated team to manage. Cloud warehouses (Snowflake, BigQuery, Redshift) flipped the model: no infrastructure to manage, elastic compute, and pay-as-you-go pricing.

Key Takeaway: A data warehouse isn't just a "big database." It's architecturally different — optimized for scanning, aggregation, and complex joins on large datasets, with separate compute and storage, columnar format, and MPP (Massively Parallel Processing) execution.

Cloud Warehouse Deep Dive

Snowflake

Snowflake's architecture has three layers that scale independently:

Storage Layer: Data is stored in a proprietary columnar format on cloud object storage (S3, GCS, or Azure Blob). You pay for storage regardless of compute usage. Data is automatically compressed and organized into **micro-partitions** — contiguous chunks of 50-500 MB that contain metadata (min/max values per column) for efficient pruning.

Compute Layer: Virtual warehouses are clusters of compute nodes that execute queries. The key insight: compute and storage are fully separated. You can spin up multiple warehouses of different

sizes, each querying the same data simultaneously. A small warehouse for ad-hoc queries, a large one for ETL, and a medium one for dashboards — all hitting the same tables without interference.

Cloud Services Layer: Handles authentication, query parsing, optimization, metadata management, and transaction coordination. This runs 24/7 and is included in your Snowflake bill (though it's typically a small percentage).

Killer features: Zero-copy cloning (instantly create a full copy of a table or database for development/testing — no additional storage until data diverges), time travel (query data as it existed up to 90 days ago), automatic clustering (Snowflake reorganizes micro-partitions in the background).

BigQuery

BigQuery is Google's serverless warehouse, built on two internal technologies:

Dremel: The distributed query execution engine. It uses a tree architecture where a root node distributes query fragments to thousands of leaf nodes, each scanning a portion of the data in parallel. This is why BigQuery can scan petabytes in seconds.

Colossus: Google's distributed file system that stores data in a columnar format called Capacitor. Data is automatically replicated and encrypted.

Serverless model: There are no clusters to manage. You submit a query, BigQuery allocates resources, runs it, and you pay for the data scanned. This is ideal for variable workloads — you're not paying for idle compute during off-hours.

Pricing options: On-demand (\$5 per TB scanned — great for variable/exploratory use) or flat-rate slots (reserved compute capacity — better for predictable, heavy workloads).

Amazon Redshift

Redshift is the oldest major cloud warehouse. It uses a **node-based architecture** — you choose a cluster size and node type. This is closer to traditional provisioning but offers Redshift Serverless as a newer option.

Strengths: Deep AWS integration, lower cost for predictable workloads with reserved instances, familiar PostgreSQL-compatible SQL.

Limitations: Less elastic than Snowflake (resizing a cluster takes time), manual vacuum/analyze required for optimal performance.

Comparison

FEATURE	SNOWFLAKE	BIGQUERY	REDSHIFT
Architecture	Separate compute/storage	Serverless	Node-based (+ serverless option)
Scaling	Instant (add/resize warehouses)	Automatic	Minutes to resize
Pricing model	Per-second compute + storage	Per-TB scanned or flat-rate	Per-node-hour + storage
Best for	Multi-workload, multi-team	Variable/unpredictable queries	Predictable AWS workloads
Concurrency	Excellent (multi-cluster)	Excellent (auto-scaled)	Good (with WLM tuning)
Ecosystem	Multi-cloud	GCP-native	AWS-native

Partitioning and Clustering

As data volumes grow, full table scans become expensive and slow. **Partitioning** and **clustering** let the query engine skip irrelevant data.

Partitioning

Partitioning divides a table into segments based on a column value — typically a date. When a query filters on the partition column, the engine only reads the relevant partitions, skipping the rest entirely.

In BigQuery, partition pruning can turn a multi-terabyte scan into a few gigabytes:

```

-- Create a partitioned table in BigQuery
CREATE TABLE analytics.user_events (
  event_id      STRING,
  user_id       STRING,
  event_type    STRING,
  event_data    JSON,
  event_date    DATE
)
PARTITION BY event_date
CLUSTER BY user_id, event_type;

-- This query only scans partitions for April 2026 – not the full table.
-- If the table has 3 years of data, you just skipped ~97% of it.
SELECT event_type, COUNT(*) as event_count
FROM analytics.user_events
WHERE event_date BETWEEN '2026-04-01' AND '2026-04-30'
GROUP BY event_type;

```

Clustering

Clustering sorts data within each partition by specified columns. When a query filters on the clustering columns, the engine can skip blocks of data within a partition. In the example above, clustering by `user_id, event_type` means queries filtering on a specific user or event type will be fast even within a single day's partition.

Snowflake's approach: Snowflake automatically clusters micro-partitions based on the order data was loaded. You can define explicit clustering keys for tables where the natural insert order doesn't match query patterns:

```

-- Snowflake: define clustering keys for optimal query pruning
ALTER TABLE analytics.user_events
CLUSTER BY (event_date, user_id);

-- Check clustering quality
SELECT SYSTEM$CLUSTERING_INFORMATION('analytics.user_events');

```

Key Takeaway: Always partition time-series data by date — it's the single most impactful optimization for analytical tables. Add clustering on frequently filtered columns (`user_id`, `region`, `product_category`) for additional speedup.

Materialized Views and Query Optimization

Materialized Views

A materialized view is a pre-computed query result stored as a table. Instead of running an expensive aggregation every time someone loads a dashboard, you compute it once and serve from the materialized view.

```
-- Create a materialized view for daily revenue by category
-- This query runs once. Dashboards read from the view instantly.
CREATE MATERIALIZED VIEW analytics.daily_revenue_by_category AS
SELECT
    DATE_TRUNC('day', order_date) AS day,
    p.category,
    COUNT(DISTINCT o.order_id) AS order_count,
    SUM(oi.quantity * oi.unit_price) AS revenue,
    COUNT(DISTINCT o.customer_id) AS unique_customers
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
GROUP BY 1, 2;

-- Dashboards query the materialized view - sub-second response
SELECT day, category, revenue
FROM analytics.daily_revenue_by_category
WHERE day >= CURRENT_DATE - INTERVAL '30 days'
ORDER BY day, revenue DESC;
```

When to use materialized views vs. caching vs. pre-aggregation tables: - **Materialized views:**

Automated refresh, query optimizer can redirect queries automatically. Best for standard aggregations that many users run. - **Result caching:** Warehouse-level caching of identical queries. Zero cost but only helps when the exact same query is repeated. - **Pre-aggregation tables:** Manually managed summary tables built by your ETL pipeline. More control but more maintenance. Best for complex transformations the warehouse can't express as a materialized view.

Query Optimization Techniques

1. Avoid SELECT *: Only select the columns you need. In a columnar warehouse, every extra column is additional I/O.

```
-- BAD: Reads all 50 columns from disk
SELECT * FROM user_events WHERE event_date = '2026-04-01';

-- GOOD: Reads only 3 columns - potentially 15x less I/O
SELECT user_id, event_type, event_data
FROM user_events
WHERE event_date = '2026-04-01';
```

2. Filter early: Push filters as close to the source tables as possible. Join smaller result sets.

3. Use approximate functions: When exact counts aren't needed, `APPROX_COUNT_DISTINCT()` is much faster than `COUNT(DISTINCT)` on large datasets.

```
-- Exact distinct count – may scan billions of values
SELECT COUNT(DISTINCT user_id) FROM user_events;

-- Approximate – ~2% error but 10-100x faster
SELECT APPROX_COUNT_DISTINCT(user_id) FROM user_events;
```

4. Understand your query execution plan: Use `EXPLAIN` to see how the warehouse plans to execute your query. Look for full table scans on large tables, missing partition pruning, and inefficient join orders.

Multi-Tenant Analytics

In most organizations, multiple teams share the same data warehouse. Product analytics, marketing, finance, data science — each with different query patterns, SLAs, and cost sensitivities. Designing for multiple tenants requires balancing isolation with efficiency.

Workload Isolation

In Snowflake, create separate virtual warehouses per team or workload type:

```
-- ETL warehouse: large, runs at night, auto-suspends quickly
CREATE WAREHOUSE etl_warehouse
  WAREHOUSE_SIZE = 'X-LARGE'
  AUTO_SUSPEND = 60
  AUTO_RESUME = TRUE;

-- Analyst warehouse: medium, available during business hours
CREATE WAREHOUSE analyst_warehouse
  WAREHOUSE_SIZE = 'MEDIUM'
  AUTO_SUSPEND = 300
  AUTO_RESUME = TRUE;

-- Dashboard warehouse: small but always on for consistent performance
CREATE WAREHOUSE dashboard_warehouse
  WAREHOUSE_SIZE = 'SMALL'
  AUTO_SUSPEND = 0
  AUTO_RESUME = TRUE;
```

This ensures the nightly ETL job doesn't slow down analyst queries, and a data scientist running an expensive query doesn't affect dashboard load times.

Cost Allocation

Track warehouse usage by team using resource monitors and tags:

```
-- Snowflake resource monitor: alert and suspend at spending limits
CREATE RESOURCE MONITOR marketing_budget
  WITH CREDIT_QUOTA = 500
  TRIGGERS
    ON 80 PERCENT DO NOTIFY
    ON 100 PERCENT DO SUSPEND;

ALTER WAREHOUSE marketing_warehouse
  SET RESOURCE_MONITOR = marketing_budget;
```

Row-Level Security

When different teams or customers should see different subsets of data, use row-level security policies:

```
-- Only show each team their own data
CREATE ROW ACCESS POLICY team_data_policy AS (team_id INT)
RETURNS BOOLEAN ->
  team_id = CURRENT_ROLE()::INT
  OR IS_ROLE_IN_SESSION('DATA_ADMIN');
```

Key Takeaway: Design your warehouse for multi-tenancy from the start. Separate workloads by compute (different warehouses), control costs with resource monitors, and enforce data access with row-level security. Retrofitting these later is painful.

Case Study: Designing a Multi-Team Analytics Warehouse

A mid-sized SaaS company with three main analytics consumers: **Product** (user behavior analysis), **Marketing** (campaign attribution, funnel analysis), and **Finance** (revenue recognition, forecasting).

Schema Design

Use a layered approach:

- **Raw layer** (`raw.*`): Direct copies of source data, minimal transformation. Owned by the data engineering team.
- **Staging layer** (`staging.*`): Cleaned and standardized data. Consistent naming, data types, and deduplication. Still resembles source structure.

- **Analytics layer** (`analytics.*`): Business-logic-enriched dimensional models. Star schemas with fact and dimension tables. This is what teams query.
- **Team layers** (`product.*`, `marketing.*`, `finance.*`): Team-specific models built on the analytics layer. Each team owns their models and can iterate independently.

```
-- Analytics layer: shared fact table all teams use
CREATE TABLE analytics.fact_user_activity (
  activity_id      BIGINT,
  activity_date    DATE,
  user_key         INT REFERENCES analytics.dim_users,
  activity_type    STRING,
  feature_used     STRING,
  session_id      STRING,
  duration_sec     INT,
  revenue_impact   DECIMAL(10,2)
);

-- Product team layer: their own aggregation
CREATE TABLE product.feature_adoption AS
SELECT
  feature_used,
  DATE_TRUNC('week', activity_date) AS week,
  COUNT(DISTINCT user_key) AS unique_users,
  SUM(duration_sec) / 3600.0 AS total_hours
FROM analytics.fact_user_activity
GROUP BY 1, 2;

-- Marketing team layer: their own attribution model
CREATE TABLE marketing.campaign_attribution AS
SELECT
  c.campaign_id,
  c.campaign_name,
  COUNT(DISTINCT a.user_key) AS attributed_users,
  SUM(a.revenue_impact) AS attributed_revenue
FROM analytics.fact_user_activity a
JOIN marketing.campaign_touchpoints c
  ON a.user_key = c.user_key
  AND a.activity_date BETWEEN c.start_date AND c.end_date
GROUP BY 1, 2;
```

This layered design gives each team autonomy over their models while maintaining a shared, governed foundation. The analytics layer is the “contract” — stable, well-tested, and owned by the data team.

What's Next

Data warehouses are powerful but store only structured, curated data. Module 7 explores data lakes and the lakehouse architecture — how to handle raw, semi-structured, and unstructured data alongside your warehouse, and how modern lakehouse formats are blurring the line between the two.

Module 7: Data Lakes & Lakehouse Architecture

The Data Lake Promise (and Pitfalls)

The data lake concept was revolutionary: dump *everything* into cheap object storage — structured tables, JSON logs, images, videos, sensor data — and figure out how to use it later. At a fraction of the cost of a warehouse, you'd have all your data in one place, ready for any analysis.

The reality? Many data lakes became **data swamps** — vast repositories of undocumented, ungoverned data that nobody could find or trust. Without schema enforcement, without access controls, without lineage tracking, the lake filled with duplicates, stale data, and mysterious files that nobody remembered creating.

The problem wasn't the concept — it was the lack of governance. A data lake without cataloging, quality checks, and access controls is just a very expensive file share.

Key Takeaway: The data lake is still the right foundation for modern data architecture — object storage is too cheap and flexible to ignore. But success requires treating it as an engineered system with zones, schemas, quality gates, and governance, not as a dumping ground.

Zone-Based Architecture

The solution to the data swamp is organizing the lake into zones with clear purposes, ownership, and quality expectations.

Raw / Landing Zone

Purpose: Exact copies of source data, as-is. No transformations, no cleaning. This is your audit trail and reprocessing safety net.

Contents: CSV files from SFTP drops, JSON from API extractions, Parquet files from database exports, log files, images, PDFs.

Rules: Append-only (never modify), partitioned by source and ingestion date, retained for 1-7 years depending on compliance requirements.

```
s3://data-lake/raw/  
├─ postgres_orders/dt=2026-04-01/orders.parquet  
├─ stripe_payments/dt=2026-04-01/payments.json  
├─ web_events/dt=2026-04-01/events.json.gz  
└─ support_tickets/dt=2026-04-01/tickets.csv
```

Cleansed / Curated Zone

Purpose: Cleaned, validated, and standardized data. Consistent column names, correct data types, deduplication applied, schema enforced.

Contents: Parquet/Delta files with enforced schemas. Each dataset has an owner and a documented SCD (slowly changing dimension) strategy.

Rules: Schema-on-write enforced, quality checks must pass before data lands here, versioned with change tracking.

Analytics / Consumption Zone

Purpose: Business-ready datasets optimized for specific use cases. Aggregations, feature tables, materialized views.

Contents: Star schemas for BI tools, feature tables for ML, pre-aggregated metrics for dashboards.

Rules: Tight access controls, SLA on freshness, documented lineage from raw to consumption.

The data flows through zones like water through a treatment plant: raw water (landing) → filtered and treated (cleansed) → safe to drink (consumption). Each stage adds quality and removes impurities.

The Lakehouse Revolution

The traditional architecture forced a choice: data lake (cheap, flexible, but no transactions or governance) or data warehouse (reliable, governed, but expensive and limited to structured data). Most companies ran both, connected by ETL pipelines that copied data from lake to warehouse.

The **lakehouse** eliminates this duplication by adding warehouse-like capabilities directly to the data lake: ACID transactions, schema enforcement, indexing, and governance — all on top of cheap object storage.

The Medallion Architecture (Bronze / Silver / Gold)

The most popular lakehouse pattern, pioneered by Databricks:

Bronze Layer (Raw): Ingested data with minimal processing. Append-only, preserves the original format. Includes metadata columns like ingestion timestamp and source.

Silver Layer (Cleansed): Cleaned, validated, and conformed data. Deduplication, type casting, null handling, schema enforcement. This is your “single source of truth.”

Gold Layer (Business): Aggregated, business-ready datasets. Star schemas, feature tables, metric tables. Optimized for specific consumers.

```

from delta.tables import DeltaTable
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, current_timestamp, when, trim, lower

spark = SparkSession.builder.appName("Medallion").getOrCreate()

# --- Bronze: Ingest raw data as-is, add metadata ---
def ingest_to_bronze(source_path: str, bronze_path: str):
    """Load raw data into the bronze layer.

    We add ingestion metadata but don't modify the source data.
    This preserves the original record for auditing and reprocessing.
    """
    raw_df = spark.read.json(source_path)
    bronze_df = raw_df \
        .withColumn("_ingested_at", current_timestamp()) \
        .withColumn("_source_file", col("_metadata.file_path"))

    bronze_df.write \
        .format("delta") \
        .mode("append") \
        .save(bronze_path)

# --- Silver: Clean and validate ---
def bronze_to_silver(bronze_path: str, silver_path: str):
    """Transform bronze data into clean, validated silver records.

    This is where data quality happens: type casting, null handling,
    deduplication, and standardization. If a record fails validation,
    it goes to a quarantine table instead of silver.
    """
    bronze_df = spark.read.format("delta").load(bronze_path)

    silver_df = bronze_df \
        .dropDuplicates(["order_id"]) \
        .filter(col("order_id").isNotNull()) \
        .withColumn("customer_email", lower(trim(col("customer_email")))) \
        .withColumn("order_total", col("order_total").cast("decimal(10,2)")) \
        .withColumn("status", when(
            col("status").isin("pending", "completed", "cancelled"),
            col("status")
        ).otherwise("unknown"))

    # Upsert: update existing records, insert new ones
    if DeltaTable.isDeltaTable(spark, silver_path):
        delta_table = DeltaTable.forPath(spark, silver_path)
        delta_table.alias("target").merge(
            silver_df.alias("source"),
            "target.order_id = source.order_id"
        ).whenMatchedUpdateAll() \
        .whenNotMatchedInsertAll() \
        .execute()
    else:
        silver_df.write.format("delta").save(silver_path)

```

```
# --- Gold: Business-ready aggregations ---
def silver_to_gold(silver_path: str, gold_path: str):
    """Build business-ready aggregations from silver data.

    Gold tables are optimized for specific consumers: dashboards,
    reports, ML features. They denormalize and pre-aggregate
    to minimize query complexity and latency.
    """
    silver_df = spark.read.format("delta").load(silver_path)

    daily_metrics = silver_df \
        .filter(col("status") == "completed") \
        .groupBy("order_date", "product_category") \
        .agg(
            {"order_total": "sum", "order_id": "countDistinct",
             "customer_id": "countDistinct"}
        ) \
        .withColumnRenamed("sum(order_total)", "revenue") \
        .withColumnRenamed("count(DISTINCT order_id)", "order_count") \
        .withColumnRenamed("count(DISTINCT customer_id)", "unique_customers")

    daily_metrics.write \
        .format("delta") \
        .mode("overwrite") \
        .option("overwriteSchema", "true") \
        .save(gold_path)
```

Lakehouse Technologies Compared

Delta Lake

Created by Databricks, Delta Lake is the most widely adopted lakehouse format. It stores data as Parquet files with a JSON-based transaction log ([_delta_log/](#)). Every operation creates a new log entry, enabling ACID transactions, time travel, and audit history.

Strengths: Tight Spark integration, mature tooling, large community, Z-ordering for multi-dimensional clustering, Change Data Feed for CDC.

Apache Iceberg

Created at Netflix, Iceberg takes a different approach to metadata management. Instead of a simple transaction log, it maintains a hierarchy of metadata files, manifest lists, and manifest files that provide efficient metadata operations even at massive scale.

Killer feature — Hidden partitioning: Users don't need to know the partitioning scheme. If a table is partitioned by `month(event_date)`, you just query `WHERE event_date = '2026-04-01'` and Iceberg automatically prunes to the April partition. No need for synthetic partition columns.

Killer feature — Partition evolution: Change your partitioning strategy without rewriting data. Start with monthly partitions; switch to daily when volume grows. Old data keeps its old partitioning; new data uses the new scheme. Queries seamlessly span both.

Apache Hudi

Created at Uber for near-real-time data lake ingestion. Hudi's strength is its two storage types that let you optimize for your read/write balance:

- **Copy-on-Write:** Every update rewrites the affected file. Read-optimized — queries always read clean Parquet files.
- **Merge-on-Read:** Updates go to a delta log; reads merge base files with deltas. Write-optimized — updates are fast, but reads do more work.

When to Choose Which

IF YOU...	CHOOSE
Use Databricks	Delta Lake (native integration)
Need multi-engine support (Spark + Trino + Flink)	Iceberg (best multi-engine story)
Need near-real-time upserts from CDC	Hudi (designed for this use case)
Need partition evolution	Iceberg (best-in-class)
Want the largest community	Delta Lake

Performance Optimization

File Compaction

Over time, streaming ingestion and frequent updates create many small files. Small files hurt query performance because each file has overhead (opening the file, reading metadata, scheduling work).

Compaction merges small files into larger, optimally-sized ones:

```
# Delta Lake: compact small files (OPTIMIZE)
spark.sql("OPTIMIZE silver.orders")

# With Z-ordering: co-locate related data for faster multi-column filters
spark.sql("OPTIMIZE silver.orders ZORDER BY (customer_id, order_date)")
```

Z-Ordering

Z-ordering is a multi-dimensional clustering technique. It co-locates rows with similar values across multiple columns in the same files, so filters on any of those columns can skip more files. It's especially useful when queries filter on different combinations of columns.

Data Skipping

Modern table formats store min/max statistics per column per file. When a query has a filter like `WHERE amount > 1000`, the engine checks each file's statistics and skips files where `max(amount) < 1000`. More data skipping = less I/O = faster queries.

Key Takeaway: Lakehouse performance depends on physical data layout. Compact small files regularly, use Z-ordering on frequently filtered columns, and monitor data skipping effectiveness. A well-organized lakehouse can match warehouse query performance at a fraction of the cost.

Multi-Format Data Handling

A key advantage of lakehouses over warehouses is handling data variety:

Structured data (CSVs, database exports): Convert to Parquet/Delta on ingestion. This is the most common and best-supported path.

Semi-structured data (JSON, XML, Avro): Modern engines handle nested and semi-structured data natively. Spark and BigQuery can query JSON fields directly, and Iceberg supports nested types in its schema.

Unstructured data (images, PDFs, audio): Store the raw files in object storage. Create metadata tables in the lakehouse that reference the file paths and contain extracted features (image embeddings, OCR text, audio transcriptions). Query the metadata to find relevant files.

```
# Semi-structured: query nested JSON fields directly
events = spark.read.format("delta").load("s3://lake/silver/events")

# Extract nested fields without flattening the entire structure
events.select(
    col("event_id"),
    col("user.name").alias("user_name"),
    col("user.preferences.theme").alias("theme"),
    col("metadata.device_type").alias("device"),
).filter(col("metadata.device_type") == "mobile")
```

Case Study: Pinterest's Data Lakehouse

Pinterest processes over 1 billion events per day from 450+ million monthly active users. They migrated from a traditional Hadoop-based data lake to a lakehouse architecture to solve three problems:

1. **Data quality:** Raw files had no schema enforcement, leading to silent data corruption that wasn't caught until dashboards showed wrong numbers days later.
2. **Slow queries:** Small files from streaming ingestion meant queries read thousands of tiny files. A query that should take 30 seconds took 10 minutes.
3. **No updates/deletes:** GDPR required deleting user data on request. With plain Parquet files, "deleting" a user meant rewriting entire partitions — expensive and slow.

Solution: They adopted Apache Iceberg as their table format, which gave them ACID transactions (schema enforcement and consistent reads), hidden partitioning (simpler queries, automatic pruning), and row-level deletes (GDPR compliance without rewriting entire partitions).

Results: Query performance improved 3-5x from better file organization and data skipping. GDPR deletion requests went from hours to minutes. Data quality issues dropped by 80% with schema enforcement. Storage costs decreased 30% from better compression and reduced data duplication.

Key Takeaway: The lakehouse isn't just a theoretical improvement — it solves real operational problems: data quality, query performance, compliance, and cost. If you're running a data lake today, adopting a table format (Delta, Iceberg, or Hudi) is the single highest-ROI improvement you can make.

What's Next

With storage and processing covered, Module 8 shifts to one of the fastest-growing areas in data engineering: ML platforms and feature stores. You'll learn how to bridge the gap between experimental ML in notebooks and production ML serving millions of predictions per day.

Module 8: ML Platforms & Feature Stores

Why ML Infrastructure Matters

There's a well-known gap in machine learning: the distance between a model that works in a Jupyter notebook and a model that serves predictions reliably in production. Data scientists often joke that building the model is 10% of the work — the other 90% is getting it deployed, monitored, and maintained.

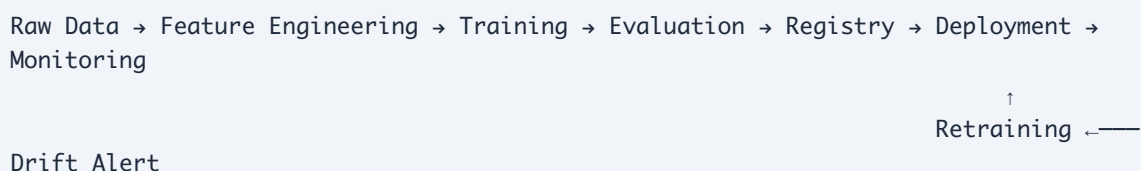
The core problem is **training-serving skew**: the features your model sees during training are different from what it sees in production. Maybe your training pipeline computes features using Pandas on a CSV dump, but your serving pipeline uses SQL queries against a live database. Subtle differences — timezone handling, null behavior, aggregation windows — cause the model to perform differently in production than it did in your notebook.

ML infrastructure exists to close this gap: ensuring that the same features, computed the same way, are used in both training and serving; that models are versioned and can be rolled back; and that you know when model performance degrades.

Key Takeaway: The value of ML isn't in the model — it's in the system around it. Feature stores, model registries, serving infrastructure, and monitoring are what separate “cool demo” from “business impact.”

ML Pipelines

A production ML pipeline is a series of automated steps that turn raw data into deployed predictions:



Each step must be reproducible, versioned, and observable. If a model's performance degrades next month, you need to know exactly what data it trained on, what features it used, and what hyperparameters were set.

MLflow is the most widely used platform for managing this lifecycle. It tracks experiments, packages models, and provides a registry for versioning:

```
import mlflow
import mlflow.sklearn
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score, precision_score, recall_score

# Every experiment is tracked: parameters, metrics, artifacts
mlflow.set_experiment("churn_prediction_v3")

with mlflow.start_run(run_name="gradient_boosting_tuned"):
    # Log the parameters so we can reproduce this exact training run
    params = {
        "n_estimators": 200,
        "max_depth": 6,
        "learning_rate": 0.1,
        "min_samples_split": 20,
    }
    mlflow.log_params(params)

    # Train
    model = GradientBoostingClassifier(**params)
    X_train, X_test, y_train, y_test = train_test_split(
        features, labels, test_size=0.2, random_state=42
    )
    model.fit(X_train, y_train)

    # Evaluate and log metrics
    predictions = model.predict(X_test)
    probabilities = model.predict_proba(X_test)[:, 1]

    mlflow.log_metric("auc_roc", roc_auc_score(y_test, probabilities))
    mlflow.log_metric("precision", precision_score(y_test, predictions))
    mlflow.log_metric("recall", recall_score(y_test, predictions))

    # Save the model to the registry with versioning
    mlflow.sklearn.log_model(
        model,
        artifact_path="model",
        registered_model_name="churn_predictor",
    )
```

After training, you can compare runs in MLflow's UI, promote the best model to "Production" stage, and your serving infrastructure automatically picks up the new version.

Feature Engineering and Feature Stores

The Problem

A “feature” is an input to an ML model — a numeric or categorical value computed from raw data.

Examples: `avg_purchase_amount_30d`, `days_since_last_login`, `num_support_tickets_7d`.

The problem: different teams compute the same features differently. The fraud team calculates `transaction_velocity_1h` one way; the risk team calculates it another way. Both train models that work in notebooks but disagree in production.

A **feature store** is a centralized system that:

- Defines features once, uses them everywhere
- Serves features consistently for both training and inference
- Provides an **online store** (Redis/DynamoDB) for low-latency serving (<10ms)
- Provides an **offline store** (S3/warehouse) for training data generation
- Tracks lineage: which models use which features, computed from which data sources

Feast: An Open-Source Feature Store

Here’s how you define and use features with Feast. The feature definition serves as the single source of truth — both training and serving use this definition:

```

from feast import Entity, FeatureView, Field, FileSource
from feast.types import Float32, Int64, String
from datetime import timedelta

# Define the entity (the "who" or "what" features are about)
customer = Entity(
    name="customer_id",
    join_keys=["customer_id"],
    description="Unique customer identifier",
)

# Define the data source
customer_features_source = FileSource(
    path="s3://feature-store/customer_features.parquet",
    timestamp_field="event_timestamp",
)

# Define features with metadata and types
customer_behavior = FeatureView(
    name="customer_behavior",
    entities=[customer],
    ttl=timedelta(days=1), # Features expire after 1 day
    schema=[
        Field(name="total_purchases_30d", dtype=Int64),
        Field(name="avg_order_value_30d", dtype=Float32),
        Field(name="days_since_last_purchase", dtype=Int64),
        Field(name="support_tickets_7d", dtype=Int64),
        Field(name="preferred_category", dtype=String),
    ],
    source=customer_features_source,
)

# --- Using features for training ---
from feast import FeatureStore

store = FeatureStore(repo_path="./feature_repo")

# Get historical features for training - point-in-time correct
# This ensures no data leakage: for each training example at time T,
# only features available before T are used.
training_data = store.get_historical_features(
    entity_df=training_entities, # DataFrame with customer_id + timestamp
    features=[
        "customer_behavior:total_purchases_30d",
        "customer_behavior:avg_order_value_30d",
        "customer_behavior:days_since_last_purchase",
    ],
).to_df()

# --- Using features for real-time inference ---
# Same features, same computation, consistent with training
online_features = store.get_online_features(
    features=[
        "customer_behavior:total_purchases_30d",

```

```
        "customer_behavior:avg_order_value_30d",
        "customer_behavior:days_since_last_purchase",
    ],
    entity_rows=[{"customer_id": "cust_12345"}],
).to_dict()
```

The critical function here is `get_historical_features` with **point-in-time correctness**. For each training example at time T, it only uses feature values that were available *before* T. Without this, your model trains on future data — a subtle but devastating form of data leakage that makes training metrics look great but production performance terrible.

Model Serving

Once a model is trained and registered, it needs to serve predictions. There are two paradigms:

Batch inference: Score a large dataset all at once, store results in a table. Good for recommendations that update daily, risk scores computed overnight, or any prediction that doesn't need to be real-time.

Real-time inference: Score individual requests on demand with low latency. Required for fraud detection, search ranking, dynamic pricing — anything that needs a prediction in milliseconds.

Here's a real-time model serving endpoint using FastAPI. It loads the model from MLflow, retrieves features from the feature store, and returns predictions:

```

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import mlflow.pyfunc
from feast import FeatureStore

app = FastAPI(title="Churn Prediction API")

# Load the production model from MLflow on startup
model = mlflow.pyfunc.load_model("models:/churn_predictor/Production")
feature_store = FeatureStore(repo_path="./feature_repo")

class PredictionRequest(BaseModel):
    customer_id: str

class PredictionResponse(BaseModel):
    customer_id: str
    churn_probability: float
    risk_level: str # low, medium, high

@app.post("/predict", response_model=PredictionResponse)
def predict_churn(request: PredictionRequest):
    """Score a customer's churn risk in real-time.

    1. Retrieve features from the online feature store (<5ms)
    2. Run model inference (<10ms)
    3. Return prediction with risk categorization
    """
    # Get features – same features used in training, guaranteed consistent
    features = feature_store.get_online_features(
        features=[
            "customer_behavior:total_purchases_30d",
            "customer_behavior:avg_order_value_30d",
            "customer_behavior:days_since_last_purchase",
            "customer_behavior:support_tickets_7d",
        ],
        entity_rows=[{"customer_id": request.customer_id}],
    ).to_dict()

    if not features.get("total_purchases_30d"):
        raise HTTPException(status_code=404, detail="Customer not found")

    # Model inference
    probability = model.predict([features])[0]

    # Categorize risk
    if probability > 0.7:
        risk_level = "high"
    elif probability > 0.3:
        risk_level = "medium"
    else:
        risk_level = "low"

    return PredictionResponse(
        customer_id=request.customer_id,

```

```
    churn_probability=round(probability, 4),  
    risk_level=risk_level,  
)
```

A/B Testing Models

Never deploy a new model to 100% of traffic immediately. Use gradual rollouts:

1. **Shadow mode:** New model runs alongside the old one but doesn't serve results. Compare predictions offline.
2. **Canary deployment:** Route 5% of traffic to the new model. Monitor metrics closely.
3. **Gradual rollout:** Increase traffic to 25%, 50%, 100% over days, monitoring at each step.
4. **Automatic rollback:** If key metrics (prediction latency, error rate, business KPIs) degrade beyond a threshold, automatically revert to the previous model.

ML Monitoring and Observability

A model that works today may not work tomorrow. The world changes — customer behavior shifts, product catalogs evolve, competitors enter the market. **Model drift** means your model's predictions are becoming less accurate over time.

Three types of drift to monitor:

Data drift: The distribution of input features has changed. Example: a model trained on US-only customers starts receiving international traffic with different purchasing patterns.

Concept drift: The relationship between features and the target has changed. Example: a fraud detection model trained pre-COVID sees a surge in online transactions that look “unusual” but aren't fraudulent.

Prediction drift: The distribution of model outputs has shifted. If your churn model suddenly predicts 50% of customers as high-risk (up from 10%), something has changed.

```

from scipy import stats
import numpy as np

def detect_feature_drift(
    reference: np.ndarray,
    current: np.ndarray,
    threshold: float = 0.05
) -> dict:
    """Detect distribution drift using the Kolmogorov-Smirnov test.

    Compares the current feature distribution against the reference
    (typically the training distribution). A p-value below the threshold
    indicates statistically significant drift.
    """
    statistic, p_value = stats.ks_2samp(reference, current)
    return {
        "drifted": p_value < threshold,
        "p_value": p_value,
        "ks_statistic": statistic,
        "severity": "high" if p_value < 0.001 else
                   "medium" if p_value < 0.01 else
                   "low" if p_value < threshold else "none"
    }

# Check drift for each feature weekly
for feature_name in ["avg_order_value", "days_since_login", "session_count"]:
    result = detect_feature_drift(
        reference=training_data[feature_name].values,
        current=this_week_data[feature_name].values,
    )
    if result["drifted"]:
        alert(f"Drift detected in {feature_name}: {result['severity']}")

```

Key Takeaway: Deploy monitoring *before* deploying the model. Track feature distributions, prediction distributions, and business metrics. Set up alerts so you know when the model degrades before your users or stakeholders notice.

Vector Databases and RAG Systems

The rise of large language models has created a new class of data infrastructure: systems for storing and querying high-dimensional **vector embeddings**.

What Are Vector Embeddings?

An embedding is a numeric representation of content — text, images, audio — as a list of numbers (typically 384-1536 dimensions). Similar content produces similar vectors, meaning you can find related items by computing vector distance (cosine similarity, Euclidean distance).

Vector Databases

Traditional databases index by exact values (`WHERE id = 123`). Vector databases index by similarity (`find the 10 most similar vectors to this query vector`). This enables semantic search — finding documents by meaning, not keywords.

Popular options: Pinecone (managed, easy to start), Weaviate (open-source, hybrid search), Milvus (open-source, high performance), pgvector (PostgreSQL extension — good for smaller scale).

Retrieval-Augmented Generation (RAG)

RAG combines a retrieval system with a language model. Instead of asking an LLM to answer from its training data (which may be outdated or wrong), you first retrieve relevant documents from your own data, then include them in the prompt:

```

import faiss
import numpy as np
from sentence_transformers import SentenceTransformer

class SimpleRAG:
    """A basic RAG system using FAISS for vector search.

    1. Embed your documents into vectors
    2. Store vectors in a FAISS index for fast similarity search
    3. At query time, find the most relevant documents
    4. Pass them as context to the LLM
    """

    def __init__(self):
        self.encoder = SentenceTransformer('all-MiniLM-L6-v2')
        self.index = None
        self.documents = []

    def add_documents(self, docs: list[str]):
        """Embed documents and add them to the vector index."""
        self.documents.extend(docs)
        embeddings = self.encoder.encode(docs)
        dimension = embeddings.shape[1]

        if self.index is None:
            self.index = faiss.IndexFlatIP(dimension) # Inner product similarity

        # Normalize for cosine similarity
        faiss.normalize_L2(embeddings)
        self.index.add(embeddings.astype('float32'))

    def query(self, question: str, top_k: int = 3) -> list[str]:
        """Find the most relevant documents for a question."""
        query_embedding = self.encoder.encode([question])
        faiss.normalize_L2(query_embedding)

        scores, indices = self.index.search(
            query_embedding.astype('float32'), top_k
        )
        return [self.documents[i] for i in indices[0]]

    def answer(self, question: str, llm_client) -> str:
        """Retrieve relevant context and generate an answer.

        The LLM only uses the retrieved documents as context,
        so it can answer questions about your specific data
        without having seen it during training.
        """
        relevant_docs = self.query(question, top_k=3)
        context = "\n\n".join(relevant_docs)

        prompt = f"""Answer the question based only on the provided context.
        If the context doesn't contain enough information, say so.

```

```
Context:
{context}

Question: {question}

Answer: ""

return llm_client.generate(prompt)
```

Key Takeaway: RAG is the practical way to give LLMs access to your organization's data without fine-tuning. The quality of your retrieval (embedding model, chunking strategy, vector index) directly determines the quality of the generated answers.

Case Study: Building a Recommendation System

Let's design a product recommendation system end-to-end — from data collection through serving.

Requirements: An e-commerce site with 1M products and 10M users. Generate “you might also like” recommendations on product pages, updated daily, served in <50ms.

Architecture:

```
graph TD
    A[User Events (Kafka) → Feature Pipeline (Spark) → Feature Store (Feast)] --> B[Training Data → Model Training (MLflow) → Model Registry → Serving API (FastAPI)]
    B --> C[Product Page (<50ms)]
```

- Data Collection:** Every user interaction (view, add-to-cart, purchase, wishlist) flows through Kafka. A Spark job computes daily features: user purchase history, product co-purchase matrix, category affinities, price sensitivity.
- Feature Store:** Feast manages two feature views — `user_preferences` (online + offline) and `product_signals` (popularity, average rating, view-to-purchase ratio). The online store (Redis) serves features in <5ms.
- Model Training:** Weekly training job using collaborative filtering (matrix factorization) plus content-based features. MLflow tracks experiments. Best model is promoted to Production stage.
- Serving:** FastAPI endpoint retrieves user features from the feature store, runs the model, applies business rules (filter out-of-stock items, boost sponsored products), and returns top 20 recommendations.

5. **Monitoring:** Track recommendation click-through rate, add-to-cart rate, and revenue attribution. Alert on drift in prediction distribution or feature values. A/B test new models against the current production model.

The total latency budget: feature lookup (5ms) + model inference (15ms) + business rules (5ms) + network (10ms) = 35ms, well within the 50ms target.

What's Next

Module 9 explores event-driven and distributed systems — the architectural patterns that tie everything together when your system grows from a single application to a fleet of microservices communicating through events.

Module 9: Event-Driven & Distributed Systems

Microservices Data Architecture

When a monolithic application splits into microservices, data architecture becomes the hardest problem. In a monolith, different features share one database — they can join tables, use transactions, and maintain consistency easily. In microservices, each service owns its data. That shared database is gone, and with it goes the easy consistency.

The database-per-service pattern is the fundamental rule: each microservice owns its database and no other service accesses it directly. If the Order Service needs customer data, it asks the Customer Service through an API — it doesn't query the customers table.

This seems wasteful, but the alternative is worse. A shared database creates tight coupling: changing a column in the customers table can break the Order Service, the Billing Service, and the Analytics Service simultaneously. With database-per-service, each service can evolve its schema independently.

The challenge: how do services stay in sync without a shared database? The answer is **events**.

Key Takeaway: Database-per-service is non-negotiable for true microservices. The cost is complexity in keeping data synchronized. The benefit is independent deployment, scaling, and evolution of each service.

Event Sourcing Deep Dive

In Module 5, we introduced event sourcing briefly. Now let's go deeper, because it's one of the most powerful — and most misunderstood — patterns in distributed systems.

The Core Idea

Traditional systems store **current state**: "The order status is 'shipped'." Event sourcing stores **every state change**: "Order was created → payment was received → order was packed → order was shipped." The current state is derived by replaying all events.

Why This Matters

1. **Complete audit trail:** Regulators, compliance teams, and debuggers love this. You can answer “what happened to order #12345 and when?” by reading the event log.
2. **Time travel:** Reconstruct the state of any entity at any point in time. “What was this customer’s subscription status on March 15th?” Just replay events up to that date.
3. **New projections:** When you need a new read model — say, a report that didn’t exist when the system was designed — you can create it by replaying historical events through new logic. No data migration needed.

Implementation

An event store is fundamentally an append-only log per entity. Here’s a practical implementation:

```

from dataclasses import dataclass, field
from datetime import datetime
from typing import List, Dict, Any
from enum import Enum
import json

class OrderStatus(Enum):
    CREATED = "created"
    PAID = "paid"
    PACKED = "packed"
    SHIPPED = "shipped"
    DELIVERED = "delivered"
    CANCELLED = "cancelled"

@dataclass
class DomainEvent:
    event_type: str
    aggregate_id: str
    data: Dict[str, Any]
    timestamp: datetime = field(default_factory=datetime.utcnow)
    version: int = 0

class EventStore:
    """Append-only store for domain events.

    In production, this would be backed by a database optimized for
    append-heavy workloads (PostgreSQL, EventStoreDB, or Kafka).
    """

    def __init__(self):
        self.events: Dict[str, List[DomainEvent]] = {}

    def append(self, event: DomainEvent):
        if event.aggregate_id not in self.events:
            self.events[event.aggregate_id] = []
        events = self.events[event.aggregate_id]
        event.version = len(events) + 1
        events.append(event)

    def get_events(self, aggregate_id: str,
                  after_version: int = 0) -> List[DomainEvent]:
        """Retrieve events, optionally after a specific version.

        Used for rebuilding state or catching up a projection.
        """
        events = self.events.get(aggregate_id, [])
        return [e for e in events if e.version > after_version]

class Order:
    """An event-sourced order aggregate.

    State is never stored directly – it's always derived by
    replaying events through the apply methods.
    """

```

```

"""

def __init__(self, order_id: str, event_store: EventStore):
    self.order_id = order_id
    self.store = event_store
    self._status = None
    self._items = []
    self._total = 0.0
    self._customer_id = None
    self._replay_events()

def _replay_events(self):
    """Rebuild current state from the event history."""
    for event in self.store.get_events(self.order_id):
        self._apply(event)

def _apply(self, event: DomainEvent):
    """Apply a single event to update internal state."""
    if event.event_type == "order_created":
        self._status = OrderStatus.CREATED
        self._customer_id = event.data["customer_id"]
        self._items = event.data["items"]
        self._total = event.data["total"]
    elif event.event_type == "payment_received":
        self._status = OrderStatus.PAID
    elif event.event_type == "order_shipped":
        self._status = OrderStatus.SHIPPED
    elif event.event_type == "order_cancelled":
        self._status = OrderStatus.CANCELLED

def create(self, customer_id: str, items: list, total: float):
    event = DomainEvent(
        event_type="order_created",
        aggregate_id=self.order_id,
        data={"customer_id": customer_id, "items": items, "total": total},
    )
    self.store.append(event)
    self._apply(event)

def pay(self, payment_id: str):
    if self._status != OrderStatus.CREATED:
        raise ValueError(f"Cannot pay order in status {self._status}")
    event = DomainEvent(
        event_type="payment_received",
        aggregate_id=self.order_id,
        data={"payment_id": payment_id},
    )
    self.store.append(event)
    self._apply(event)

```

Snapshots

Replaying thousands of events to rebuild state gets slow. **Snapshots** are periodic checkpoints: “As of event #500, the state was X.” To rebuild, load the latest snapshot and replay only events after it.

CQRS: Command Query Responsibility Segregation

CQRS pairs naturally with event sourcing. The idea: use different models for reading and writing.

- **Command side** (writes): Accepts commands (“place order”, “cancel order”), validates business rules, and emits events. Optimized for consistency.
- **Query side** (reads): Maintains one or more denormalized **projections** built from events. Optimized for fast queries.

```

class OrderReadModel:
    """A denormalized read model built from order events.

    This projection is optimized for the dashboard's needs:
    fast lookups by order ID, filtering by status, and
    listing orders by customer. It's eventually consistent
    with the event store – there's a small delay.
    """

    def __init__(self):
        self.orders = {} # order_id -> denormalized order dict

    def handle_event(self, event: DomainEvent):
        """Update the read model when a new event arrives."""
        if event.event_type == "order_created":
            self.orders[event.aggregate_id] = {
                "order_id": event.aggregate_id,
                "customer_id": event.data["customer_id"],
                "items": event.data["items"],
                "total": event.data["total"],
                "status": "created",
                "created_at": event.timestamp.isoformat(),
                "updated_at": event.timestamp.isoformat(),
            }
        elif event.event_type == "payment_received":
            if event.aggregate_id in self.orders:
                self.orders[event.aggregate_id]["status"] = "paid"
                self.orders[event.aggregate_id]["updated_at"] = (
                    event.timestamp.isoformat()
                )

    def get_order(self, order_id: str) -> dict:
        return self.orders.get(order_id)

    def get_orders_by_customer(self, customer_id: str) -> list:
        return [
            o for o in self.orders.values()
            if o["customer_id"] == customer_id
        ]

    def get_orders_by_status(self, status: str) -> list:
        return [o for o in self.orders.values() if o["status"] == status]

```

You can have multiple read models for different use cases: one optimized for the customer dashboard (orders by customer), one for the operations team (orders by status and date), one for analytics (aggregated metrics). All built from the same events.

Key Takeaway: CQRS makes the most sense when reads and writes have very different requirements. If your app is simple CRUD, CQRS adds complexity without benefit. If reads need denormalized data across multiple aggregates while writes need strict validation, CQRS shines.

Change Data Capture (CDC)

CDC captures every change (insert, update, delete) in a database and publishes it as an event. It's the bridge between your transactional database and the rest of your data infrastructure — without modifying application code.

Debezium is the leading open-source CDC platform. It reads the database's transaction log (WAL in PostgreSQL, binlog in MySQL) and publishes change events to Kafka.

Here's a Debezium connector configuration for capturing changes from a PostgreSQL database:

```
{
  "name": "postgres-orders-connector",
  "config": {
    "connector.class": "io.debezium.connector.postgresql.PostgresConnector",
    "database.hostname": "orders-db.internal",
    "database.port": "5432",
    "database.user": "debezium",
    "database.password": "${DEBEZIUM_PASSWORD}",
    "database.dbname": "orders",
    "database.server.name": "orders-db",
    "table.include.list": "public.orders,public.order_items",
    "plugin.name": "pgoutput",
    "slot.name": "debezium_orders",
    "publication.name": "dbz_publication",
    "topic.prefix": "cdc",
    "transforms": "route",
    "transforms.route.type": "org.apache.kafka.connect.transforms.RegexRouter",
    "transforms.route.regex": "([^.]+)\\.([^.]+)\\.([^.]+)",
    "transforms.route.replacement": "cdc.$3"
  }
}
```

This produces Kafka messages on topics like `cdc.orders` and `cdc.order_items`, each containing the before and after state of the changed row. Common CDC use cases:

- **Cache invalidation:** When a product's price changes in the database, invalidate the cache entry
- **Search index sync:** When a product is updated, update the Elasticsearch document
- **Cross-service sync:** When a customer updates their address in the Customer Service, the Shipping Service sees the change via CDC events
- **Data lake ingestion:** Capture all database changes for the analytics pipeline without querying the production database

API Design for Data Services

When microservices need to exchange data synchronously (not through events), API design determines the developer experience and system performance.

REST remains the default for most service-to-service communication. Keep it simple: resources as nouns (`/orders` , `/customers`), HTTP methods as verbs (GET, POST, PUT, DELETE), consistent error format.

GraphQL excels when clients need flexibility. A mobile app might need a subset of the data that a web app needs. Instead of building separate REST endpoints, GraphQL lets clients specify exactly what they want.

gRPC wins for performance-critical internal communication. Binary serialization (Protocol Buffers) is 3-10x faster than JSON. HTTP/2 multiplexing reduces connection overhead. Streaming support for real-time data flows.

For data-intensive APIs, pay special attention to:

- **Pagination:** Never return unbounded lists. Use cursor-based pagination for large datasets (more stable than offset-based when data is being inserted).
- **Filtering:** Let consumers filter at the API level rather than fetching everything and filtering client-side.
- **Rate limiting:** Protect your data services from runaway consumers. Return `429 Too Many Requests` with a `Retry-After` header.

Data Mesh Concepts

Data Mesh is an organizational and architectural approach that treats data as a product, owned by the domain teams that produce it.

The Problem It Solves

In traditional centralized data architectures, one data engineering team is responsible for ingesting, cleaning, and serving data from every domain in the company. This team becomes a bottleneck — they don't understand the business context of each domain, yet they're responsible for its data quality and modeling.

The Four Principles

1. **Domain-oriented data ownership:** The team that produces the data owns it. The Payments team owns payment data, not the data platform team. They're responsible for its quality, schema, and availability.
2. **Data as a product:** Each domain's data is treated like a product with consumers, SLAs, and documentation. It should be discoverable, addressable, trustworthy, and self-describing.
3. **Self-serve data infrastructure:** A platform team provides the tools and infrastructure that domain teams use to publish and consume data products. Think of it like Heroku for data — domain teams focus on their data, not on configuring Spark clusters.
4. **Federated computational governance:** Global standards (naming conventions, quality minimums, security requirements) that all domain teams follow, enforced by automation rather than central review.

When Data Mesh Makes Sense

Data Mesh is an organizational pattern, not a technology. It makes sense for large organizations (100+ engineers) with clearly separated domains and mature data engineering practices. For smaller teams, a centralized data team with good communication is simpler and more effective.

Key Takeaway: Data Mesh doesn't mean "no governance" or "every team for themselves." It means distributing ownership while maintaining standards. The platform team shifts from "building pipelines for everyone" to "building the platform that everyone uses to build their own pipelines."

Case Study: Migrating a Monolith to Event-Driven Microservices

A mid-sized e-commerce company has a monolithic Django application with a single PostgreSQL database. As the team grew to 40 engineers, deployments became painful — changing the order logic risked breaking the inventory system, and database migrations were terrifying.

The Strangler Fig Pattern

Instead of a risky "big bang" rewrite, they used the strangler fig pattern: gradually replacing pieces of the monolith with microservices, routing traffic to the new service as each piece is ready.

Phase 1 — Extract the first service (Month 1-2): The Notification Service is extracted first — it's the simplest, with clear boundaries. Events (`order.confirmed` , `shipment.dispatched`) are pub-

lished to Kafka by the monolith. The Notification Service consumes them and sends emails/SMS. The monolith's notification code is disabled.

Phase 2 — Extract the Inventory Service (Month 3-4): Inventory management moves to its own service with its own database. CDC (Debezium) captures inventory changes from the monolith's database and publishes them. The new Inventory Service consumes these events and builds its own state. Once validated, the monolith's inventory code is redirected to call the new Inventory Service's API.

Phase 3 — Extract the Order Service (Month 5-8): The most complex extraction. The Order Service gets its own event-sourced store. A saga orchestrator handles the distributed transaction that was previously a single database transaction: reserve inventory → charge payment → confirm order. If any step fails, compensating actions undo the previous steps.

Key lessons: - Start with the simplest, lowest-risk service - Run old and new systems in parallel and compare results before cutting over - Events are the glue — they decouple services and enable gradual migration - Accept eventual consistency where possible; use sagas where you need coordination - The migration took 8 months, not the 2 months originally estimated

Key Takeaway: Migrating to microservices is a marathon, not a sprint. Use the strangler fig pattern, start with low-risk services, and use events as the communication backbone. Plan for 2-4x your initial time estimate.

What's Next

Building distributed systems is only half the challenge — keeping them running is the other half. Module 10 covers monitoring, observability, and data quality: how to know when something is wrong, why it's wrong, and how to fix it before your users notice.

Module 10: Monitoring, Observability & Data Quality

Why Observability Matters

Building a system is the easy part. Keeping it running — and knowing when it's *not* running correctly — is where most engineering effort goes. The worst kind of failure isn't a crash; it's a system that keeps running while producing wrong results. A dashboard showing incorrect revenue for three weeks before anyone notices is far more damaging than an outage.

Monitoring tells you *something is wrong*: “CPU is at 95%,” “Error rate spiked,” “Pipeline failed.”

Observability tells you *why it's wrong*: “CPU is at 95% because this specific query is doing a full table scan due to a missing partition filter, triggered by a code change deployed two hours ago.”

The difference matters. Monitoring gives you alerts. Observability gives you answers.

Key Takeaway: Invest in observability before you think you need it. By the time you're debugging a production incident at 2 AM, it's too late to add instrumentation.

The Monitoring Pyramid

Think of monitoring as a pyramid with four levels. Each level builds on the one below it:

Level 1: Infrastructure Metrics (Foundation)

CPU, memory, disk I/O, network throughput. These are the vital signs of your servers. If a machine runs out of memory, everything on it fails.

Tools: Prometheus + Grafana, CloudWatch, Datadog.

Key metrics: CPU utilization (alert at 80%), memory usage (alert at 85%), disk space (alert at 80%), network errors, container restarts.

Level 2: Application Metrics

Request latency, error rates, throughput, queue depths. These tell you how your application is performing from the user's perspective.

The RED method: Rate (requests per second), Errors (failed requests per second), Duration (latency distribution — p50, p95, p99).

Key metrics: p99 latency (alert if >500ms), error rate (alert if >1%), request throughput (alert on sudden drops), connection pool utilization.

Level 3: Data Pipeline Metrics

Pipeline completion times, data freshness, row counts, schema changes. These are specific to data systems and often overlooked.

Key metrics: Pipeline duration (alert if >2x normal), data freshness (alert if table hasn't been updated in expected window), row count (alert on >20% deviation from normal), failed quality checks.

Level 4: Business Metrics (Top)

Revenue, conversion rates, user signups, churn. These are what actually matter to the business. Technical metrics are proxies; business metrics are the truth.

Key metrics: Vary by business, but examples include daily revenue (alert on >15% drop), conversion rate, active user count, order volume.

Key Takeaway: Most teams over-invest in Level 1 (infrastructure) and under-invest in Levels 3-4 (pipeline and business). A data quality bug that silently corrupts revenue numbers is more damaging than a CPU spike that triggers auto-scaling.

Data Quality Framework

Data quality has four levels, from basic to sophisticated:

Level 1: Syntax — Are the values in the right format?

- Data types are correct (dates are dates, numbers are numbers)
- Required fields are not null
- Values match expected patterns (email format, phone format)

Level 2: Semantic — Do the values make sense?

- Prices are positive
- Dates are in valid ranges (no orders from 1970 or 2099)

- Status values are from an expected set
- Foreign keys reference existing records

Level 3: Temporal — Is the data fresh and consistent over time?

- Tables are updated within expected windows
- Row counts don't fluctuate wildly day-to-day
- No gaps in time-series data
- Late-arriving data is handled correctly

Level 4: Contextual — Does the data match business expectations?

- Revenue totals are within expected ranges for the day/season
- Conversion rates haven't shifted suspiciously
- New data is consistent with historical patterns

Here's a practical data quality monitoring system that covers all four levels:

```

from dataclasses import dataclass
from datetime import datetime, timedelta
from typing import List, Optional
import statistics

@dataclass
class QualityAlert:
    level: str          # syntax, semantic, temporal, contextual
    severity: str        # warning, critical
    table: str
    check: str
    message: str
    timestamp: datetime

class DataQualityMonitor:
    """Monitor data quality across all four levels.

    Run this after every pipeline completion. If any critical check
    fails, block downstream consumption and alert the team.
    """

    def __init__(self, db_connection):
        self.db = db_connection
        self.alerts: List[QualityAlert] = []

    def run_all_checks(self, table: str, date: str) -> List[QualityAlert]:
        self.alerts = []
        self.check_syntax(table, date)
        self.check_semantic(table, date)
        self.check_temporal(table, date)
        self.check_contextual(table, date)
        return self.alerts

    def check_syntax(self, table: str, date: str):
        """Level 1: Are values in the right format?"""
        # Check for null primary keys
        null_count = self.db.execute(
            f"SELECT COUNT(*) FROM {table} "
            f"WHERE id IS NULL AND date = '{date}'"
        )
        if null_count > 0:
            self.alerts.append(QualityAlert(
                level="syntax", severity="critical", table=table,
                check="null_primary_key",
                message=f"{null_count} rows with null primary key",
                timestamp=datetime.utcnow(),
            ))

    def check_semantic(self, table: str, date: str):
        """Level 2: Do values make sense?"""
        # Check for negative revenue
        negative_count = self.db.execute(
            f"SELECT COUNT(*) FROM {table} "
            f"WHERE amount < 0 AND date = '{date}'"

```

```

    )
    if negative_count > 0:
        self.alerts.append(QualityAlert(
            level="semantic", severity="warning", table=table,
            check="negative_amounts",
            message=f"{negative_count} rows with negative amounts",
            timestamp=datetime.utcnow(),
        ))

def check_temporal(self, table: str, date: str):
    """Level 3: Is the data fresh and consistent over time?"""
    # Check row count vs historical average
    today_count = self.db.execute(
        f"SELECT COUNT(*) FROM {table} WHERE date = '{date}'"
    )
    historical_counts = self.db.execute(
        f"SELECT COUNT(*) as cnt FROM {table} "
        f"WHERE date >= '{date}':::date - INTERVAL '30 days' "
        f"AND date < '{date}' GROUP BY date"
    )
    if historical_counts:
        avg_count = statistics.mean(historical_counts)
        deviation = abs(today_count - avg_count) / avg_count
        if deviation > 0.5: # >50% deviation
            self.alerts.append(QualityAlert(
                level="temporal", severity="critical", table=table,
                check="row_count_anomaly",
                message=(
                    f"Row count {today_count} deviates {deviation:.0%} "
                    f"from 30-day average {avg_count:.0f}"
                ),
                timestamp=datetime.utcnow(),
            ))

def check_contextual(self, table: str, date: str):
    """Level 4: Does the data match business expectations?"""
    # Example: daily revenue should be within expected range
    daily_revenue = self.db.execute(
        f"SELECT SUM(amount) FROM {table} WHERE date = '{date}'"
    )
    # Revenue below $1,000 or above $10,000,000 is suspicious
    if daily_revenue < 1000 or daily_revenue > 10_000_000:
        self.alerts.append(QualityAlert(
            level="contextual", severity="critical", table=table,
            check="revenue_out_of_range",
            message=f"Daily revenue ${daily_revenue:,.2f} is outside expected
range",
            timestamp=datetime.utcnow(),
        ))

```

The Three Pillars of Observability

Metrics

Numeric measurements collected at regular intervals. “Request count per second,” “CPU usage percentage,” “Queue depth.”

Prometheus is the standard for collecting and querying metrics. It uses a pull model: Prometheus scrapes metrics endpoints that your services expose. **Grafana** visualizes Prometheus data as dashboards.

Logs

Detailed records of individual events. “User X requested /api/orders at 14:30:05, response 200, 45ms.”

Structured logging is essential — use JSON instead of plain text so logs can be searched and aggregated:

```

import logging
import json
from datetime import datetime

class StructuredLogger:
    """Structured JSON logger for data pipelines.

    Structured logs can be ingested by Elasticsearch, Loki, or
    CloudWatch Logs and queried by any field – much more useful
    than grep-ing through plain text.
    """

    def __init__(self, service_name: str):
        self.service = service_name
        self.logger = logging.getLogger(service_name)

    def log(self, level: str, message: str, **context):
        entry = {
            "timestamp": datetime.utcnow().isoformat(),
            "service": self.service,
            "level": level,
            "message": message,
            **context, # Add any context as top-level fields
        }
        self.logger.log(
            getattr(logging, level.upper()),
            json.dumps(entry)
        )

# Usage in a pipeline
log = StructuredLogger("etl-pipeline")

log.log("info", "Pipeline started",
        pipeline="daily_orders", run_date="2026-04-01")

log.log("info", "Extraction complete",
        pipeline="daily_orders", rows_extracted=50432,
        duration_seconds=12.5, source="postgres")

log.log("error", "Quality check failed",
        pipeline="daily_orders", check="null_primary_keys",
        failed_rows=3, severity="critical")

```

Traces

Distributed traces follow a single request across multiple services. When a user's API call touches the API gateway, the order service, the inventory service, and the database, a trace connects all the spans (individual operations) into one timeline.

OpenTelemetry is the standard for instrumentation. Here's how to instrument a data pipeline:

```

from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.exporter.otlp.proto.grpc.trace_exporter import OTLPSpanExporter

# Set up tracing
provider = TracerProvider()
provider.add_span_processor(
    BatchSpanProcessor(OTLPSpanExporter(endpoint="otel-collector:4317"))
)
trace.set_tracer_provider(provider)
tracer = trace.get_tracer("etl-pipeline")

def run_daily_pipeline(date: str):
    """A traced pipeline - each stage is a span in the trace.

    In Jaeger or Tempo, you'll see a timeline showing exactly
    how long each stage took and where bottlenecks are.
    """
    with tracer.start_as_current_span("daily_pipeline") as pipeline_span:
        pipeline_span.set_attribute("pipeline.date", date)

        with tracer.start_as_current_span("extract") as extract_span:
            rows = extract_data(date)
            extract_span.set_attribute("extract.row_count", rows)

        with tracer.start_as_current_span("transform"):
            transformed = transform_data(rows)

        with tracer.start_as_current_span("quality_check"):
            passed = run_quality_checks(transformed)
            if not passed:
                pipeline_span.set_status(
                    trace.Status(trace.StatusCode.ERROR, "Quality check failed")
                )
                raise Exception("Quality check failed")

        with tracer.start_as_current_span("load"):
            load_to_warehouse(transformed)

```

Key Takeaway: Use all three pillars together. Metrics tell you *something* is wrong (alert). Logs tell you *what* happened (investigate). Traces tell you *where* in the system it happened (pinpoint).

Data Lineage Tracking

Data lineage answers two critical questions:

Impact analysis (downstream): “If I change this table’s schema, what breaks?” If `raw.orders` feeds `staging.clean_orders`, which feeds `analytics.fact_orders`, which feeds the Revenue Dashboard — you need to know the full chain before modifying anything.

Root cause analysis (upstream): “This dashboard is showing wrong numbers. Where did the bad data come from?” Trace backward from the dashboard through every transformation to find where the corruption was introduced.

Tools: Apache Atlas, OpenLineage, Marquez, and dbt (which generates lineage from model dependencies).

Lineage should be captured automatically by your orchestration and transformation tools — not manually maintained, because manual lineage documentation goes stale immediately.

Anomaly Detection and Circuit Breakers

Circuit Breakers for Data Pipelines

Just as electrical circuit breakers prevent fires by cutting power when current spikes, **data circuit breakers** stop bad data from propagating through your system.

The pattern: if a quality check detects an anomaly (row count dropped 80%, null rate spiked, unexpected schema change), the pipeline halts and serves the *last known good* data while alerting the team.

This is far better than the alternative: loading bad data into the warehouse, where it corrupts dashboards, feeds wrong data to ML models, and triggers incorrect business decisions. Stale-but-correct data is almost always better than fresh-but-wrong data.

Cost Monitoring and Security Compliance

Cost Attribution

Track costs by team, pipeline, and query so you know who’s spending what:

```
-- Snowflake: query cost by user/team over the last 30 days
SELECT
  user_name,
  warehouse_name,
  COUNT(*) AS query_count,
  SUM(credits_used_cloud_services) AS total_credits,
  ROUND(SUM(credits_used_cloud_services) * 3.0, 2) AS estimated_cost_usd
FROM snowflake.account_usage.query_history
WHERE start_time > DATEADD('day', -30, CURRENT_TIMESTAMP())
GROUP BY user_name, warehouse_name
ORDER BY total_credits DESC;
```

Security and Compliance Monitoring

For regulated industries, monitoring must cover:

- **PII detection:** Automatically scan new data for personally identifiable information (names, emails, SSNs, credit card numbers). Flag unencrypted PII in non-approved tables.
- **GDPR compliance:** Track data lineage to know where personal data flows. Ensure deletion requests are fully honored across all systems.
- **SOX compliance:** Maintain audit trails for financial data. All changes to revenue-affecting data must be logged with who, what, and when.
- **Access auditing:** Log every query against sensitive tables. Alert on unusual access patterns (new user querying the full customers table at 3 AM).

Key Takeaway: Observability isn't just about performance and reliability — it's about trust. Data teams that invest in quality monitoring, lineage, and compliance build trust with stakeholders. Teams that don't eventually lose it, usually after a painful incident.

What's Next

Observability tells you how your system is behaving. Module 11 tackles the financial side: cost optimization and capacity planning. You'll learn how to keep cloud data platform costs under control while ensuring the system can handle growth.

Module 11: Cost Optimization & Capacity Planning

The Cost Problem

Here's a story that plays out at hundreds of companies: a startup launches their data platform on Snowflake. Month one costs \$2,000. Easy to ignore. The team adds more data sources, more dashboards, more analysts. A year later, the bill is \$50,000/month. Eighteen months in, it's \$200,000/month — growing faster than revenue. The CEO asks: "Why are we spending more on data infrastructure than on marketing?"

Cloud data platforms make it dangerously easy to spend money. There's no upfront sticker shock — just a smooth ramp that accelerates until someone notices. And because data platform costs are distributed across many teams, queries, and pipelines, nobody feels individually responsible.

Cost optimization isn't about being cheap. It's an engineering discipline — like performance optimization or security. It requires measurement, analysis, and deliberate trade-offs.

Key Takeaway: If you're not actively managing data platform costs, they will grow faster than your data. Treat cost as a first-class engineering concern, not an afterthought.

Cloud Pricing Models

Understanding pricing is the foundation of cost optimization. Each warehouse has a fundamentally different model.

Snowflake

Compute: Billed in "credits" per second of warehouse runtime. A warehouse size determines credits/hour:

SIZE	CREDITS/HOUR	APPROXIMATE \$/HOUR
X-Small	1	\$2-3
Small	2	\$4-6
Medium	4	\$8-12
Large	8	\$16-24
X-Large	16	\$32-48

Auto-suspend saves money when warehouses are idle. Auto-resume starts them on demand.

Storage: ~\$23/TB/month for active storage, less for time travel and fail-safe.

Key cost lever: Warehouse size and runtime. A Large warehouse running 24/7 costs ~\$12,000-17,000/month. The same warehouse auto-suspending after 1 minute of idle time might run only 8 hours/day: ~\$4,000-6,000/month.

BigQuery

On-Demand: \$5 per TB scanned. Simple, predictable per-query, but expensive at scale. A single query scanning 10 TB costs \$50.

Flat-Rate (Editions): Purchase reserved “slots” (units of compute). Starts around \$1,700/month for 100 slots. Better for predictable, heavy workloads. Queries don’t incur per-TB costs.

Storage: \$0.02/GB/month for active, \$0.01/GB/month for long-term (>90 days, automatic).

Key cost lever: For on-demand, query efficiency (scan less data). For flat-rate, slot utilization (are you using what you’re paying for?).

Amazon Redshift

Provisioned: Pay per node per hour. RA3 nodes (storage-separated) start ~\$3.26/node/hour. A 3-node cluster costs ~\$7,000/month.

Serverless: Pay per RPU-hour (Redshift Processing Unit). \$0.375/RPU-hour. Scales automatically.

Reserved Instances: 1-year or 3-year commitments for 30-75% discount on provisioned clusters.

Compute Cost Optimization

Auto-Scaling Strategies

Configure warehouses to scale based on demand rather than peak capacity:

```
-- Snowflake: Multi-cluster warehouse that scales with concurrency
CREATE WAREHOUSE analytics_wh
  WAREHOUSE_SIZE = 'MEDIUM'
  MIN_CLUSTER_COUNT = 1      -- Minimum 1 cluster
  MAX_CLUSTER_COUNT = 4      -- Scale up to 4 clusters
  SCALING_POLICY = 'STANDARD' -- Add clusters when queries queue
  AUTO_SUSPEND = 120         -- Suspend after 2 min idle
  AUTO_RESUME = TRUE;
```

Spot and Preemptible Instances

For Spark, Flink, and other batch processing frameworks, use spot instances for worker nodes:

- **AWS Spot:** Up to 90% discount. Use for Spark executors, EMR task nodes.
- **GCP Preemptible:** 60-91% discount. Use for Dataproc workers.
- **Azure Spot:** Up to 90% discount. Use for HDInsight, Databricks.

Best practice: Use on-demand instances for the driver/coordinator node (must not be interrupted) and spot instances for worker nodes (can be replaced if interrupted).

Workload Scheduling

Not all workloads need to run during business hours:

- **Night owl scheduling:** Run heavy batch jobs at night when competition for cloud resources is lower (often cheaper on shared infrastructure).
- **Weekend processing:** ML training, backfills, and reporting can run over the weekend.
- **Stagger pipelines:** Don't start all daily pipelines at the same time. Stagger them to smooth compute demand and avoid contention.

Storage Cost Optimization

Lifecycle Management

Data accessed daily costs 20x more per GB than data accessed yearly. Move data through storage tiers as it ages:

```
# AWS S3 lifecycle policy – automatically tier data
lifecycle_rules = {
    "Rules": [
        {
            "ID": "data-tiering",
            "Status": "Enabled",
            "Filter": {"Prefix": "analytics/"},
            "Transitions": [
                # After 30 days: Standard → Infrequent Access (~50% savings)
                {"Days": 30, "StorageClass": "STANDARD_IA"},
                # After 180 days: IA → Glacier Instant Retrieval (~70% savings)
                {"Days": 180, "StorageClass": "GLACIER_INSTANT_RETRIEVAL"},
                # After 365 days: Glacier → Deep Archive (~95% savings)
                {"Days": 365, "StorageClass": "DEEP_ARCHIVE"},
            ],
            "Expiration": {"Days": 2555}, # Delete after 7 years
        }
    ]
}
```

Format Optimization

The file format you choose has a massive impact on both storage cost and query cost:

FORMAT	1 TB RAW CSV	COMPRESSED PARQUET	SAVINGS
Storage	\$23/month	\$4-6/month	75-80%
BigQuery scan cost	\$5/query	\$0.50-1.50/query	70-90%

Converting from CSV/JSON to Parquet is often the single highest-ROI optimization for a data lake.

Data Retention Policies

Not all data needs to live forever. Define retention policies by data category:

- **Raw event data:** 90 days in hot storage, 1 year in cold, then delete (unless regulatory requirements say otherwise)
- **Aggregated metrics:** Keep indefinitely (small footprint)
- **Development/staging data:** 7-30 days maximum
- **Backups:** Follow the 3-2-1 rule (3 copies, 2 media types, 1 offsite), retain per compliance requirements

Query Cost Optimization

In pay-per-query systems like BigQuery on-demand, the most expensive part isn't storage — it's compute. A single bad query can cost more than a month of storage.

Identify Expensive Queries

```
-- BigQuery: Find your most expensive queries in the last 30 days
SELECT
  user_email,
  query,
  total_bytes_processed / POW(10, 12) AS tb_scanned,
  total_bytes_processed / POW(10, 12) * 5 AS estimated_cost_usd,
  creation_time
FROM `region-us`.INFORMATION_SCHEMA.JOBS_BY_PROJECT
WHERE creation_time > TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 30 DAY)
  AND job_type = 'QUERY'
  AND state = 'DONE'
ORDER BY total_bytes_processed DESC
LIMIT 20;
```

Result Caching

Both Snowflake and BigQuery automatically cache query results. If the same query runs again and the underlying data hasn't changed, the cached result is returned instantly at zero cost. Design your dashboards to benefit from this — use consistent query patterns rather than dynamic SQL with changing parameters.

Materialized Views

Pre-compute expensive aggregations so dashboards read from the materialized view (fast, cheap) instead of scanning raw data (slow, expensive):

```
-- Instead of this running every dashboard refresh (scans full table):
SELECT date, region, SUM(revenue) FROM orders GROUP BY 1, 2;

-- Create a materialized view (runs once, updated incrementally):
CREATE MATERIALIZED VIEW daily_revenue_by_region AS
SELECT
  DATE_TRUNC('day', order_date) AS date,
  region,
  SUM(revenue) AS total_revenue,
  COUNT(*) AS order_count
FROM orders
GROUP BY 1, 2;
```

Workload Management

Implement priority-based query queues to prevent expensive ad-hoc queries from starving time-sensitive dashboard queries:

- **Critical** (dashboards, production APIs): Highest priority, smallest queue delay, dedicated compute
- **High** (scheduled reports, ETL): High priority, moderate limits
- **Medium** (analyst ad-hoc queries): Medium priority, cost limits per query
- **Low** (data science experiments, backfills): Lowest priority, strictest cost limits, can be cancelled

Key Takeaway: The 80/20 rule applies: 20% of queries typically drive 80% of costs. Find and optimize those queries first — adding a partition filter, using a materialized view, or caching results can save thousands of dollars per month.

Capacity Planning

Capacity planning is predicting future resource needs so you can scale proactively rather than reactively (scrambling during an outage) or wastefully (paying for resources you don't use).

Growth Modeling

Model three scenarios for every major resource (storage, compute, cost):

Conservative (50% probability): Historical growth rate continues. No major new features or data sources.

Baseline (30% probability): Growth rate increases moderately. One or two new data sources. Moderate user growth.

Aggressive (15% probability): Rapid growth. New product launch. Major marketing push. 3-5x data volume increase.

Explosive (5% probability): Acquisition, viral moment, or regulatory change. 10x data volume. This scenario isn't likely but you need a plan for it.

For each scenario, project monthly storage, compute hours, and cost over the next 12-18 months. Compare against current capacity and budget.

When to Optimize vs. When to Add Capacity

- **Average utilization below 40%:** You're over-provisioned. Rightsize down.
- **Average utilization 40-70%:** The sweet spot. Monitor trends.

- **Average utilization 70-85%:** Start planning expansion. You'll hit limits during peak periods.
- **Average utilization above 85%:** You're likely experiencing performance degradation during peaks. Scale up now.

The goal isn't 100% utilization — that leaves no headroom for traffic spikes. Target 60-70% average with the ability to handle 2-3x peak loads.

The Capacity Planning Conversation

Capacity planning isn't a purely technical exercise. It requires input from:

- **Product managers:** What features are coming? Will they generate new data?
- **Sales teams:** What's the customer growth forecast?
- **Engineering teams:** Are there architecture changes that will affect resource usage?
- **Finance:** What's the budget? What's the cost of downtime vs. over-provisioning?

A single new feature (like adding real-time analytics or ML-powered recommendations) can change capacity needs more than six months of organic growth.

Case Study: TechFlow SaaS Cost Optimization

TechFlow, a B2B SaaS company, saw their data platform costs grow from \$20K/month to \$200K/month in 18 months as they scaled from 1,000 to 50,000 customers. They used three platforms: Snowflake (primary warehouse), BigQuery (ML workloads), and Redshift (legacy reporting).

The Analysis

They discovered:

- **40% of Snowflake costs** came from two dashboards that refreshed every 5 minutes, each running a full table scan. Changing the refresh to 15 minutes and adding materialized views cut those costs by 80%.
- **25% of costs** were from development and staging warehouses left running 24/7. Auto-suspend + scheduled scaling saved \$50K/month.
- **15% of storage costs** were from raw data that was never queried after initial processing. Lifecycle policies moved it to cold storage.
- **Multi-cloud overhead:** Running three platforms had 30% overhead from data duplication and cross-cloud transfer. They consolidated ML workloads to Snowflake, eliminating BigQuery and the associated data movement.

The Results

After a 90-day optimization sprint:

- Monthly cost: \$200K → \$130K (35% reduction)
- Query performance: 2x improvement (materialized views, better partitioning)
- Capacity headroom: Could handle 3x current load without proportional cost increase
- Ongoing governance: Automated cost alerts, per-team budgets, weekly cost review

Key Takeaway: Cost optimization isn't a one-time project — it's an ongoing discipline. Set up automated monitoring, per-team accountability, and regular reviews. The biggest savings usually come from a few high-impact changes, not hundreds of small ones.

What's Next

With the technical foundations, processing patterns, and operational practices covered, Module 12 brings it all together with deep-dive case studies of five real-world systems: Netflix, Uber, Spotify, Airbnb, and Twitter. You'll see how everything you've learned applies at massive scale.

Module 12: Case Studies — Real-World Systems

This module examines five systems operating at extraordinary scale. For each, we'll focus on the business problem, the architectural decisions, and the lessons you can apply to your own work.

12.1 Netflix: The Analytics Platform

The Business Challenge

Netflix serves 200+ million subscribers across 190 countries. Every interaction — every play, pause, search, scroll, and skip — generates events that feed recommendations, A/B tests, content decisions, and business intelligence. The platform processes **500 billion events per day**.

The core challenge: Netflix needs both real-time processing (personalize what you see *right now*) and batch processing (understand what content to license and produce). These have fundamentally different latency requirements, reliability needs, and computational characteristics.

Scale

- 500 billion events/day from 200M+ subscribers
- Real-time recommendations served in <100ms
- Thousands of A/B experiments running simultaneously
- Processing infrastructure spans multiple AWS regions

Architecture Overview

Netflix's architecture cleanly separates two data paths. The **real-time path** uses Mantis (their custom stream processing framework) and Keystone (their event collection pipeline) to process member interactions in milliseconds. When you open Netflix, your recommendations are generated in under 100ms based on your most recent behavior.

The **batch path** uses Spark on EMR, orchestrated by Genie (their job management system), to process the complete event history. This produces the business intelligence that drives content strategy — which shows to renew, what genres to invest in, which markets to expand to.

Events flow through Kafka, with regional clusters handling data residency (EU data stays in EU for GDPR compliance). Every event includes a schema ID from their Schema Registry, enabling safe evolution without breaking consumers.

Key Technical Innovation: The Experimentation Platform

Netflix's secret weapon isn't their recommendation algorithm — it's their ability to test *everything*. They run thousands of A/B experiments simultaneously, each with rigorous statistical controls.

Here's a simplified view of how experiment assignment works — the critical insight is that assignment is deterministic (same user always gets the same variant) using consistent hashing:

```
import hashlib

def assign_experiment_variant(member_id: str, experiment_id: str,
                             variants: list, traffic_split: list) -> str:
    """Deterministic experiment assignment using consistent hashing.

    The same member always gets the same variant for a given experiment,
    even across different servers and sessions. This is critical for
    accurate measurement – inconsistent assignment corrupts results.
    """
    hash_input = f"{member_id}:{experiment_id}".encode()
    hash_value = int(hashlib.sha256(hash_input).hexdigest(), 16)
    bucket = hash_value % 10000 # 10,000 buckets for 0.01% granularity

    cumulative = 0
    for variant, split in zip(variants, traffic_split):
        cumulative += split * 10000
        if bucket < cumulative:
            return variant
    return variants[-1] # Fallback to last variant
```

Every experiment has guardrail metrics (like churn rate) that automatically halt the experiment if they degrade, regardless of how well the primary metric is performing.

Lessons Learned

1. **Separate real-time and batch** — they have different requirements and shouldn't compromise each other
2. **Schema evolution is a first-class concern** — with thousands of producers and consumers, breaking changes are catastrophic
3. **Experimentation culture requires infrastructure** — the ability to test everything requires a platform that makes experiments easy, reliable, and statistically rigorous
4. **Build custom when off-the-shelf can't meet your requirements** — Netflix built Mantis because existing stream processors couldn't handle their scale and latency needs

12.2 Uber: Surge Pricing and Geospatial Processing

The Business Challenge

Uber operates a two-sided marketplace connecting riders and drivers across 10,000+ cities. The fundamental problem: supply (drivers) and demand (riders) are unevenly distributed and constantly shifting. Surge pricing is the mechanism that balances this marketplace in real-time — every 30 seconds.

Scale

- 15 million trips per day across 10,000+ cities
- 5 million driver location updates per minute (GPS every 5 seconds)
- Pricing decisions every 30 seconds per geographic cell
- Sub-100ms latency for pricing calculations

Architecture Overview

Uber's pricing system is built on their **H3 hexagonal grid** — an open-source library that divides the world into hexagonal cells at various resolutions. At resolution 9 (~0.5km hexagons), each city is divided into thousands of cells. For each cell, every 30 seconds, the system calculates supply (available drivers), demand (recent ride requests), and a surge multiplier.

Data flows through Kafka from rider apps and driver apps, processed by Flink for real-time aggregation. An ML demand forecasting model predicts demand 15-30 minutes ahead, enabling proactive driver positioning rather than reactive surge pricing.

Key Technical Innovation: H3 Hexagonal Grid

Why hexagons instead of squares? Hexagons have a unique property: every neighbor is equidistant from the center. In a square grid, diagonal neighbors are $\sqrt{2}$ times farther than edge neighbors. This matters for spatial queries like “find all drivers within 2km” — hexagons give consistent distance calculations in all directions.

```

import h3

def calculate_surge_for_cell(h3_cell: str, drivers: dict,
                           requests: list, config: dict) -> float:
    """Calculate surge multiplier for a single hexagonal cell.

    The algorithm balances three objectives:
    1. Minimize rider wait times (attract more drivers)
    2. Ensure fair driver earnings (incentivize driving)
    3. Maintain rider satisfaction (don't price out customers)
    """
    # Count available drivers in this cell and neighbors
    ring_cells = h3.k_ring(h3_cell, 2) # Cell + 2 rings of neighbors
    supply = sum(len(drivers.get(cell, [])) for cell in ring_cells)

    # Count recent demand (requests in last 5 minutes)
    recent_cutoff = time.time() - 300
    demand = len([r for r in requests if r['timestamp'] > recent_cutoff])

    if demand == 0:
        return 1.0 # No demand, no surge

    ratio = supply / demand

    # Supply/demand → surge multiplier (exponential curve)
    if ratio >= 2.0:
        # Excess supply – slight discount to attract riders
        return max(0.8, 1.0 - (ratio - 2.0) * 0.1)
    elif ratio >= 1.0:
        # Balanced – no surge
        return 1.0
    else:
        # Supply shortage – surge to attract drivers
        sensitivity = config.get('sensitivity', 2.0)
        max_surge = config.get('max_multiplier', 5.0)
        raw_surge = 1.0 / (ratio ** sensitivity)
        return min(raw_surge, max_surge)

```

The surge price is smoothed over time (exponential smoothing prevents jarring price jumps) and constrained by regulatory limits per city.

Lessons Learned

1. **Geospatial indexing is a foundational capability** — H3 enables consistent spatial queries at any scale
2. **Forecasting beats reacting** — predicting demand 15 minutes ahead is more valuable than optimizing surge after demand has already spiked
3. **Smoothing matters for user experience** — prices that change every 30 seconds feel chaotic; smoothed prices feel fair

4. **Local context matters** — surge parameters differ by city due to regulation, culture, and market dynamics
-

12.3 Spotify: The Recommendation Engine

The Business Challenge

Spotify has 400+ million users and 80+ million songs. The challenge isn't having enough content — it's matching each user with the right content at the right moment. Too much familiar music and users get bored. Too much new music and they feel lost. The balance between **exploitation** (play what you know you like) and **exploration** (discover something new) is Spotify's core technical challenge.

Scale

- 30+ billion events per day from 400M+ users
- 100,000 new songs added daily
- 2 billion recommendation requests per day
- Discover Weekly: 400 million personalized playlists generated weekly

Architecture Overview

Spotify's recommendation system maintains a real-time **taste profile** for every user. As you listen, skip, save, or add to playlists, your profile updates in real-time using exponential moving averages. The system tracks preferences across multiple dimensions: audio features (danceability, energy, tempo), genre weights, artist affinities, and even time-of-day preferences (you might prefer upbeat music during workouts and calm music at night).

A key design choice: **asymmetric learning rates**. The system learns from positive signals (completing a song, saving it) at a rate of 0.1, but learns from negative signals (skipping) at only 0.05. This prevents a single accidental skip from dramatically shifting your profile.

Key Technical Innovation: Multi-Armed Bandits for Strategy Selection

Spotify doesn't use a single recommendation algorithm. They maintain eight strategies (collaborative filtering, content-based, popularity-based, genre exploration, etc.) and use a **multi-armed bandit** to select which strategy to use for each recommendation slot.

The Upper Confidence Bound (UCB) algorithm balances exploiting the best-known strategy with exploring less-tried ones:

```

import math

class RecommendationBandit:
    """Multi-armed bandit for selecting recommendation strategies.

    Each strategy is an 'arm'. The bandit tracks the average reward
    (user engagement) for each strategy and explores less-tried ones
    to discover if they might be better.
    """

    def __init__(self, strategies: list):
        self.strategies = strategies
        self.pulls = {s: 0 for s in strategies}    # Times each was used
        self.rewards = {s: 0.0 for s in strategies} # Cumulative reward
        self.total_pulls = 0

    def select_strategy(self) -> str:
        """Select a strategy using Upper Confidence Bound (UCB).

        UCB = average_reward + confidence_bonus
        The confidence bonus is high for rarely-tried strategies,
        ensuring they get explored. It decreases with more pulls,
        so well-tested strategies are eventually chosen by merit.
        """
        self.total_pulls += 1

        # Ensure each strategy is tried at least 10 times
        for strategy in self.strategies:
            if self.pulls[strategy] < 10:
                return strategy

        # UCB selection
        best_strategy = None
        best_score = -1

        for strategy in self.strategies:
            avg_reward = self.rewards[strategy] / self.pulls[strategy]
            confidence = math.sqrt(
                2 * math.log(self.total_pulls) / self.pulls[strategy]
            )
            ucb_score = avg_reward + confidence

            if ucb_score > best_score:
                best_score = ucb_score
                best_strategy = strategy

        return best_strategy

    def record_feedback(self, strategy: str, reward: float):
        """Update strategy stats based on user engagement.

        Reward signals: completed song (+1.0), saved (+3.0),
        added to playlist (+2.5), shared (+4.0), skipped (-1.0).
        """

```

```
self.pulls[strategy] += 1
self.rewards[strategy] += reward
```

Discover Weekly generation adds an additional layer: **diversity optimization**. The system uses Maximal Marginal Relevance (MMR) to ensure playlists have variety in genre, artist, and energy level. No more than 2 songs per artist, and the energy arc follows a deliberate shape — moderate start, peak in the middle, gentle wind-down.

Lessons Learned

1. **Exploration is a feature, not a bug** — users value discovering new music, not just hearing favorites
2. **Asymmetric learning prevents over-correction** — learn faster from positive signals than negative ones
3. **Contextual recommendations outperform static ones** — what you want at 7 AM is different from 11 PM
4. **A/B test everything** — Spotify treats every feature as an experiment and measures engagement rigorously

12.4 Airbnb: Search and Discovery

The Business Challenge

Airbnb matches travelers with accommodations from 7+ million listings across 220 countries. Unlike hotel search (where listings are standardized), Airbnb listings are wildly diverse — a Tokyo capsule hotel, a Tuscan villa, a treehouse in Costa Rica. The search system needs to understand not just *where* you want to go, but *what kind of experience* you're looking for.

Scale

- 500 million searches per year
- 7+ million active listings across 220 countries
- <200ms search latency requirement
- 8% search-to-booking conversion rate

Architecture Overview

Airbnb's search system processes queries through multiple stages: **query understanding** (NLP to extract intent, dates, guest count, location), **candidate retrieval** (find relevant listings from the 7M

catalog), **personalized ranking** (score and order results based on user preferences), and **diversity optimization** (ensure varied results).

The key insight: travel booking is highly contextual. The same user wants different things for a business trip (WiFi, workspace, instant book) versus a family vacation (multiple bedrooms, safety features, kitchen) versus a romantic getaway (unique property, good reviews, scenic location). Airbnb classifies search intent using regex patterns and ML models, then adjusts ranking accordingly.

Key Technical Innovation: Multi-Signal Personalized Ranking

Airbnb's ranking considers far more than relevance to the query. It combines:

- **Preference match** (40% weight): Does this listing match the user's historical preferences (property type, price range, location type)?
- **Interaction signals** (30% weight): Has the user viewed, saved, or wishlisted similar listings? Have similar users booked this listing?
- **Segment score** (30% weight): Segment-specific features — business travelers value instant book and WiFi; families value capacity and safety; luxury seekers value ratings and uniqueness.

After scoring, a **diversity optimizer** using MMR ensures the top results aren't all the same type of listing. This is crucial — showing 20 similar apartments isn't as helpful as showing a mix of apartments, unique stays, and different neighborhoods.

Lessons Learned

1. **Intent matters more than keywords** — understanding *why* someone is searching unlocks better results
2. **Diversity is a feature** — homogeneous results feel limiting; varied results feel empowering
3. **Personalization requires restraint** — over-personalizing creates filter bubbles; Airbnb deliberately shows some unexpected options
4. **Multi-modal search is the future** — combining text, images, location, and behavior outperforms any single signal

12.5 Twitter: Real-Time Analytics

The Business Challenge

Twitter processes 500+ million tweets per day from 400+ million users. The system must detect trending topics within minutes, identify viral content within seconds, filter spam in real-time, and

generate personalized timelines for every user — all while handling 10x traffic spikes during breaking news events.

Scale

- 500 million tweets/day (25,000 peak tweets/second)
- 50 billion timeline requests per day
- 200 billion timeline generation requests per day
- Trending topics updated every 60 seconds

Architecture Overview

Twitter's architecture prioritizes speed over perfection. Their trending topic detection uses a **sliding window** approach: for each topic (hashtag or keyword), the system tracks mention frequency over a 15-minute window. The "trending" score isn't just frequency — it's **momentum**: how quickly the topic's mention rate is accelerating compared to its recent baseline.

A topic mentioned 10,000 times per hour isn't trending if it's always at that level. A topic that jumped from 100 to 5,000 mentions in 5 minutes is trending even though its absolute volume is lower.

Key Technical Innovation: Speed Over Perfection

Twitter's engineering philosophy is a masterclass in pragmatic trade-offs:

```

class TrendingTopicDetector:
    """Detect trending topics using frequency + momentum.

    The key insight: trending = acceleration, not volume.
    A topic with 10x growth in 5 minutes is more "trending"
    than a topic with 100x the volume but stable rate.
    """

    def __init__(self, window_minutes: int = 15):
        self.window = window_minutes * 60 # seconds
        self.topic_timestamps = {} # topic -> list of timestamps

    def record_mention(self, topic: str, timestamp: float):
        if topic not in self.topic_timestamps:
            self.topic_timestamps[topic] = []
        self.topic_timestamps[topic].append(timestamp)

    def get_trending(self, now: float, top_n: int = 50) -> list:
        """Calculate trending topics using frequency x momentum.

        Uses approximate algorithms – we accept ~90% accuracy
        for 10x lower latency. In a system processing 25K tweets/sec,
        exact counting would be too expensive.
        """

        cutoff = now - self.window
        scores = []

        for topic, timestamps in self.topic_timestamps.items():
            # Remove old mentions outside the window
            recent = [t for t in timestamps if t > cutoff]
            self.topic_timestamps[topic] = recent

            if len(recent) < 5:
                continue # Not enough signal

            # Split window: recent half vs earlier half
            midpoint = now - (self.window / 2)
            recent_half = len([t for t in recent if t > midpoint])
            earlier_half = len(recent) - recent_half

            # Momentum: ratio of recent to earlier activity
            if earlier_half > 0:
                momentum = recent_half / earlier_half
            else:
                momentum = recent_half # New topic, all recent

            # Score = recent volume x momentum
            score = recent_half * momentum
            scores.append({"topic": topic, "score": score,
                          "volume": len(recent), "momentum": momentum})

        # Return top N by score
        scores.sort(key=lambda x: x["score"], reverse=True)
        return scores[:top_n]

```

This algorithm uses approximate counting and simple arithmetic rather than sophisticated statistical models. It's not as accurate as a full probabilistic model, but it runs in milliseconds across thousands of topics — and at Twitter's scale, that trade-off is the right one.

Lessons Learned

1. **90% accurate in 100ms beats 99% accurate in 1 second** — for real-time systems, speed wins
 2. **Momentum matters more than volume** — acceleration is a better signal than absolute count
 3. **Design for spikes** — breaking news can 10x your traffic in minutes; your system must absorb this gracefully
 4. **Approximate algorithms are often sufficient** — HyperLogLog for cardinality, Count-Min Sketch for frequency, sampling for analytics
-

Cross-Company Patterns

Looking across all five case studies, several patterns emerge:

Event-driven architecture is universal. Every company uses Kafka (or equivalent) as the backbone for data flow. Events decouple producers from consumers and enable both real-time and batch processing.

Custom infrastructure at scale. Netflix built Mantis, Uber built H3, Twitter built custom trending detection. At extreme scale, off-the-shelf solutions hit limits. But most companies should exhaust existing tools before building custom ones — these companies have thousands of engineers maintaining their custom systems.

ML is deeply integrated into the data path. Recommendations, pricing, ranking, and content moderation all use ML models served in real-time with strict latency budgets. The feature store pattern appears at every company.

Experimentation culture requires infrastructure. Netflix, Spotify, and Airbnb all invest heavily in A/B testing platforms. Making experimentation easy and rigorous accelerates learning and product improvement.

Trade-offs are explicit and deliberate. Twitter chooses speed over accuracy. Netflix separates real-time from batch. Uber smooths prices for user experience. Every company makes trade-offs consciously — and documents why.

Key Takeaway: These systems weren't designed on a whiteboard and built in one go. They evolved over years, driven by real problems at real scale. The patterns they settled on — event-driven, separation of concerns, ML integration, experimentation — are battle-tested. Learn from their decisions, but always evaluate trade-offs in the context of *your* specific requirements and scale.

What's Next

Module 13 takes everything you've learned and prepares you for system design interviews and portfolio projects. You'll practice applying these concepts under time pressure and learn to communicate your design decisions clearly.

Module 13: Interview Preparation & Portfolio Projects

The Interview Mindset

After conducting 150+ system design interviews at three companies, I can tell you: the difference between candidates who get offers and those who don't is rarely technical knowledge. Most senior candidates know Kafka, know SQL, know the CAP theorem. The difference is **how they think and communicate**.

Wrong mindset: "I need to design the perfect system and impress the interviewer with cutting-edge technology."

Right mindset: "I need to understand the business problem, solve it effectively, and communicate my reasoning clearly."

The best candidates treat the interview as a **collaborative design session** — like a meeting with a colleague to whiteboard an architecture. They ask clarifying questions, state assumptions explicitly, explain trade-offs, and invite the interviewer into the decision-making process.

Key Takeaway: System design interviews test three things: (1) Can you break down ambiguous problems? (2) Can you design reasonable solutions? (3) Can you communicate your thinking? Technical depth is table stakes — communication and structured thinking are the differentiators.

The 45-Minute Structure

Most system design interviews are 45 minutes. Here's how to allocate that time:

Minutes 1-8: Requirements and Scale Estimation

This phase is critical and most candidates rush through it. Slow down. Ask questions. The interviewer deliberately leaves the problem ambiguous to see if you'll clarify before building.

Sample dialogue:

You: “Before I start designing, I’d like to understand the requirements. When you say ‘design a data warehouse for analytics,’ who are the primary users?”

Interviewer: “We have about 20 analysts, a few executives, and a data science team.”

You: “Got it. And what’s the data volume? Are we talking gigabytes or petabytes?”

Interviewer: “We process about 50,000 orders per day, so maybe a few terabytes total.”

You: “Okay, and for freshness — do stakeholders need real-time data, or is daily sufficient?”

Interviewer: “Daily is fine for most use cases, but the fraud team wants near-real-time.”

In 5 minutes, you’ve established: user count, data volume, freshness requirements, and identified a specialized need (fraud team). This shapes every subsequent decision.

Always estimate scale: “50,000 orders/day × 500 bytes ≈ 25 MB/day raw. Over 5 years, that’s ~45 GB. With dimension tables, transformations, and 3x overhead: ~150 GB total. This is well within a single PostgreSQL instance, but a cloud warehouse gives us elasticity and separation of concerns.”

Minutes 8-20: High-Level Architecture

Draw the major components and data flow. Keep it simple — 5-7 boxes connected by arrows. For each component, state the technology choice and a one-sentence justification.

```
[Data Sources] → [Ingestion (Airbyte)] → [Raw Storage (S3)]
                → [Transform (dbt on Snowflake)] → [Analytics Tables]
                                                    → [BI (Tableau)]
                                                    → [ML (Python)]
```

What to include: Data sources, ingestion method, storage, processing/transformation, serving layer, key data flows.

What NOT to include yet: Internal component details, specific configurations, error handling. Those come in the deep dive.

Minutes 20-35: Component Deep Dive

The interviewer will steer you toward the most interesting components. Pick 1-2 and go deep: data models, API design, algorithms, scaling strategies.

This is where your technical knowledge shows. If you’re asked to deep-dive on the data model, walk through your star schema design — fact tables, dimension tables, how SCD Type 2 handles customer changes. If you’re asked about the ingestion pipeline, discuss exactly-once delivery, schema evolution, and error handling.

Minutes 35-40: Scale and Optimize

Identify bottlenecks in your design and explain how to address them:

- “The transformation step is the bottleneck at 10x scale. I’d switch from a single dbt run to incremental models and parallelize independent transformations.”
- “At 100x the current query load, I’d add materialized views for common dashboard queries and implement a caching layer.”

Minutes 40-45: Trade-offs and Alternatives

End by acknowledging what you sacrificed and why:

- “I chose Snowflake over BigQuery because the team is multi-cloud, but BigQuery would be cheaper for sporadic query patterns.”
- “I went with batch processing for simplicity, but if the fraud team’s real-time requirement grows, I’d add a Kafka + Flink pipeline alongside the batch path.”

Advanced Interview Techniques

Technique 1: Assumption Surfacing

Vocalize your assumptions instead of making them silently:

“I’m assuming this is a read-heavy system — maybe 100:1 read-to-write ratio. Does that match your expectations?”

This shows structured thinking and gives the interviewer a chance to redirect you. Silent assumptions lead to wrong designs; vocalized assumptions lead to useful corrections.

Technique 2: Iterative Design

Start with the simplest architecture that works, then add complexity:

“For the initial design with 50K orders/day, I’d start with PostgreSQL and a daily cron job running Python scripts. As we scale to 500K orders/day, I’d move to Airflow for orchestration and Snowflake for the warehouse. At 5M orders/day, I’d add streaming with Kafka for the real-time use cases.”

This demonstrates that you understand when complexity is justified — which is far more impressive than jumping straight to a distributed architecture for a small-scale problem.

Technique 3: Failure Mode Analysis

Identify critical failure scenarios and your mitigation:

FAILURE SCENARIO	IMPACT	MITIGATION	RECOVERY TIME
Primary database down	No new data ingestion	Read replicas + auto-failover	<5 minutes
ETL pipeline fails	Stale dashboard data	Retry logic + alerting, serve last-known-good	<30 minutes
Data warehouse corruption	Wrong analytics	Point-in-time restore from immutable raw data	4-8 hours

Key Takeaway: These techniques — assumption surfacing, iterative design, failure mode analysis — separate candidates who “know the technology” from candidates who “think like architects.” Practice them until they’re natural.

Common Mistakes and How to Avoid Them

Mistake 1: Jumping to Technology Too Quickly

Bad: “I’d use Kafka, Spark, and Snowflake.” **Good:** “The first question is whether we need real-time processing. Given that daily freshness is acceptable for most users, I’d start with a simpler batch architecture. For the fraud team’s real-time needs, I’d add a focused streaming pipeline — Kafka for ingestion, Flink for processing — rather than making the entire system real-time.”

Mistake 2: Over-Engineering

Bad: 15 microservices, event sourcing, CQRS, and a custom ML platform for a system that processes 1 GB/day for 10 analysts.

Good: “At this scale — 1 GB/day, 10 users — the architecture should be simple: Airbyte for ingestion, dbt for transformation, Snowflake for warehousing, Tableau for visualization. Total cost: ~\$500/month. I’d invest complexity only where the business requires it.”

Mistake 3: Ignoring Business Context

Bad: Technically perfect system that costs 10x the budget or requires a team of 20 to operate when the company has 3 data engineers.

Good: “Given a 3-person team and a 6-month timeline, I’d choose managed services over self-hosted: BigQuery over self-managed Spark, Airbyte Cloud over custom connectors. The total cost is higher per unit, but the operational cost is dramatically lower.”

Mistake 4: Not Handling Ambiguity

Bad: Making assumptions without clarifying.

Good: “You mentioned ‘real-time analytics’ — could you clarify what that means here? Are we talking sub-second latency for fraud detection, or a dashboard that refreshes every few minutes? Those are very different architectures.”

Communication Frameworks

The Three-Level Explanation

When explaining a design decision, adjust depth based on your audience:

- **Executive summary:** “I’m choosing Kafka because it’s the most reliable way to handle our event volume without data loss.”
- **Engineering peer:** “I’m choosing Kafka over RabbitMQ because we need message replay for reprocessing, topic-based routing for multiple consumers, and the throughput handles our 50K events/second peak.”
- **Deep dive:** “I’d configure Kafka with 12 partitions per topic — matching our consumer parallelism — replication factor 3 for durability, and 7-day retention for reprocessing capability. We’d use Avro with Schema Registry for schema evolution.”

In an interview, start with the executive summary and go deeper based on interviewer interest.

The Assumption → Implication Pattern

Structure your reasoning as a chain:

“Given that we have 10x more reads than writes (assumption) → we need to optimize our read path (implication) → so I’m choosing a denormalized data model with pre-computed aggregations (decision) → which gives us sub-second dashboard queries but requires more complex write logic and 2x storage (trade-offs).”

Practice Questions with Solutions

Question 1: Design a Data Warehouse for E-Commerce

Setup: E-commerce company. 1M customers, 50K orders/day, sources include PostgreSQL, Stripe, Google Analytics, Zendesk.

Requirements phase: 20 analysts, 5 executives, 10 marketers. Daily freshness. 7-year retention for compliance. \$5K/month budget.

Scale estimation: 50K orders × 500 bytes = 25 MB/day raw. With all sources: ~700 MB/day. Over 7 years: ~1.8 TB. This is modest — a single cloud warehouse handles this easily.

Architecture: Sources → Airbyte (ingestion) → S3 raw layer → dbt (transform) → Snowflake (warehouse) → Tableau (dashboards). Airflow orchestrates everything.

Deep dive on data model: Star schema with `fact_orders` (one row per order line item, measures: quantity, revenue, discount, profit) joined to dimensions: `dim_customer` (SCD Type 2 for address changes), `dim_product` (category, brand, supplier), `dim_date` (fiscal calendar, holidays).

Scaling strategy: Incremental dbt models, materialized views for top 10 dashboard queries, Snowflake auto-suspend for cost control, resource monitors per team.

Question 2: Design a Real-Time Fraud Detection System

Setup: Payment processor. 10K transactions/second, decisions in <100ms, <2% false positive rate.

Architecture: Transaction → Kafka (partitioned by user_id) → Flink (feature enrichment) → ML Ensemble (gRPC) → Decision Engine → Response.

Latency budget: Kafka (15ms) + Feature lookup from Redis (5ms) + ML inference (30ms) + Rules engine (10ms) + Network (15ms) = 75ms. Under budget with headroom.

ML approach: Four-model ensemble — gradient boosting (tabular features), neural network (complex interactions), graph neural network (fraud rings), anomaly detection (novel patterns). Weighted voting with dynamic weights.

Key points to mention: Feature store for training/serving consistency, model A/B testing with gradual rollout, graceful degradation (rules-only if ML fails), PCI DSS compliance, real-time model monitoring for drift.

Question 3: Design a Content Recommendation System

Setup: Media platform. 10M users, 1M content items, real-time recommendations on homepage.

Architecture: User events (Kafka) → Feature pipeline (Spark) → Feature Store (Feast) → Training (weekly, MLflow) → Serving (FastAPI + Redis).

Algorithm: Collaborative filtering (users who watched X also watched Y) + content-based (similar genre/tags/duration) + popularity baseline. Multi-armed bandit to balance strategies.

Key points to mention: Cold start problem (new users: use popularity and demographics; new content: use content features and boost exploration), diversity in recommendations (MMR), A/B testing framework, latency budget (<50ms), offline evaluation (precision@k, recall@k, NDCG) vs online evaluation (click-through rate, watch time, retention).

Portfolio Projects

Build 2-3 projects that demonstrate system design thinking, not just coding ability. Below are detailed blueprints for each — follow them step by step, and you'll have portfolio pieces that stand out in any interview.

Project 1: Production-Grade Data Pipeline (3-4 weeks)

What it demonstrates: orchestration, data modeling, quality engineering, operational maturity.

The scenario: You're building a pipeline that ingests public weather data and NYC taxi trip records, transforms them into an analytics-ready star schema, and serves a dashboard showing how weather affects taxi demand. This mirrors what real data teams build daily.

Week 1: Set Up Infrastructure and Ingestion

Step 1 — Create your project structure:

```
weather-taxi-pipeline/
├── dags/                    # Airflow DAG definitions
│   └── weather_taxi_dag.py
├── dbt/                    # dbt transformation project
│   ├── models/
│   │   ├── staging/        # Clean raw data
│   │   ├── intermediate/   # Join and enrich
│   │   └── marts/          # Final star schema
│   ├── tests/              # Data quality tests
│   └── dbt_project.yml
├── scripts/
│   └── extract.py          # Extraction scripts
├── docker-compose.yml      # Local Airflow + Postgres
├── requirements.txt
└── README.md
```

Step 2 — Stand up local Airflow with Docker Compose. Use the official Apache Airflow Docker Compose file as a starting point. You need three services: the Airflow webserver, the scheduler, and a Postgres database that doubles as your local warehouse.

```
# docker-compose.yml (simplified – extend the official Airflow compose)
services:
  postgres:
    image: postgres:15
    environment:
      POSTGRES_DB: warehouse
      POSTGRES_USER: pipeline
      POSTGRES_PASSWORD: pipeline
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
```

Step 3 — Write extraction scripts. Pull data from two free public APIs:

- **Weather:** NOAA Climate Data Online (free API key) or Open-Meteo (no key required). Pull daily temperature, precipitation, and wind speed for New York City.
- **Taxi trips:** NYC Taxi & Limousine Commission publishes Parquet files on their website. Download one month of yellow taxi trip data (~3M rows).

```

# scripts/extract.py
import requests
import pandas as pd
from pathlib import Path

def extract_weather(start_date: str, end_date: str) -> pd.DataFrame:
    """Pull daily weather from Open-Meteo (free, no API key).

    Returns a DataFrame with columns: date, temperature_max,
    temperature_min, precipitation, wind_speed_max.
    """
    url = "https://archive-api.open-meteo.com/v1/archive"
    params = {
        "latitude": 40.7128,
        "longitude": -74.0060,
        "start_date": start_date,
        "end_date": end_date,
        "daily": "temperature_2m_max,temperature_2m_min,precipitation_sum,wind_speed_10m_max",
        "timezone": "America/New_York",
    }
    response = requests.get(url, params=params)
    response.raise_for_status()
    data = response.json()["daily"]
    return pd.DataFrame(data)

def extract_taxi_trips(year: int, month: int) -> pd.DataFrame:
    """Download NYC yellow taxi trip data (Parquet format).

    The TLC publishes monthly files. Each file is ~100MB
    with 2-3 million rows.
    """
    url = (
        f"https://d37ci6vzurychx.cloudfront.net/trip-data/"
        f"yellow_tripdata_{year}-{month:02d}.parquet"
    )
    return pd.read_parquet(url)

```

Step 4 — Load raw data into Postgres. Write a simple loader that creates `raw_weather` and `raw_taxi_trips` tables. Use `pandas.to_sql()` for simplicity, or `COPY` for performance. Don't transform anything yet — raw means raw.

Week 2: Build the Transformation Layer with dbt

Step 5 — Initialize dbt and connect to Postgres:

```

cd dbt/
dbt init weather_taxi --adapter postgres

```

Configure `profiles.yml` to point at your local Postgres.

Step 6 — Build staging models. These clean raw data without changing its grain (one row in = one row out):

```
-- dbt/models/staging/stg_weather.sql
-- Clean and rename columns from the raw weather extract.
-- One row per day for NYC.

SELECT
    time::date                AS weather_date,
    temperature_2m_max        AS temp_high_f,
    temperature_2m_min        AS temp_low_f,
    precipitation_sum          AS precipitation_inches,
    wind_speed_10m_max        AS wind_speed_max_mph,
    CASE
        WHEN precipitation_sum > 0.5 THEN 'rainy'
        WHEN temperature_2m_max < 32 THEN 'freezing'
        WHEN wind_speed_10m_max > 25 THEN 'windy'
        ELSE 'clear'
    END                       AS weather_category
FROM {{ source('raw', 'raw_weather') }}
```

```
-- dbt/models/staging/stg_taxi_trips.sql
-- Clean taxi trips: filter invalid records, normalize columns.
-- Drop rows with null pickup times or impossible fares.

SELECT
    tpep_pickup_datetime::date AS trip_date,
    tpep_pickup_datetime       AS pickup_at,
    tpep_dropoff_datetime      AS dropoff_at,
    passenger_count,
    trip_distance,
    fare_amount,
    tip_amount,
    total_amount,
    PULocationID               AS pickup_location_id,
    DOLocationID               AS dropoff_location_id
FROM {{ source('raw', 'raw_taxi_trips') }}
WHERE tpep_pickup_datetime IS NOT NULL
    AND fare_amount > 0
    AND trip_distance > 0
```

Step 7 — Build the star schema in the marts layer:

```
-- dbt/models/marts/fact_daily_trips.sql
-- Fact table: one row per day with aggregated trip metrics
-- joined to weather conditions. This is the table dashboards query.

SELECT
    t.trip_date,
    w.weather_category,
    w.temp_high_f,
    w.precipitation_inches,
    COUNT(*)                AS total_trips,
    AVG(t.trip_distance)    AS avg_distance,
    AVG(t.fare_amount)      AS avg_fare,
    SUM(t.total_amount)     AS total_revenue,
    AVG(t.tip_amount)       AS avg_tip
FROM {{ ref('stg_taxi_trips') }} t
LEFT JOIN {{ ref('stg_weather') }} w
    ON t.trip_date = w.weather_date
GROUP BY 1, 2, 3, 4
```

Step 8 — Add dbt tests for data quality:

```
# dbt/models/marts/schema.yml
models:
  - name: fact_daily_trips
    description: "Daily taxi trip aggregates joined with weather"
    columns:
      - name: trip_date
        tests:
          - not_null
          - unique
      - name: total_trips
        tests:
          - not_null
      - name: avg_fare
        tests:
          - not_null
          # Fares should be reasonable
          - dbt_utils.accepted_range:
              min_value: 5
              max_value: 200
```

Week 3: Orchestrate with Airflow and Add Monitoring

Step 9 — Write the Airflow DAG that ties everything together:

```

# dags/weather_taxi_dag.py
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.operators.bash import BashOperator
from datetime import datetime, timedelta

default_args = {
    "retries": 2,
    "retry_delay": timedelta(minutes=5),
}

with DAG(
    "weather_taxi_pipeline",
    default_args=default_args,
    schedule_interval="@daily",
    start_date=datetime(2024, 1, 1),
    catchup=False,
    tags=["portfolio", "weather", "taxi"],
) as dag:

    extract_weather = PythonOperator(
        task_id="extract_weather",
        python_callable=extract_weather_task, # Wraps extract.py
    )

    extract_taxi = PythonOperator(
        task_id="extract_taxi",
        python_callable=extract_taxi_task,
    )

    # dbt run handles staging → intermediate → marts
    dbt_run = BashOperator(
        task_id="dbt_run",
        bash_command="cd /opt/airflow/dbt && dbt run --profiles-dir .",
    )

    dbt_test = BashOperator(
        task_id="dbt_test",
        bash_command="cd /opt/airflow/dbt && dbt test --profiles-dir .",
    )

    # Extract in parallel, then transform, then test
    [extract_weather, extract_taxi] >> dbt_run >> dbt_test

```

Step 10 — Add monitoring and alerting. Configure Airflow email alerts on failure. Add a Slack webhook notification for pipeline completion. Log row counts at each stage so you can spot anomalies.

Week 4: Polish and Document

Step 11 — Build a simple dashboard. Use Streamlit (Python, quick to build) or Metabase (free, SQL-based). Show: trips per day colored by weather category, average fare by weather type, a heatmap of rainy days vs. trip volume.

Step 12 — Write your design document (see the Documentation section below).

Step 13 — Deploy to the cloud (optional but impressive). Push the pipeline to Astronomer (free trial), GCP Cloud Composer, or AWS MWAA. Use BigQuery or Snowflake (free trial) instead of Postgres. This shows you can operate in a real cloud environment.

Project 2: Real-Time Analytics Platform (4-6 weeks)

What it demonstrates: streaming architecture, multi-backend design, real-time processing.

The scenario: You're building a platform that ingests live Reddit posts from specific subreddits, analyzes sentiment in real time, and serves a live dashboard showing trending topics, sentiment shifts, and volume spikes. This mirrors what companies like Spotify and Twitter build for real-time content analytics.

Week 1: Set Up Kafka and Data Ingestion

Step 1 — Stand up Kafka locally with Docker Compose:

```
# docker-compose.yml
services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.5.0
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181

  kafka:
    image: confluentinc/cp-kafka:7.5.0
    ports:
      - "9092:9092"
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

  redis:
    image: redis:7
    ports:
      - "6379:6379"
```

Step 2 — Build a Reddit producer. Use the `praw` library (Reddit's official Python wrapper) to stream new posts and comments from chosen subreddits:

```

# producer/reddit_producer.py
import praw
import json
from kafka import KafkaProducer
from datetime import datetime

# Reddit API credentials (free – create an app at reddit.com/prefs/apps)
reddit = praw.Reddit(
    client_id="YOUR_CLIENT_ID",
    client_secret="YOUR_CLIENT_SECRET",
    user_agent="streaming-analytics:v1.0",
)

producer = KafkaProducer(
    bootstrap_servers="localhost:9092",
    value_serializer=lambda v: json.dumps(v).encode("utf-8"),
)

# Stream comments from multiple subreddits in real time
subreddits = reddit.subreddit("technology+programming+datascience")
for comment in subreddits.stream.comments(skip_existing=True):
    event = {
        "id": comment.id,
        "subreddit": comment.subreddit.display_name,
        "author": str(comment.author),
        "body": comment.body[:1000], # Truncate long comments
        "created_utc": comment.created_utc,
        "score": comment.score,
        "ingested_at": datetime.utcnow().isoformat(),
    }
    producer.send("reddit-comments", value=event)

```

Step 3 — Verify the topic is receiving data. Use the Kafka console consumer to see messages flowing: `kafka-console-consumer --bootstrap-server localhost:9092 --topic reddit-comments --from-beginning`

Week 2: Build the Stream Processor

Step 4 — Build a Faust stream processor (Python-native stream processing, simpler than Flink for a portfolio project):

```

# processor/stream_processor.py
import faust
from textblob import TextBlob
from datetime import datetime

app = faust.App(
    "reddit-analytics",
    broker="kafka://localhost:9092",
    store="rocksdb://", # Local state store for aggregations
)

# Input topic
comments_topic = app.topic("reddit-comments", value_type=bytes)

# Output topics
enriched_topic = app.topic("reddit-enriched", value_type=bytes)
alerts_topic = app.topic("reddit-alerts", value_type=bytes)

# Windowed table: track comment volume per subreddit (5-min windows)
volume_table = app.Table(
    "subreddit_volume",
    default=int,
).tumbling(300) # 5-minute tumbling window

@app.agent(comments_topic)
async def process_comments(comments):
    """Enrich each comment with sentiment analysis and
    update the per-subreddit volume counter.

    If volume spikes above 2x the recent average,
    emit an alert to the alerts topic.
    """
    async for event in comments:
        comment = json.loads(event)

        # Sentiment analysis using TextBlob
        blob = TextBlob(comment["body"])
        sentiment = blob.sentiment

        enriched = {
            **comment,
            "sentiment_polarity": sentiment.polarity, # -1.0 to 1.0
            "sentiment_subjectivity": sentiment.subjectivity,
            "sentiment_label": (
                "positive" if sentiment.polarity > 0.1
                else "negative" if sentiment.polarity < -0.1
                else "neutral"
            ),
            "word_count": len(comment["body"].split()),
            "processed_at": datetime.utcnow().isoformat(),
        }

        # Update volume counter
        subreddit = comment["subreddit"]

```

```

volume_table[subreddit] += 1

# Send enriched event downstream
await enriched_topic.send(value=json.dumps(enriched).encode())

```

Step 5 — If you want to use Spark Streaming instead (more widely used in industry but heavier to set up), replace Faust with PySpark:

```

# Alternative: Spark Structured Streaming
from pyspark.sql import SparkSession
from pyspark.sql.functions import from_json, col, udf
from pyspark.sql.types import StringType, FloatType

spark = SparkSession.builder \
    .appName("reddit-analytics") \
    .getOrCreate()

# Read from Kafka
raw_stream = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "reddit-comments") \
    .load()

# Parse JSON and add sentiment (via UDF)
@udf(returnType=FloatType())
def sentiment_score(text):
    from textblob import TextBlob
    return float(TextBlob(text).sentiment.polarity)

enriched = raw_stream \
    .select(from_json(col("value").cast("string"), schema).alias("data")) \
    .select("data.*") \
    .withColumn("sentiment", sentiment_score(col("body")))

# Write to console for debugging, or to another Kafka topic
enriched.writeStream \
    .outputMode("append") \
    .format("console") \
    .start() \
    .awaitTermination()

```

Week 3: Multi-Backend Storage

Step 6 — Write enriched data to three backends (this is the key architectural decision that shows multi-backend thinking):

- **Redis** — for real-time aggregations (last 5 minutes of sentiment by subreddit). Dashboards read from Redis for instant response.
- **Elasticsearch** — for full-text search over comments. Powers a “search comments by keyword” feature.

- **S3 or local Parquet files** — for historical analysis. Write hourly Parquet partitions for batch analytics.

```
# sinks/redis_sink.py
import redis
import json

r = redis.Redis(host="localhost", port=6379)

def write_to_redis(enriched_event: dict):
    """Update real-time aggregations in Redis.

    Stores: current 5-min average sentiment per subreddit,
    total comment count, and the last 100 comments.
    """
    subreddit = enriched_event["subreddit"]
    pipeline = r.pipeline()

    # Increment comment count
    pipeline.incr(f"count:{subreddit}")

    # Push to a capped list of recent comments
    pipeline.lpush(f"recent:{subreddit}", json.dumps(enriched_event))
    pipeline.ltrim(f"recent:{subreddit}", 0, 99)

    # Update running sentiment average (simplified)
    pipeline.lpush(f"sentiment:{subreddit}", enriched_event["sentiment_polarity"])
    pipeline.ltrim(f"sentiment:{subreddit}", 0, 299) # Keep last 300

    pipeline.execute()
```

Weeks 4-5: Dashboard and Polish

Step 7 — Build a live dashboard with Streamlit:

```

# dashboard/app.py
import streamlit as st
import redis
import json
import pandas as pd
import time

r = redis.Redis(host="localhost", port=6379)

st.title("Reddit Real-Time Sentiment Dashboard")

# Auto-refresh every 10 seconds
placeholder = st.empty()

while True:
    with placeholder.container():
        subreddits = ["technology", "programming", "datascience"]

        for sub in subreddits:
            count = int(r.get(f"count:{sub}") or 0)
            sentiments = r.lrange(f"sentiment:{sub}", 0, 299)
            if sentiments:
                avg_sentiment = sum(float(s) for s in sentiments) / len(sentiments)
            else:
                avg_sentiment = 0

            st.metric(
                label=f"r/{sub}",
                value=f"{count} comments",
                delta=f"Sentiment: {avg_sentiment:.2f}",
            )

        # Show recent comments
        st.subheader("Recent Comments")
        recent = r.lrange("recent:technology", 0, 9)
        for item in recent:
            comment = json.loads(item)
            emoji = "🟢" if comment["sentiment_polarity"] > 0.1 else "🔴" if comment["sentiment_polarity"] < -0.1 else "⚪"
            st.text(f"{emoji} [{comment['subreddit']}] {comment['body'][:120]}")

        time.sleep(10)
        st.rerun()

```

Step 8 — Add a Grafana dashboard showing Kafka consumer lag, processing latency, and throughput metrics. This proves you think about operational concerns, not just features.

Step 9 — Write your design document covering the architecture diagram, why you chose each storage backend, how you'd scale each component, and what breaks first under 100x load.

Project 3: ML-Powered Churn Prediction System (3-4 weeks)

What it demonstrates: ML engineering, feature engineering, model deployment, monitoring, and the full lifecycle from training to serving.

The scenario: You're building a system that predicts which users of a SaaS product will churn in the next 30 days, serves those predictions via an API, and monitors model performance over time. This is one of the most common ML applications in industry.

Week 1: Data Preparation and Feature Engineering

Step 1 — Get the data. Use the Telco Customer Churn dataset from Kaggle (free, ~7K rows) or generate synthetic data that mimics SaaS usage patterns:

```
# data/generate_synthetic.py
"""Generate synthetic SaaS user data for churn prediction.

Each row represents a user with their activity metrics
over the last 90 days and whether they churned.
"""

import pandas as pd
import numpy as np

np.random.seed(42)
n_users = 50_000

users = pd.DataFrame({
    "user_id": range(n_users),
    "signup_days_ago": np.random.randint(30, 730, n_users),
    "plan": np.random.choice(["free", "basic", "pro", "enterprise"],
                             n_users, p=[0.4, 0.3, 0.2, 0.1]),
    "logins_last_30d": np.random.poisson(12, n_users),
    "logins_last_7d": np.random.poisson(3, n_users),
    "features_used_last_30d": np.random.poisson(5, n_users),
    "support_tickets_last_90d": np.random.poisson(1, n_users),
    "api_calls_last_30d": np.random.poisson(50, n_users),
    "team_size": np.random.choice([1, 2, 5, 10, 25, 50], n_users,
                                   p=[0.3, 0.2, 0.2, 0.15, 0.1, 0.05]),
    "days_since_last_login": np.random.exponential(7, n_users).astype(int),
})

# Churn probability increases with inactivity, small teams, free plan
churn_score = (
    0.3 * (users["days_since_last_login"] > 14).astype(float)
    + 0.2 * (users["logins_last_30d"] < 5).astype(float)
    + 0.15 * (users["plan"] == "free").astype(float)
    + 0.1 * (users["team_size"] == 1).astype(float)
    + 0.1 * (users["support_tickets_last_90d"] > 3).astype(float)
    + np.random.normal(0, 0.15, n_users)
)
users["churned"] = (churn_score > 0.4).astype(int)

users.to_parquet("data/users.parquet", index=False)
print(f"Generated {n_users} users, churn rate: {users['churned'].mean():.1%}")
```

Step 2 — Build a feature store with Feast:

```
# feature_store/feature_definitions.py
"""Define features in Feast so training and serving use
the exact same feature computation – eliminating
training/serving skew.
"""

from feast import Entity, Feature, FeatureView, FileSource
from feast.types import Float32, Int32, String
from datetime import timedelta

# Entity: the user we're predicting churn for
user = Entity(name="user_id", join_keys=["user_id"])

# Source: our Parquet file (in production, this would be a warehouse)
user_source = FileSource(
    path="data/users.parquet",
    timestamp_field="event_timestamp",
)

# Feature view: the set of features available for this entity
user_activity_fv = FeatureView(
    name="user_activity",
    entities=[user],
    ttl=timedelta(days=1),
    schema=[
        Feature(name="logins_last_30d", dtype=Int32),
        Feature(name="logins_last_7d", dtype=Int32),
        Feature(name="features_used_last_30d", dtype=Int32),
        Feature(name="days_since_last_login", dtype=Int32),
        Feature(name="support_tickets_last_90d", dtype=Int32),
        Feature(name="api_calls_last_30d", dtype=Int32),
        Feature(name="team_size", dtype=Int32),
        Feature(name="plan", dtype=String),
    ],
    source=user_source,
)
```

Step 3 — Train the model with MLflow tracking:

```

# training/train.py
import mlflow
import mlflow.sklearn
import pandas as pd
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    precision_score, recall_score, f1_score, roc_auc_score
)
from sklearn.preprocessing import LabelEncoder

# Load data
df = pd.read_parquet("data/users.parquet")

# Encode categorical features
le = LabelEncoder()
df["plan_encoded"] = le.fit_transform(df["plan"])

feature_cols = [
    "logins_last_30d", "logins_last_7d", "features_used_last_30d",
    "days_since_last_login", "support_tickets_last_90d",
    "api_calls_last_30d", "team_size", "plan_encoded",
]

X = df[feature_cols]
y = df["churned"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Train with MLflow experiment tracking
mlflow.set_experiment("churn-prediction")

with mlflow.start_run(run_name="gradient_boosting_v1"):
    # Log parameters
    params = {
        "n_estimators": 200,
        "max_depth": 5,
        "learning_rate": 0.1,
        "min_samples_leaf": 20,
    }
    mlflow.log_params(params)

    # Train
    model = GradientBoostingClassifier(**params, random_state=42)
    model.fit(X_train, y_train)

    # Evaluate
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[:, 1]

    metrics = {
        "precision": precision_score(y_test, y_pred),
        "recall": recall_score(y_test, y_pred),
    }

```

```

        "f1": f1_score(y_test, y_pred),
        "auc_roc": roc_auc_score(y_test, y_prob),
    }
    mlflow.log_metrics(metrics)
    print(f"Metrics: {metrics}")

# Log the model – this saves it in MLflow's model registry
mlflow.sklearn.log_model(model, "churn_model")

# Log feature importance
importance = pd.DataFrame({
    "feature": feature_cols,
    "importance": model.feature_importances_,
}).sort_values("importance", ascending=False)
print(f"\nFeature importance:\n{importance}")

```

Week 2: Model Serving API

Step 4 — Build a FastAPI serving endpoint:

```

# serving/app.py
"""Serve churn predictions via REST API.

The API loads the latest model from MLflow's registry
and returns a churn probability for any user.
"""

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import mlflow.sklearn
import numpy as np

app = FastAPI(title="Churn Prediction API")

# Load the latest model from MLflow
model = mlflow.sklearn.load_model("models:/churn_model/production")

class PredictionRequest(BaseModel):
    logins_last_30d: int
    logins_last_7d: int
    features_used_last_30d: int
    days_since_last_login: int
    support_tickets_last_90d: int
    api_calls_last_30d: int
    team_size: int
    plan_encoded: int # 0=free, 1=basic, 2=pro, 3=enterprise

class PredictionResponse(BaseModel):
    churn_probability: float
    risk_level: str # low, medium, high
    top_risk_factors: list[str]

@app.post("/predict", response_model=PredictionResponse)
def predict_churn(request: PredictionRequest):
    features = np.array([
        request.logins_last_30d, request.logins_last_7d,
        request.features_used_last_30d, request.days_since_last_login,
        request.support_tickets_last_90d, request.api_calls_last_30d,
        request.team_size, request.plan_encoded,
    ])

    probability = float(model.predict_proba(features)[0, 1])

    # Determine risk level
    if probability > 0.7:
        risk = "high"
    elif probability > 0.3:
        risk = "medium"
    else:
        risk = "low"

    # Identify top risk factors by comparing to population averages
    risk_factors = []
    if request.days_since_last_login > 14:
        risk_factors.append("inactive_14_plus_days")

```

```

    if request.logins_last_30d < 5:
        risk_factors.append("low_engagement")
    if request.plan_encoded == 0:
        risk_factors.append("free_plan")
    if request.team_size == 1:
        risk_factors.append("single_user_account")

    return PredictionResponse(
        churn_probability=round(probability, 4),
        risk_level=risk,
        top_risk_factors=risk_factors,
    )

@app.get("/health")
def health():
    return {"status": "healthy", "model_version": "v1"}

```

Step 5 — Test the API locally:

```

# Start the server
uvicorn serving.app:app --reload --port 8000

# Test with curl
curl -X POST http://localhost:8000/predict \
-H "Content-Type: application/json" \
-d '{
    "logins_last_30d": 2,
    "logins_last_7d": 0,
    "features_used_last_30d": 1,
    "days_since_last_login": 21,
    "support_tickets_last_90d": 4,
    "api_calls_last_30d": 3,
    "team_size": 1,
    "plan_encoded": 0
}'

```

Week 3: Monitoring and Dashboard

Step 6 — Add model monitoring. Track prediction distributions over time to detect model drift:

```

# monitoring/monitor.py
"""Track model predictions over time.

Compares recent prediction distributions to the training
distribution. If they diverge significantly, the model
may need retraining (concept drift).
"""

from collections import deque
from datetime import datetime
import statistics

class ModelMonitor:
    def __init__(self, training_churn_rate: float = 0.25):
        self.predictions = deque(maxlen=10_000)
        self.training_churn_rate = training_churn_rate
        self.alerts = []

    def log_prediction(self, probability: float):
        self.predictions.append({
            "probability": probability,
            "timestamp": datetime.utcnow().isoformat(),
        })

    def check_drift(self) -> dict:
        """Compare recent prediction distribution to training baseline."""
        if len(self.predictions) < 100:
            return {"status": "insufficient_data"}

        recent_probs = [p["probability"] for p in self.predictions]
        current_churn_rate = sum(
            1 for p in recent_probs if p > 0.5
        ) / len(recent_probs)

        drift_magnitude = abs(current_churn_rate - self.training_churn_rate)

        status = "healthy"
        if drift_magnitude > 0.10:
            status = "warning"
            self.alerts.append(f"Churn rate shifted by {drift_magnitude:.1%}")
        if drift_magnitude > 0.20:
            status = "critical"
            self.alerts.append(f"Major drift detected: {drift_magnitude:.1%}")

        return {
            "status": status,
            "current_churn_rate": round(current_churn_rate, 4),
            "training_churn_rate": self.training_churn_rate,
            "drift_magnitude": round(drift_magnitude, 4),
            "sample_size": len(recent_probs),
            "mean_probability": round(statistics.mean(recent_probs), 4),
        }

```

Step 7 — Build a Streamlit monitoring dashboard that shows: prediction volume over time, churn probability distribution (histogram), drift metrics, top risk factors across all recent predictions, and model performance metrics from MLflow.

Step 8 — Write your design document and deploy. Containerize everything with Docker, push to a cloud provider, and document your architecture decisions.

Documentation Matters

For each project, write a design document that covers:

- **Problem statement:** What business problem does this solve? Who are the users? What's the current pain point?
- **Architecture diagram:** A clean diagram (use draw.io, Excalidraw, or even ASCII) showing every component and how data flows between them.
- **Technology choices and justifications:** For each component, state what you chose and why. "I chose Redis for real-time aggregations because it supports atomic increments and has sub-millisecond reads" is far better than "I used Redis."
- **Trade-offs:** What alternatives did you consider? Why did you reject them? "I considered Elasticsearch for real-time aggregations but chose Redis because our access pattern is key-value lookups, not full-text search. If we needed search, I'd add Elasticsearch as a secondary store."
- **Data model:** Show your schema designs with explanations. Include entity-relationship diagrams for relational data.
- **What breaks first at 10x scale:** Identify the bottleneck. "At 10x volume, the single Redis instance becomes the bottleneck. I'd shard by subreddit using Redis Cluster and add read replicas for the dashboard."
- **What I'd do differently:** Be honest about shortcuts you took and how you'd improve them. "I used TextBlob for sentiment analysis because it's simple, but in production I'd use a fine-tuned transformer model for much higher accuracy."

A well-documented project where you explain your decisions is far more impressive than a complex project with no documentation.

Career Development: Senior to Staff

The jump from senior to staff engineer isn't about deeper technical knowledge — it's about broader impact and different ways of thinking.

Senior engineers solve well-defined problems. “Build this feature,” “Fix this bug,” “Design this service.”

Staff engineers define which problems to solve. “Our data platform costs are growing faster than revenue — what’s the strategy?” “We’re expanding to 5 new markets — what infrastructure changes do we need?”

The key shifts:

1. **From implementation to architecture:** You spend less time coding and more time designing systems, writing design documents, and reviewing others’ designs.
2. **From single-team to cross-team:** You identify problems that span multiple teams and drive alignment on solutions.
3. **From technical to sociotechnical:** You consider not just “what’s the best technology?” but “what can our team actually operate?” and “how does this decision affect the organization?”
4. **From answering to asking:** The most valuable skill is asking the right questions. “What problem are we actually solving?” “What happens if we do nothing?” “What’s the simplest thing that could work?”

Key Takeaway: To grow toward staff engineer, practice thinking about systems in terms of business impact, organizational capability, and long-term evolution — not just technical elegance. Write design documents. Mentor others. Drive cross-team initiatives. The code you write matters less than the decisions you influence.

Conclusion

You’ve now traveled from the fundamentals of system design through storage engines, processing paradigms, data warehouses, lakehouses, ML platforms, distributed systems, observability, and cost optimization. You’ve studied how Netflix, Uber, Spotify, Airbnb, and Twitter solve problems at massive scale. And you’ve learned how to communicate your design thinking in interviews and career growth.

The most important thing to remember: **there are no perfect systems, only appropriate trade-offs.** Every technology, every architecture, every design decision involves giving something up to get something else. The mark of a great engineer is not knowing every technology — it’s knowing how to evaluate trade-offs in context, make a decision, and communicate why.

Build things. Break things. Learn from both. Good luck.